App for detecting music instruments in polyphonic recordings

Final Report

H. Chunder
22659511

Submitted as partial fulfilment of the requirements of Project EPR400
in the Department of Electrical, Electronic and Computer Engineering
University of Pretoria

November 2025

Study leader: Prof. J.P. Jacobs

Part 1. Preamble

1. Project proposal and technical documentation

This main report contains an unaltered copy of the approved Project Proposal (as Part 2 of the report).

All the code that I developed appears as a separate submission on the AMS.

2. Project history

This project was proposed by Prof. J. P. Jacobs as part of EPR402, with project title JPJ1. The work reported here is entirely my own.

3. Language editing

This document has been language edited by a knowledgeable person. By submitting this document in its present form, I declare that this is the written material that I wish to be examined on.

My language editor was _____.

_____ _____

Language editor signature *Date*

4. Plagiarism declaration

I, Harshad Chunder, understand what plagiarism is and have carefully studied the plagiarism policy of the University. I hereby declare that all the work described in this report is my own, except where explicitly indicated otherwise. Although I may have discussed the design and investigation with my study leader, fellow students or consulted various books, articles or the internet, the design/investigative work is my own. I have mastered the design and I have made all the required calculations in my lab book (and/or they are reflected in this report) to authenticate this. I am not presenting a complete solution of someone else or that I found somewhere.

Wherever I have used information from other sources, I have given credit by proper and complete referencing of the source material so that it can be clearly discerned what is my own work and what was quoted from other sources. I acknowledge that failure to comply with the instructions regarding referencing will regarded as plagiarism. If there is any doubt about the authenticity of my work, I understand that I may have to attend an oral ancillary examination/evaluation about the work.

5. AI usage declaration

The author confirms that no artificial intelligence tools, including language models or code-generation tools, were used in the preparation and writing, or any analysis that appears in this report. All content, design, explanations, data analysis, and code were developed solely by the author.

6. Software library usage declaration

Unless explicitly stated otherwise in the next paragraph, the author confirms that no software library functions, packages, or toolboxes were used in the design, implementation, or analysis presented in this report. All code and computations were developed entirely by the author

7. Unaltered Project Proposal

I certify that the Project Proposal appearing as the Introduction section of the report is an unaltered copy of the approved Project Proposal.

Signature

With my signature below, I hereby certify that all statements, declarations, and information presented above are complete.

H. Chunder	Date

TABLE OF CONTENTS

Part 3. Main Report

1. Literature study 1
    - 1.1. Principles of music 1
    - 1.2. The shift to deep learning 1
    - 1.3. Acoustic representation 2
    - 1.4. Deep neural network architectures 2
    - 1.5. Intended contribution 3

2. Approach 4
    - 2.1. Concept design 4
    - 2.2. Alternatives analysis 4

3. Design and implementation 6
    - 3.1. Design summary 6
    - 3.2. System block diagram 7
    - 3.3. Hardware design 7
    - 3.4. Dataset compilation 8
        - 3.4.1. Analysis of available data 8
        - 3.4.2. Data stratification & analysis 9
    - 3.5. Feature representation 11
        - 3.5.1. Analysis of Mel spectrogram, MFCCs & Constant Q Transform 11
        - 3.5.2. Constant Q Transform implementation & verification 13
    - 3.6. Convolutional neural network 16
        - 3.6.1. Analysis of ResNet-style architecture 16
        - 3.6.2. Analysis of VGG-style architecture 19
        - 3.6.3. Activation function 22
        - 3.6.4. Model architecture & design 24
        - 3.6.5. Forward pass implementation & verification 27
    - 3.7. Data augmentation 30
        - 3.7.1. Pitch shifting & Time warping 30
        - 3.7.2. Synthetic data generation 31
    - 3.8. Training pipeline 32
        - 3.8.1. Loss function implementation & verification 32
        - 3.8.2. Training optimizer implementation & verification 34
        - 3.8.3. Learning rate scheduler 37
        - 3.8.4. Regularization methods 38
        - 3.8.5. Hyperparameter optimization 40
    - 3.9. Onset detection 45
        - 3.9.1. Analysis of SuperFlux method 45
        - 3.9.2. Analysis of probability flux method & implementation 46
        - 3.9.3. Parameter optimisation 48

3.10. System deployment 48
3.10.1. Audio capture implementation 48
3.10.2. Analysis of recording domain 49

4. Results 51
4.1. Summary of results achieved 51
4.2. Qualification tests 52
4.2.1. Design of evaluation software 52
Qualification test 1: Macro F1-score
Objectives of the test or experiment 54
Equipment used 54
Test setup and experimental parameters 54
Steps followed in the test or experiment 54
Results or measurements 55
Observations 55
Qualification test 2: Macro precision
Objectives of the test or experiment 56
Equipment used 56
Test setup and experimental parameters 56
Steps followed in the test or experiment 56
Results or measurements 57
Observations 57
Qualification test 3: Macro recall
Objectives of the test or experiment 58
Equipment used 58
Test setup and experimental parameters 58
Steps followed in the test or experiment 58
Results or measurements 59
Observations 59
Qualification test 4: Total processing time
Objectives of the test or experiment 60
Equipment used 60
Test setup and experimental parameters 60
Steps followed in the test or experiment 60
Results or measurements 61
Observations 61

5. Discussion 62
5.1. Critical evaluation of the design 62
5.1.1. Interpretation of results 62
5.1.2. Critical evaluation 66
5.1.3. Unsolved problems 67
5.1.4. Strong points of the design 67
5.1.5. Expected failure conditions 68
5.2. Considerations in the design 68
5.2.1. Design ergonomics 68
5.2.2. Health and safety 69
5.2.3. Environmental impact 69
5.2.4. Social and legal impact 69

5.2.5. Ethics clearance 69

6. Conclusion 70
6.1. Summary of the work completed 70
6.2. Summary of observations and findings 70
6.3. Contribution 70
6.4. Suggestions for future work 70

7. References 71

Appendix

Ensemble definitions 75
Dataset statistics 76
Instrument feature representations 77
Hyperparameter optimisation 83

LIST OF ABBREVIATIONS

MIR	Music Information Retrieval
NMF	Non-Negative Matrix Factorisation
MFCC	Mel-Frequency Cepstral Coefficient
GMM	Gaussian Mixture Model
HMM	Hidden Markov Model
DNN	Deep Neural Network
RNN	Recurrent Neural Network
CNN	Convolutional Neural Network
CQT	Constant-Q Transform
STFT	Short-Time Fourier Transform
DCT	Discrete Cosine Transform
LSTM	Long Short-Term Memory
GRU	Gated Recurrent Unit
HCQT	Harmonic Constant-Q Transform
CRNN	Convolutional Recurrent Neural Network
RWC	Real World Computing
URMP	University of Rochester Multi-modal Music Performance
AIST	National Institute of Advanced Industrial Science and Technology
DFT	Discrete Fourier Transform
FFT	Fast Fourier Transform
SIMD	Single Instruction Multiple Data
CPU	Central Processing Unit
FLOPs	Floating Point Operations
ReLU	Rectified Linear Unit
PReLU	Parametric Rectified Linear Unit
ELU	Exponential Linear Unit
2D	Two-dimensional
BCE	Binary Cross-Entropy
TPE	Tree-structured Parzen Estimator
GP	Gaussian Process
ALSA	Advanced Linux Sound Architecture
PCM	Pulse Code Modulation
TP	True Positive
FP	False Positive
SDFP	Spurious Detection False Positive
ICFP	Instrument Confusion False Positive
TEFP	Timing Error False Positive
FN	False Negative
TN	True Negative
ADSR	Attack, Decay, Sustain, Release

Part 3. Main Report

1. Literature study

To elucidate on the design of the completed system, the principles of music, acoustic representation and the shift to deep learning in music information retrieval (MIR) will be reviewed.

1.1 Principles of music

Western music primarily uses the 12-note chromatic scale, consisting of seven natural notes (A, B, C, D, E, F, G) and five accidentals, being the sharps and flats of certain notes. Each of the seven natural notes has distinctive pitches and can be played at varying levels of loudness. Pitch and loudness are subjective concepts in the field of music; however, they are closely related to frequency and amplitude [1]. Polyphony refers to a musical piece which contains two or more simultaneous lines of independent melody. Polyphonic musical pieces are referred to as ensembles, with the number of independent instruments representing the size of the ensemble. Ensemble size can vary from duets to orchestras. Timbre is the subjective quality of sound or the perceived "colour" of sound; whilst pitch and loudness are measurable qualities of sound, timbre is a complex, multi-dimensional phenomenon that is difficult to quantify yet is used to distinguish different instruments that may be playing the same note at the same loudness simultaneously.

1.2 The shift to deep learning

Prior to the adoption of deep learning methods, common approaches for instrument detection in polyphonic recordings included classical signal processing techniques or conventional machine learning classifiers. Classical signal processing techniques aimed to simplify the polyphonic signal by means of source separation techniques or engineered acoustic features. A commonly employed source separation technique is non-negative matrix factorisation (NMF), which decomposes magnitude time-frequency responses of the polyphonic signal into two matrices. The first matrix represents the basic spectra of individual sound components, and the second matrix represents the time-varying activations of these sound components [2]. As dataset sizes increased, the need to reduce dimensionality, in an effort to reduce computation time and storage usage, was emphasised.

The Mel-frequency cepstral coefficients (MFCCs), a technique commonly used in speech-recognition applications, have also been used as a means to engineer acoustic features. These coefficients represent the short-term power spectrum of a sound and are transformed onto the non-linear mel scale of pitch, which approximates human auditory response. These reduced feature vectors are typically used as input to traditional classifiers, such as Gaussian mixture models (GMMs) [3] and hidden Markov models (HMMs) [4]. The performance of these classifiers is heavily dependent on the quality of the engineered features – there exists the risk that the engineered features were effective for specific, controlled datasets but are unable to generalise when exposed to different datasets. Another limitation was that the engineered features captured the superposition of the spectral behaviour of all instruments present in the polyphonic mix, which made it difficult for traditional classifiers to distinguish the constituent sound sources.

With the introduction of deep neural networks (DNNs), the need for manual feature engineering and human expertise was eliminated. The primary advantage of DNNs was their ability to learn hierarchical feature representations directly from the data [5]. Convolutional neural networks

and recurrent neural networks (RNNs) have demonstrated superior performance to traditional classifiers using engineered features. The standard practice when using DNNs involves converting the raw polyphonic audio into a two-dimensional time-frequency representation, such as the Mel-spectrogram or constant Q transform (CQT).

1.3 Acoustic representation

The Mel spectrogram is one of the simplest, yet widely adopted acoustic representation techniques in MIR. It is a two-dimensional representation of the spectrum of frequencies in an audio signal as a product of time, where the frequency axis is scaled according to the Mel scale - a quasi-logarithmic scale designed to mimic the non-linear manner in which humans perceive sound. In addition, human perception of loudness is closer to a logarithmic scale than a linear one. This transformation makes the representation biologically plausible. MFCCs are designed to capture the shape of the spectral envelope of the audio signal, which is highly relevant to timbre perception. MFCCs are derived from five consecutive processes: signal framing, computation of the power spectrum, application of a Mel filter bank, calculation of the logarithmic values of all filter banks, and application of the discrete cosine transform (DCT) [6]. The final step decorrelates the filter bank outputs and compacts the information into a set of coefficients. However, this compression inherently involves a degree of information loss, which powerful deep learning models might otherwise learn directly from richer, less processed representations. The CQT transforms a time-domain audio signal into the frequency domain in a similar fashion to the short-time Fourier transform (STFT); however, the frequency bins are logarithmically spaced. This logarithmic spacing is advantageous for musical representation as musical notes are organised logarithmically on the Western musical scale; an increment in the octave number represents a doubling of frequency. A defining characteristic of the CQT is the "constant Q-factor," which maintains the ratio of each filter's centre frequency to its bandwidth. This property results in higher frequency resolution at lower frequencies and better temporal resolution at higher frequencies, a characteristic that mirrors the human auditory system and directly enhances the representation of musical timbre. This geometric spacing between frequency bins proves advantageous when contrasted with the linear spacing of the STFT, as the fundamental frequency of several instruments resides in the low-frequency range, and very seldom is there a need for high accuracy at high-frequency ranges. The harmonics of musical notes form a characteristic pattern in the CQT domain. As the fundamental frequency of a note changes, the relative positions of the note harmonics remain constant within the CQT representation. This pitch-invariant harmonic representation optimises the learning task of a DNN by providing a consistent "fingerprint" of an instrument's timbre, irrespective of the specific note being played.

1.4 Deep neural network architectures

The task of polyphonic instrument recognition is most appropriately framed as a multi-label classification problem, as the aim is to detect all instruments active in a mix rather than a predominant instrument [7]. The CNN is designed to automatically learn features of input data through a hierarchical learning process [8]. The architecture of a CNN typically features convolutional layers for local pattern detection, pooling layers for dimensionality reduction and translation invariance and fully connected layers for final classification. Pattern detection is done by sliding a kernel filter across the input data; the degree to which the kernel is slid is

referred to as the stride. CNNs may learn hierarchical spatial-temporal patterns directly from spectrograms, by treating audio as image segments, which allows the network to automatically discover discriminative features across both time and frequency axes. Within the context of this project, this allows a CNN to learn specific harmonic structures, attack transients or spectral envelopes. Attention mechanisms may also be employed on the final feature map. This allows the model to prioritise certain regions which are critical for determining the probability of a label. For example, the cello typically spans from C2 (65.4 Hz) to A5 (880 Hz), while the violin covers a range from G3 (196 Hz) to E7 (2637 Hz) – an attention mechanism can be employed to prioritize the lower region of the feature map to detect the presence of cello, and another may be employed to focus on the mid-to-lower region of the feature map to detect the presence of violin. The RNN is designed to capture temporal patterns rather than spatial patterns by processing the input data in a sequential manner. RNNs possess an internal memory, referred to as hidden states, providing feedback from previous inputs in the sequence. Traditional RNNs, however, often struggle with learning long-range temporal patterns due to the vanishing gradient problem. Advanced variants such as long short-term memory (LSTM) networks [9] and gated recurrent units (GRUs) were introduced to overcome this limitation.

1.5 Intended contribution

The aforementioned literature demonstrates a clear trajectory from manual feature engineering with shallow classifiers to state-of-the-art deep learning methods, particularly using CNNs, RNNs or hybrid models. The intended contribution is to develop a multi-label instrument recognition model that makes use of a CNN architecture with the harmonic constant-Q transform (HCQT) as the primary input representation. The model will also detect musical instruments with improved temporal resolution compared to previous work.

2. Approach

The objective of the project is to determine the presence of an instrument and the time at which it enters the mix if it is present. Thus, the problem can be divided into two sub-problems: instrument activity detection and instrument onset detection. As it is possible to have several instruments present at any given moment in time within polyphonic music, the task of instrument activity detection is best posed as a frame-wise multi-label classification problem.

2.1 Concept design

As discussed in Sections 1.2 and 1.4, deep learning has demonstrated state-of-the-art performance for instrument activity-related tasks, particularly with the use of CNNs for hierarchical learning. An onset event is defined as the time when an instrument enters the mix for the first time. The concept design is fairly straightforward. A four-step process of acquiring an audio signal, performing feature representation to highlight the time-frequency characteristics of the instruments present, passing the feature representation in a patch-wise manner to a CNN, which yields instrument activity predictions in a frame-wise manner, and performing a means of post-processing to determine the instrument onset times. The distinction between a patch and a frame is as follows: a patch refers to a collection of frames, and a frame refers to a segment of an audio signal whereby the statistical properties of the signal remain constant. A flow diagram of the concept design is illustrated in Figure 1 below:

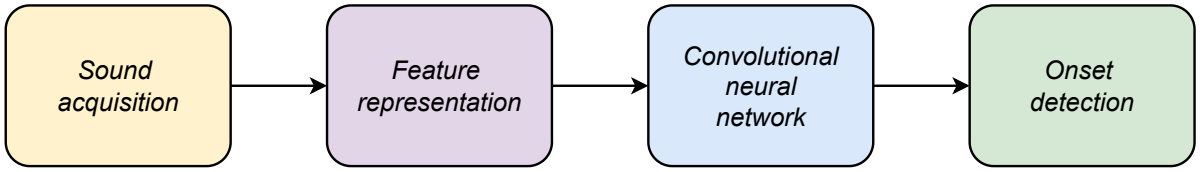


Figure 1. Flow diagram of concept design.

The Mel-spectrogram, MFCC and HCQT are considered candidate feature representation techniques. A ResNet-style and a VGG-style are considered candidate architectures for the CNN. The performance of each CNN architecture and feature representation technique is evaluated to determine the best-performing combination of feature representation and CNN architecture. The custom C implementation of the selected feature representation technique and CNN architecture is also verified against state-of-the-art library implementations. For the task of onset detection, the superflux algorithm and post-processing techniques using the frame-wise predictions of the CNN are considered. Performance of the onset detection sub-system is coupled to the performance of the activity detection subsystem; thus, selection is made based on computational feasibility on the embedded platform.

2.2 Alternatives analysis

As discussed in section 1.2, a widely adopted signal processing-based approach is the usage of NMF for source separation. The primary limitation of NMF lies in the assumption that each instrument can be represented by a single spectral template. There is no guarantee that a single

template would be able to account for pitch variations, amplitude variations and playing-style variations. Usage of NMF would require a sizeable dictionary of templates to model the different pitch, loudness and playing-style variations of each of the 11 instruments. NMF also requires that the number of basis vectors be pre-defined. The algorithm would then attempt to find the templates for each instrument, when it would be highly improbable that all 11 instruments would be active simultaneously. This would introduce significant redundant computations and computational complexity.

Alternatives to a CNN for processing image-like data include an RNN and a convolutional recurrent neural network (CRNN). RNNs process image-like data as a sequence of feature vectors rather than a two-dimensional grid of data. This is ideal for music-related tasks as the temporal dependencies between frames may be modelled. Memory of these temporal dependencies can be affected by the vanishing gradient problem. While this can be overcome with the usage of LSTM structures, they introduce significant parameter count and computational cost to the model. RNNs also do not contain spatial awareness and may not be able to model two-dimensional time-frequency patterns, such as the harmonic relationship of instruments from the partials present. On the other hand, CRNNs provide the benefit of spatial awareness and temporal modelling of the input features. A CNN-like backbone is used to extract features from the image-like data, with an RNN head to then sequence the data. The CRNN presents the ideal solution if performance were the only design consideration; however, CRNNs are highly complex structures, which are computationally expensive - a model complex enough to learn hierarchical features would not be suitable for deployment on an embedded platform.

3. Design and implementation

3.1 Design summary

Deliverable or task	Implementation	Completion of deliverable or task, and section in the report
Implement audio frequency-time analysis techniques for model input representation from first principles.	The CQT was implemented from first principles in C and verified against a state-of-the-art library method.	Completed Section 3.5.2
Design and train a multi-label classifier for instruments in polyphonic recordings.	A multi-head VGG-style convolutional neural network was designed and trained with the hyperparameters optimised.	Completed Section 3.6 Sections 3.8.4 - 3.8.5
Implement the model architecture, forward pass, loss function, optimiser and scheduler algorithms from first principles.	The model architecture and forward pass were implemented in C, from first principles, and verified against the PyTorch implementation in Python. A first-principles approach of the optimiser was also completed. An implementation of the learning rate scheduler was not needed, however, the fundamental theory is presented.	Completed Section 3.6.5 Sections 3.8.1 - 3.8.3
Optimisation of the application software for deployment on an embedded platform.	Multi-processing, single instruction multiple data (SIMD) operations and loop tiling were used in the implementation of the CQT and model forward pass.	Completed Section 3.5.2 Section 3.6.5
Design and implementation of an audio capture pipeline, including analogue to digital conversion, from first principles.	An audio capture pipeline was implemented in C, using a library for hardware interfacing between the Linux kernel and the microphone.	Completed Section 3.10.1

Table 1. Design Summary

3.2 System block diagram

The system block diagram of the project is shown in Figure 2 below:

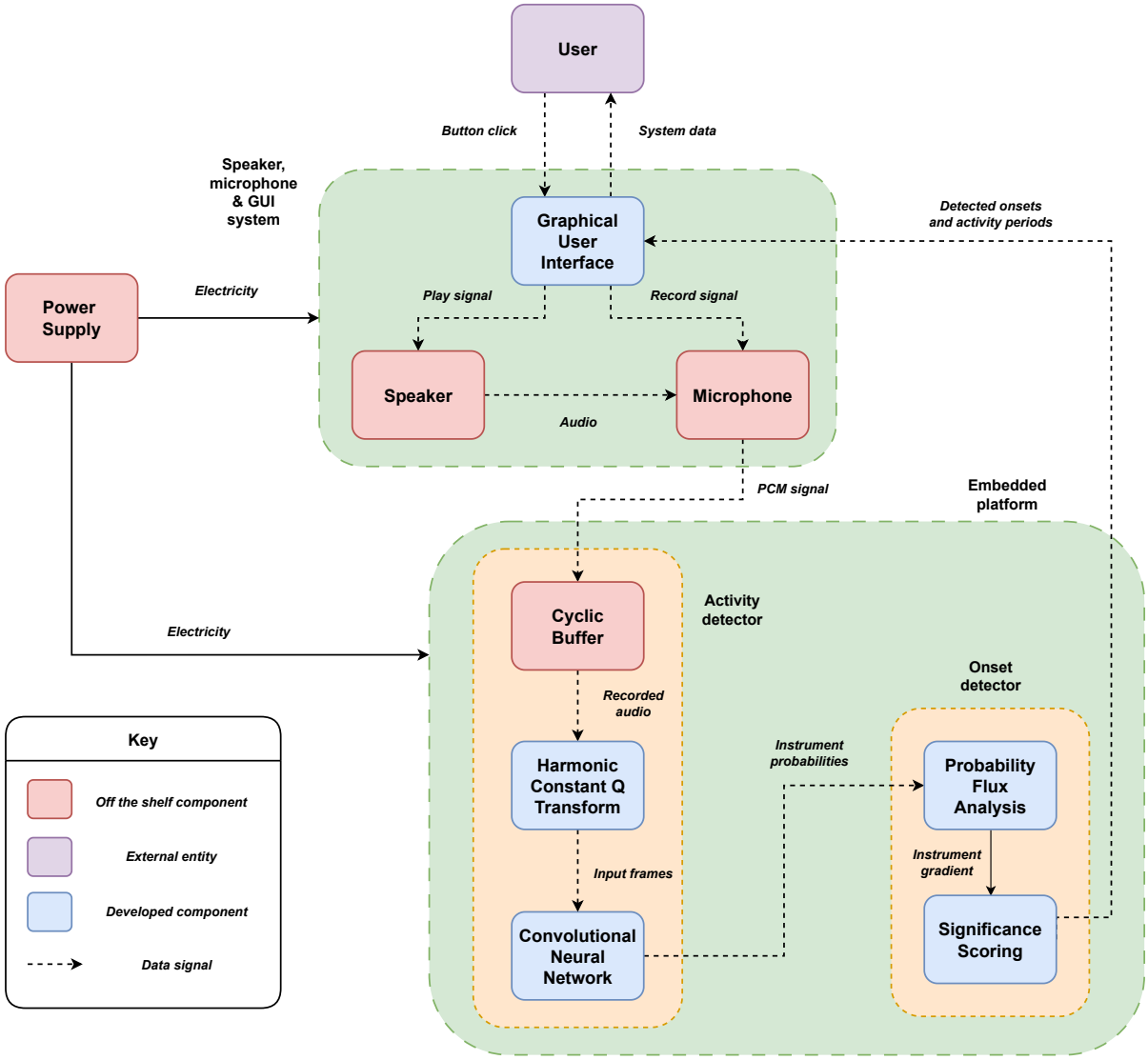


Figure 2. System block diagram.

3.3 Hardware design

The core objective of the project is the development of an app, thus, there is no first-principles hardware design elements. This section provides a trade-off analysis for each of the selected off-the-shelf components.

For audio playback, the key selection criteria is that the playback device reproduce the audio as accurately as possible. Consumer grade hi-fi speakers are often designed to alter the frequency response by boosting treble frequencies suppress mid-range frequencies. Studio monitor speakers

are designed to have a flat frequency response. The PreSonus Eris 3.5 and Mackie CR3-X are candidate speakers, with both having a frequency range of 80 Hz to 20 kHz. At the time of purchase, the PreSonus Eris 3.5 retailed for R2 500 and the Mackie CR3-X at R3 500, hence the PreSonus was selected. For audio capture, a key selection criteria is that the frequency characteristics of the audio be preserved. The Samson C01U Pro USB microphone and Audio-Technica AT2020 USB are candidate speakers, with both have a frequency range of 20 Hz to 20 kHz. The frequency response of the Audio-Technica AT2020 USB is flatter than that of the Samson C01 Pro, however, at double the price. At the time of purchase, the Samson C01U Pro retailed for R1 500 and the Audio-Technica AT2020 USB at R3 725, hence the Samson was chosen. For the embedded platform, the key selection criteria is the processing capabilities of the on-board CPU. The NVIDIA Jetson Nano features a quad-core ARM A57 with a 1.43 GHz clock speed, and the BeagleBone AI-64 a dual-core ARM cortex-A72 with a clock speed of 2.0 GHz. Cost of both embedded platforms was similar at time of purchase. The BeagleBone relies on Texas' Instruments toolchains which do not have the online support as the NVIDIA toolchains. Using the NVIDIA toolchain would allow for a simpler development workflow, hence, the NVIDIA Jetson Nano was chosen.

3.4 Dataset compilation

3.4.1 Analysis of available data

There exist several candidate datasets for the use of instrument detection of polyphonic recordings within the field of MIR. These candidates include: IRMAS, OpenMIC-2018, RWC (Real World Computing) Music, MusicNet and URMP (University of Rochester Multi-modal Music Performance).

The IRMAS dataset [10] is a dataset created for the task of predominant instrument recognition within multi-instrument audio clips. The instruments covered are cello, clarinet, flute, acoustic guitar, electric guitar, organ, piano, saxophone, trumpet, violin, and human singing voice. The annotations of this dataset are considered weak, as they merely indicate the name of the predominant instrument, with no labelling of exact onset times. The OpenMic-2018 dataset [11] is a large dataset containing 20,000 ten-second audio clips for 20 different instruments. It is a multi-genre dataset containing music from classical, experimental, pop and electronic music. Similar to the IRMAS dataset, the annotations of this dataset are weak. The RWC Music dataset [12] is a comprehensive multi-genre dataset that includes collections of popular music, jazz, classical music, and royalty-free music. This dataset is by far the largest available dataset, such that there is an instrument-specific subset of data. The dataset annotations are strong, containing the beat structure, the chorus sections, the melody line and the active instrument. However, this instrument-specific subset only contains 50 classical ensembles. Crucially, this dataset must be purchased from the National Institute of Advanced Industrial Science and Technology (AIST) in Japan. The MedleyDB V2 dataset [13] consists of 122 multitrack audio files, with an additional 69 songs available that contain melody-only annotations. There is great benefit in multitrack recordings, as they allow for training on the track stems and the final mixed stereo track. Annotations are of high quality - containing the instrument activation, the pitch, melody and audio genre. The database is also multi-genre, containing rock, pop, jazz and classical. Despite being a strong candidate, there is a limited classical repertoire within the 122 audio files. The MusicNet

dataset [14] is a large-scale dataset that contains 330 freely-licensed classical musical recordings from Johann Christian Bach, Carl Philipp Emanuel Bach, Beethoven, Brahms, Dvorak, Faure, Haydn, Mozart, Schubert, and Schumann catalogues. Audio files do not contain the stems; however, they are paired with fine-grained note-level annotations derived from associated MIDI files for each track. The audio annotations include the start time, end time, instrument, note pitch, and position in the musical score for every single note. The dataset contains 11 instruments: piano, violin, viola, cello, contrabass, clarinet, bassoon, horn, flute, oboe, and harpsichord. The URMP dataset [15] contains 44 recordings of polyphonic classical musical pieces, with ensembles varying from solos to quintets. Similar to MedleyDB V2, it provides the audio stems for each instrument present in the final mix. Annotations are of high quality, containing note pitch, start time, end time, and active instrument. Similar to the MusicNet dataset, the focus is on chamber music.

The application must detect instrument onsets; thus, the annotations need to indicate when an instrument enters a mix. This can be achieved with the note-level annotations of MedleyDB V2, MusicNet and URMP. Considering that the scope of classification is polyphonic classical music, the scope of valid candidates narrows down to MusicNet and URMP. Thus, the dataset used for the project is a combination of MusicNet and URMP.

3.4.2 Data stratification & analysis

Common training, validation, and testing splits range from 80/10/10 to 60/20/20 proportions of the total number of data samples. Stratification was done using a 70/15/15 split, as it presents a middle ground of the two extremes: there is sufficient data to train the model within the training set without overfitting, while also ensuring sufficient data to tune hyperparameters within the validation set and sufficient data to provide an unbiased real-world evaluation of generalisation capability within the testing set. Stratification began by isolating all polyphonic recordings within the compiled dataset and keeping the solo files for minority classes. The duration statistics of each instrument present were calculated by iterating through all annotation files, calculating the difference between all start and end time labels per instrument, and then subtracting all overlapping periods to account for instances where an instrument is playing multiple notes at once. The stratification algorithm grouped all tracks into their respective ensembles and randomly allocated tracks to each data subset. The scope of the project ranges from duets to quintets; thus, only ensembles within this range were allocated to the validation and testing set. Once all tracks had been allocated, an optimisation loop swapped tracks between subsets to minimise the difference between total instrument duration and target instrument duration. The instrument duration statistics, ensemble statistics and onset distribution of the resultant dataset stratification are shown in Tables 25, 26 and 27 in section 6.4 of the Appendix. To obtain a better understanding of the dataset, an instrument co-occurrence matrix and instrument against ensemble distribution matrix were plotted, shown in Figures 3 and 4 on the subsequent page.

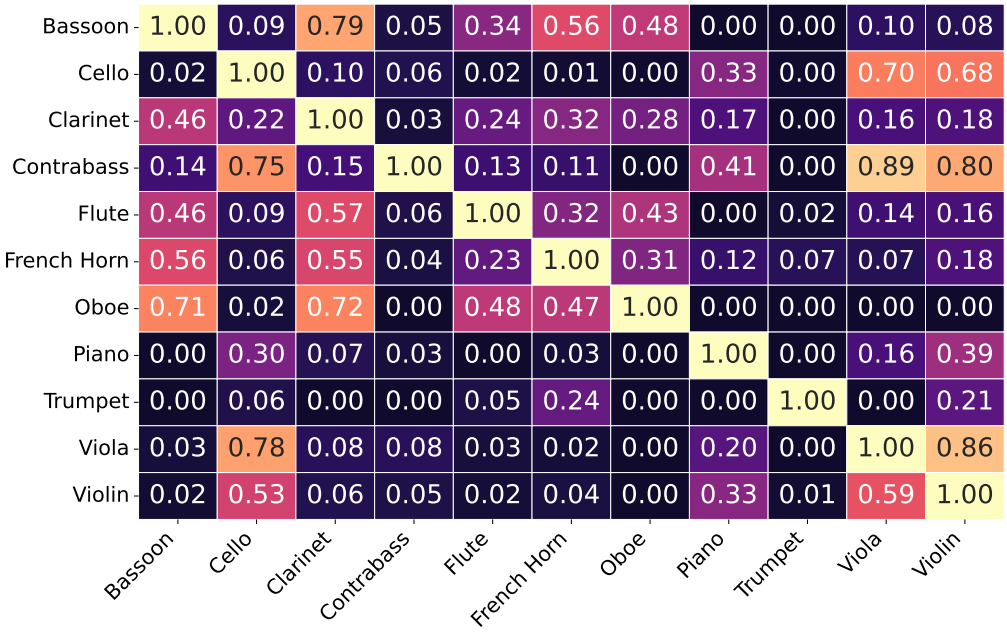


Figure 3. Instrument co-occurrence matrix.

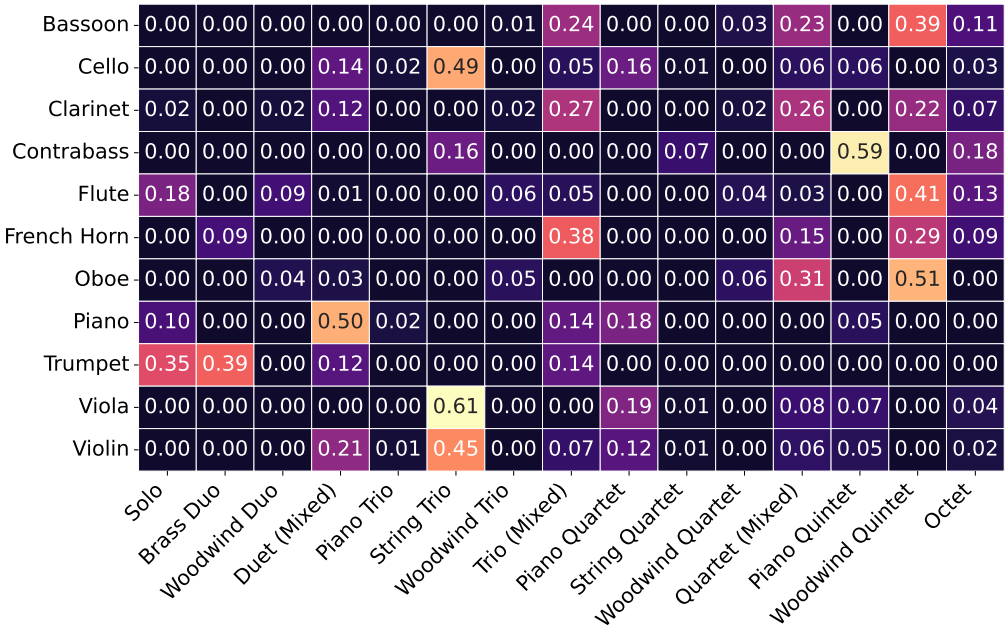


Figure 4. Instrument vs. ensemble distribution matrix.

The formal definitions of the ensembles can be found in section 6.4 of the Appendix.

Evident from Table 25, a primary constraint is the significant label imbalance among instruments: the total duration of the majority classes, violin and piano, is 15 times that of the minority classes, contrabass and trumpet. This disparity carries negative implications. Any deep learning model trained on this data will be biased toward the majority class, as it would be the easiest path to minimising the cost function. The model will also not be exposed to a diverse set of

examples to learn complex features, such as varying playing styles (muting and hand-stopping for trumpet and bowing, arco, plucking and pizzicato for contrabass). Lastly, there is potential for potentially misleading results - a lack of test samples could present either an unfair assessment that is too difficult for the model, or an assessment where the model gets "lucky." Whilst a technical challenge, this imbalance provides an accurate representation of instrument occurrence in classical chamber music. The violin has been a principal instrument since the Baroque era, and the piano has a significant solo repertoire and is often the centrepiece in piano-based ensembles such as piano trios. The contrabass is primarily an orchestral instrument, which typically features in larger ensembles with very limited roles. It is also not a staple piece of string-based ensembles. In a similar light, the trumpet can be considered an orchestral instrument, albeit with a stronger presence in classical chamber music.

From Figure 3, a high co-occurrence between instruments of the string family - an example being 78% of viola's playing time is accompanied by cello and 86% with violin. This co-occurrence is a result of common string-based ensembles where viola, violin and cello often play simultaneously to create harmony and texture. From Figure 4, it can be seen that 49% of cello activity occurs within string trios and an additional 16% within piano quartets. The same can be said for the co-occurrence of woodwind instruments - flute, oboe, bassoon and clarinet show strong inter-correlation. From Figure 4, it can be seen that 51% of oboe activity occurs in woodwind quartets. The French horn, a wind-based brass instrument, has high co-occurrence with the woodwind instruments - more than 50% of French horn activity occurs with bassoon and clarinet. The implication of such co-occurrence biases is that the model will consistently be exposed to examples where these instruments are present together - there are relatively limited examples where it would be exposed to instruments in isolation, so that it may learn features unique to each instrument. Thus, the model is likely to be biased toward predicting that several instruments are present when exposed to string or woodwind behaviour, as it may struggle to distinguish instrument-specific features. This constraint presents a data trade-off; the inclusion of more string-based solos may assist in teaching the model instrument-specific features, however, worsening the disparity between the majority and minority classes, which do not contain solo ensembles. In this event, the model would likely become an expert for that specific instrument family. The piano, being the lone instrument within the percussion family, has a strong presence across all ensemble sizes. This aligns with the historical importance of the piano, being one of the most significant instruments, taking the role as a solo instrument or an accompaniment.

3.5 Feature representation

3.5.1 Analysis of Mel spectrogram, MFCCs & Constant Q Transform

The Mel spectrogram plots the spectrum of frequencies within a signal as they vary with time. The Mel scale used is a modified frequency axis used to mimic human pitch perception, making it useful for audio and speech processing tasks. To compute the Mel spectrogram of a signal, it is first segmented into smaller frames - a frame refers to a segment of the signal whereby the statistical properties of the signal are constant, typically segments of < 30 ms. A window function is applied to each frame to smooth out the edges and reduce spectral leakage. The square of the discrete Fourier transform (DFT) of each frame is then computed to determine the power spectrum of the frequencies present. The power spectrum is passed through a Mel filter bank,

which converts the linearly-spaced frequencies to the Mel scale using the conversion shown in Equation 1:

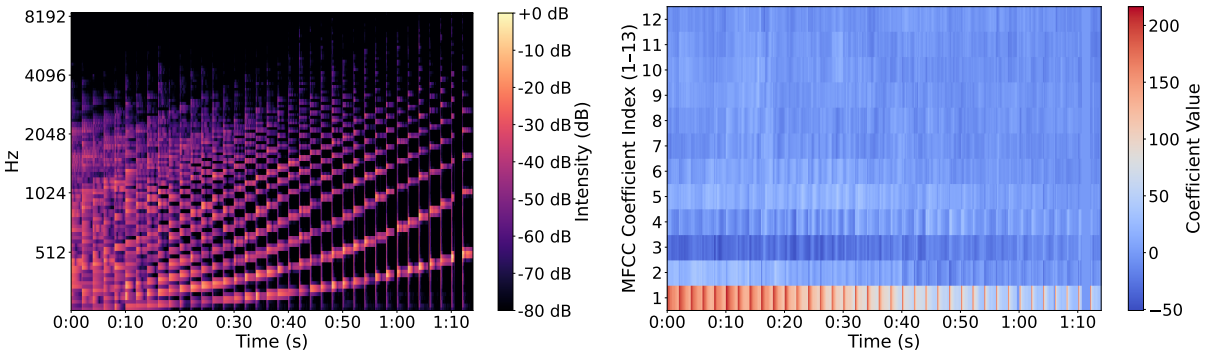
$$\text{Mel frequency} = 2595 \log_{10}\left(1 + \frac{f}{700}\right) \tag{1}$$

The output of the Mel filter bank is then converted to the logarithmic scale, as human perception of loudness is logarithmic in nature. The computation of the MFCCs is built upon the steps for computing the Mel spectrogram, with the addition of a DCT, shown in Equation 2, that is applied to the logarithmic output of the Mel filter bank to decorrelate the output coefficients, resulting in a compact representation which captures the spectral envelope of the signal.

$$X(k) = \sum_{n=0}^{N-1} x_n \cos\left(\frac{2\pi jnk}{N}\right), \quad k = 1, 2, \ldots, N-1 \tag{2}$$

where $X(k)$ is the k-th MFCCs and x_n is the log-energy output of the n-th Mel filter.

A side-by-side comparison of the Mel spectrogram and the MFCCs of a piano playing a note sweep is shown in Figure 5 below:



(a) Mel spectrogram of piano note sweep. **(b) MFCCs of piano note sweep.**

Figure 5. Acoustic representation of piano note sweep ranging from C2 to C5.

From Figure 5a, the logarithmic relationship between time and frequency of the note sweep can be seen from the horizontal bands of colour - the lowest of which is the fundamental frequency. The bands above the fundamental band are the harmonics, which are integer multiples of the fundamental. From Figure 5b, the relationship between the coefficients and time can be seen by the intensity of each coefficient at each point in time. MFCCs are mostly abstract, where the coefficients represent different aspects of the spectral envelope rather than the time and frequency relationship. The first coefficient typically relates to the overall energy of the signal, and the higher-order coefficients may capture information about the rate of change of energy in the spectrum. As discussed in section 1.3, the defining characteristic of the CQT is the constant Q factor, which is defined as the ratio of a filter's centre frequency to the bandwidth of the filter, shown in Equation 3 below:

$$Q = \frac{f_k}{\text{BW}_k} \tag{3}$$

where f_k is the center frequency of the k-th bin, and BW_k is its corresponding bandwidth.

The geometric nature of this ratio allows for the logarithmic spacing between the notes on the western musical scale to be modelled, where each filter on the CQT directly corresponds to the fundamental frequency of a note on the western musical scale. It is shown in [16] that semitone resolution of filters can be obtained by setting $Q = 16.82$. Table 2 illustrates how the geometric spacing approximates the bandwidth between notes within the first octave.

Table 2. CQT filter bandwidth vs. Western musical semitone spacing ($Q \approx 16.82$).

Note	Fundamental frequency (Hz)	Bandwidth to next note (Hz)	CQT filter bandwidth (Hz)
C1	32.70	1.94	1.94
C#1/Db1	34.65	2.06	2.06
D1	36.71	2.18	2.18
D#1/Eb1	38.89	2.31	2.31
E1	41.20	2.45	2.45
F1	43.65	2.60	2.60
F#1/Gb1	46.25	2.75	2.75
G1	49.00	2.91	2.91
G#1/Ab1	51.91	3.09	3.09
A1	55.00	3.27	3.27
A#1/Bb1	58.27	3.46	3.46
B1	61.74	3.67	3.67

By holding Q constant, the CQT provides increased frequency resolution at lower frequencies and increased temporal resolution at higher frequencies - this property is desirable as the increased frequency resolution provides more information at the bass range of frequencies that would ordinarily be lost to the linear spacing of the STFT, and the increased temporal resolution at high-frequency sounds contain rapid transients that define the timbre of an instrument. The harmonic relationship between notes played of a piano note sweep is shown in Figure 6 below:

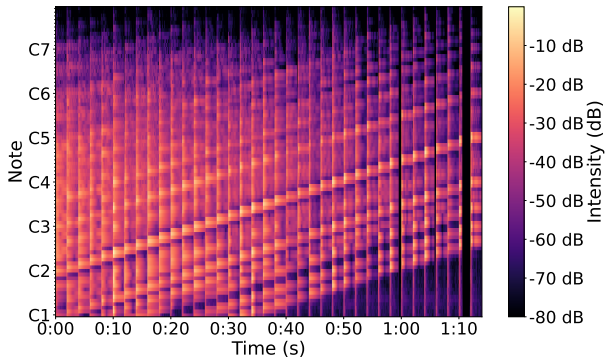


Figure 6. HCQT of piano note sweep ranging from C2 to C5.

3.5.2 Constant Q Transform implementation & verification

There are two methods in which the CQT can be implemented: the direct method proposed by [16], which utilises a fixed fast Fourier transform (FFT) size and leverages the property that a convolution in the time domain is equivalent to an element-wise multiplication in the

frequency domain, and the recursive method proposed by [17], which leverages octave-based downsampling. The direct method defines a single large FFT window that is large enough to provide the frequency resolution required at the lowest frequency kernel, f_{min}. The window length is therefore dictated by the lowest analysis frequency. Large window sizes are computationally inefficient at higher frequency kernels, where short FFT windows are required. The recursive method provides significantly improved computational efficiency by computing the FFT for the highest octave and recursively time-stretching and pitch-shifting the kernel matrix of the octave above to obtain the kernel matrix of the octave below, rather than recomputing the FFT at each octave. This approach is used in state-of-the-art libraries, such as the `librosa.cqt()` method; however, it involves complex implementation logic. Considering deployment on the Jetson Nano, computational efficiency and accuracy of the implementation are crucial considerations. The operational complexity of the direct method, shown in Equation 4, and the recursive method, shown in Equation 5 below:

$$O\left(\frac{L}{H}\cdot(M\log M)\right) \quad (4) \qquad O\left(2\cdot\frac{L}{H}\cdot(M_{\text{oct}}\log M_{\text{oct}})\right) \quad (5)$$

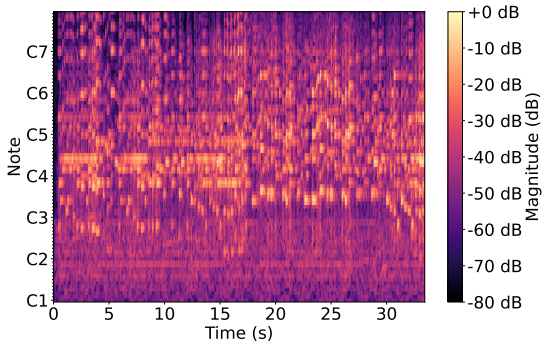
where L is the total number of audio samples, H is the hop length in samples, M is the FFT size required by the direct method to resolve the lowest frequency, and M_{oct} is the fixed FFT size used by the recursive method for a single octave ($M_{\text{oct}} \ll M$).

It is seen that the O complexity of the direct method grows with respect to the frequency range under analysis and the total number of audio samples. By holding the analysis frequency range constant to match that of the western musical scale, the complexity is then linearly correlated to the number of audio samples, i.e, the length of the audio. The scope of the project limits the duration of the audio clips to 30 seconds; thus, the theoretical speedup obtained from using the recursive method is limited to the theoretical speedup obtained using a 30-second clip. The decision becomes a matter of ease of implementation against the limited theoretical speedup. Performance optimisations by techniques of caching, parallelisation of operations and the use of `fftw3` library - a state-of-the-art library for optimised FFT computation - can be used to reduce the performance difference between the two implementations to yield reasonable performance at reduced implementation complexity. The implementation of the direct method begins with padding the audio signal with zero padding on both ends to prevent energy drop-offs at the boundaries of the signal. The Q factor is then calculated based on the input `bins_per_octave` and `num_bins` parameters, using Equation 6.

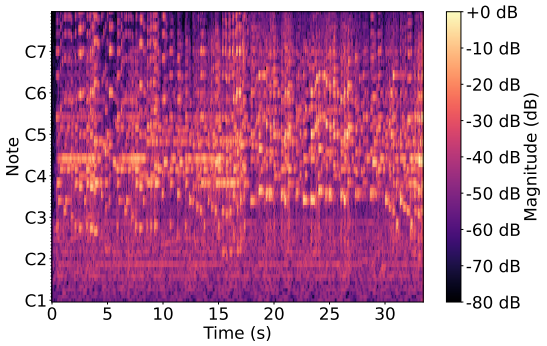
$$Q = \frac{1.0}{2^{\left(\frac{1.0}{\text{bins_per_octave}}\right)} - 1} \quad (6)$$

Thereafter, the required window length for the lowest frequency is calculated based on the calculated Q value. The `next_power_of_2` method is used the find the required FFT size. For each centre frequency, f_k, of each CQT kernel, a complex sinusoid is generated and multiplied with a Hanning window of corresponding length to the kernel. Once again, this is done to smooth the edges of the complex sinusoid and prevent spectral leakage into adjacent bins. The energy of the newly created complex sinusoid is calculated and used to normalise the amplitude such that the L2-norm of the total energy is 1 so that a consistent energy scale across all kernels is created - it is possible that kernels of different frequencies have different lengths and different energy levels. Kernels which have window lengths shorter than that of the FFT size are zero-padded on both the left and right-hand side to match the FFT size. The FFT of each kernel is then computed, resulting in the spectral kernels used for multiplication in the frequency domain.

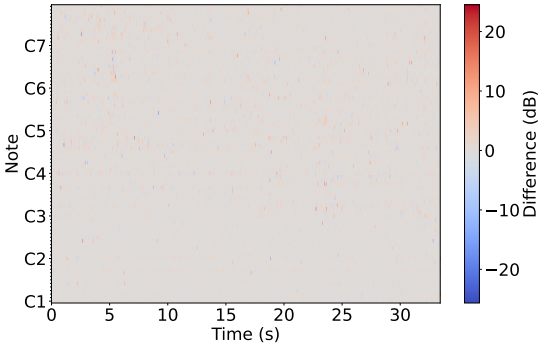
The implementation makes use of two caching structures to minimise redundant computations: a WindowCache and a KernelCache. The WindowCache is a dynamic array which stores the pre-computed Hanning windows for the different CQT kernels to remove redundant calculations for kernels which require the same Hanning window. The KernelCache is a global structure which stores the set of spectral kernels, removing the process of generating and transforming the kernels on subsequent runs. A `for` loop iterates over the samples of the audio signal, processing samples with a window length corresponding to the FFT size and incrementing the loop counter based on the input hop length. This loop is parallelised using OpenMP to distribute the workload of the CQT computation across the CPU cores of the Jetson Nano. As a safety precaution against race conditions, each thread is allocated a private memory buffer during FFT computations at runtime. The FFT of each frame is computed, and a nested loop iterates through each of the pre-computed spectral kernels within the KernelCache. The dot product between the audio frame's spectrum and the complex conjugate of the kernel's spectrum is calculated, and the resulting single complex number is stored as the CQT coefficient for the analysis time frame and analysis frequency bin. The dot product computation was optimised by using the ARM NEON vector instructions - there exists a single instruction multiple data (SIMD) method to perform multiple dot product operations in a single clock cycle. The C implementation was verified by passing the same test audio file to both the librosa CQT method and to the custom C implementation, computing the Pearson correlation coefficient between the flattened output of both methods, and displaying the difference plot between the two CQT outputs. The verification test used an audio duration of 33.6 s, sampled at 44.1 kHz, minimum frequency kernel set at 32.7 Hz (C1), hop length of 512 samples and 12 bins per octave. The results of the verification test are shown in Table 3, and a comparison of the method's output plots is shown in Figure 7.



(a) Output of Librosa CQT method.



(b) Output of custom CQT method.



(c) Difference plot of implementations.

Figure 7. Comparison of Librosa CQT and custom C output plots.

Table 3. Comparison between custom C implementation and Librosa implementation.

Parameter	Custom Imp.	Librosa Imp.
Computation time (s)	0.6333	0.1047
Pearson correlation coefficient	0.999630	

3.6 Convolutional neural network

To determine the optimal combination of feature representation technique and CNN architecture, a baseline assessment is conducted. The set of experiment parameters used for the baseline assessment is as follows: a sample rate of 44.1 kHz as the Nyquist rate captures the audible spectrum of humans, an FFT size of 2048 samples for the Mel-spectrogram and MFCCs, as it provides a frequency resolution that can distinguish musical pitches, and a minimum frequency of 32.7 Hz corresponding to note C1. The Mel bank of the Mel-spectrogram and MFCCs consists of 84 bins to distinguish 84 semitones across 7 octaves. An MFCC count of 13 is a common choice for determining baseline performance of MFCCs in deep learning applications [18]. The HCQT uses 5 harmonics to capture the fundamental harmonic and 4 additional partials - sufficient information to capture the timbral information.

The task of instrument activity detection requires sufficient temporal context and fine frequency resolution to distinguish different timbral features; however, the task of onset detection requires fine temporal resolution to accurately pinpoint the time at which an onset occurs. Müller [1] describes the relationship between frame size and smearing effects on the STFT; thus, the design tradeoff is apparent. Using a hop size of 512 samples for each of the feature representation methods yields an analysis frame size of 11.61 ms, shown by Equation 7 below:

$$\Delta t_{\text{frame}} = \frac{H}{f_s} = \frac{512 \text{ samples}}{44100 \text{ samples/sec}} \approx 0.01161 \text{ s} \approx 11.61 \text{ ms} \tag{7}$$

where Δt_{frame} is the duration of an analysis frame, H is the hop length and f_s is the sample rate.

Krause et al [7] uses a patch size of 2.3 seconds for instrument activity detection; however, such a patch size would be coarse, and the CNN would struggle to distinguish an onset in the presence of a dense feature map. A patch size of 500 ms is selected as it is long enough to capture an instrument onset, while providing sufficient temporal context to learn different timbral characteristics. The input patch dimensions for the Mel-spectrogram, MFCCs, and HCQT are `(1, 84, 44)`, `(1, 13, 44)` and `(5, 84, 44)`, respectively.

3.6.1 Analysis of ResNet-style architecture

The residual network (ResNet) [19] architecture was developed to provide a solution to the vanishing gradient problem, where extremely high or low input values are passed to the activation function, causing it to saturate and the backpropagated gradients to become extremely small. The staple design feature of ResNet is the skip connections within residual blocks, which consist of convolution layers, activation functions, pooling operations and a direct connection that bypasses all layers, from the input to the output. Residual blocks can occur in two forms: identity blocks

and projection blocks. The key difference between these blocks is that the identity block is used when input dimensions are to be preserved, and the projection block is used when downsampling of the input occurs. The architecture of both residual blocks are shown in Figures 8 and 9 below:

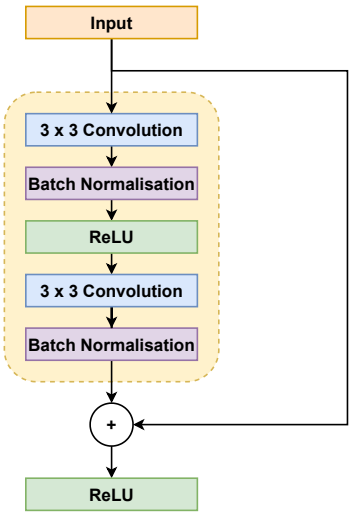


Figure 8. Architecture of identity block.

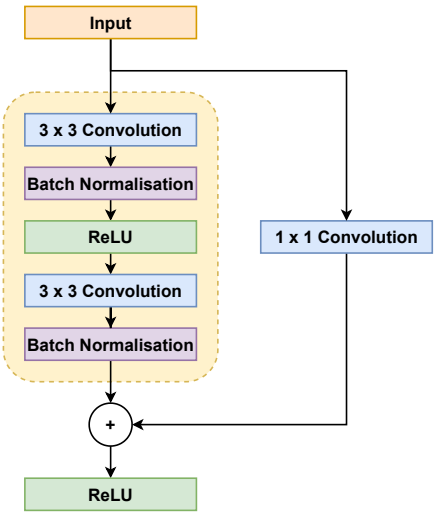


Figure 9. Architecture of projection block.

Each residual block learns an identity function, shown in equation 8 below:

$$y = \sigma\left(\mathcal{F}(x, \{W_i\}) + h(x, W_s)\right) \tag{8}$$

where y is the output, x is the input tensor, σ is the ReLU activation function, $\mathcal{F}(x, \{W_i\}) = \text{BatchNorm}_2(\text{Conv2}_{3\times3}(\sigma(\text{BatchNorm}_1(\text{Conv1}_{3\times3}(x)))))$ is the main path with weights $\{W_i\}$. For the case of the projection block, $h(x, W_s) = \text{Conv}_{1\times1}(x)$ is the skip connection with weights W_s used to match the output dimensions of the main path; otherwise, it is equal to the input.

These skip connections create a direct path for gradients to flow backwards during backpropagation, preserving the gradient's magnitude as it propagates throughout the layers of the network. The gradient of the loss function during backpropagation is shown in Equation 9 below:

$$\frac{\partial \mathcal{L}}{\partial x_l} = \frac{\partial \mathcal{L}}{\partial x_L}\left(1 + \frac{\partial}{\partial x_l}\sum_{i=l}^{L-1}\mathcal{F}(x_i, W_i)\right) \tag{9}$$

where $\frac{\partial \mathcal{L}}{\partial x_l}$ is the gradient of the loss function $\mathcal{L}$ with respect to the output of an earlier layer x_l, and $\frac{\partial \mathcal{L}}{\partial x_L}$ is the gradient at a deeper layer x_L.

The constant term in the parentheses indicates that the gradient can flow unchanged throughout the residual connection. It should be noted that the residual connection architectures shown in Figures 8 and 9 are used in ResNetV1. ResNetV2 [20] proposed that the addition path be completely clean, meaning the addition occurs after the activation of the batch normalisation block. This architecture building block serves as the foundation of many variants of ResNet, such as ResNeXt [21] and DenseNet [22], which have exhibited state-of-the-art performance in several applications of deep learning. Considering deployment on the Jetson Nano, the computational efficiency of these models becomes a consideration; thus, a simpler variant of ResNet is considered. ResNet18 is the most lightweight and computationally efficient variant

in the ResNet family - the architecture is defined by the 18 learnable layers (17 convolutional layers and a single fully connected layer). It begins with a single $7x7$ convolution block using a stride of 2, followed by batch normalisation, a ReLU activation, and a max-pooling layer. Thereafter, there are four sequential stages, each containing two residual connection blocks, and the number of filters doubles at the beginning of each new stage, starting from 64 and growing to 512. This variant has a relatively low parameter count of 11.7 million and requires 1.8 GFLOPS for a standard $224x224x3$ input [23]; however, this parameter count is still complex in terms of an embedded platform that needs to meet the proposed inference time specification. A custom variant of ResNet18 is proposed for the analysis of a ResNet-based architecture; "**SlimResNet**." An architecture block diagram of SlimResNet is shown in Figure 10 below:

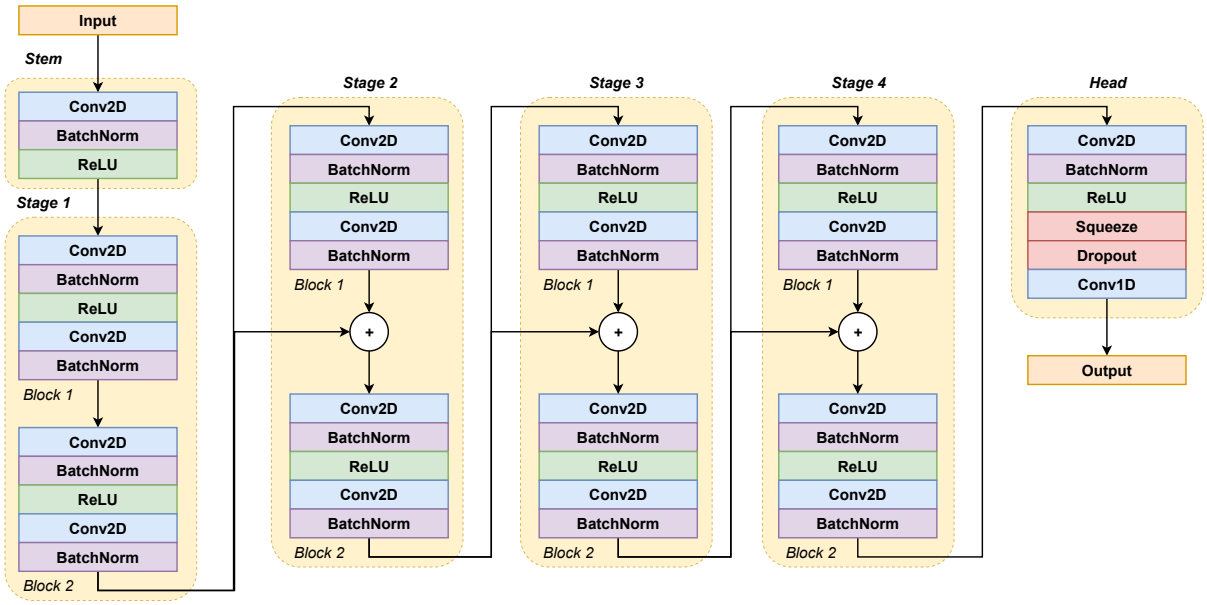


Figure 10. Architecture diagram of SlimResNet.

The proposed SlimResNet maintains the depth of ResNet18, but reduces the number of channels per layer by a factor of 4 to reduce both the parameter count and floating point operations (FLOPs). An approximation of the number of FLOPS for a single convolutional layer is shown in Equation 10 below:

$$\text{FLOPs} \approx 2 \times H_{\text{out}} \times W_{\text{out}} \times C_{\text{out}} \times C_{\text{in}} \times K_H \times K_W \tag{10}$$

where H_{out} and W_{out} are the height and width of the output feature map, C_{out} is the number of output channels, C_{in} is the number of input channels, and K_H and K_W are the height and width of the convolutional kernel.

The task of onset detection requires fine temporal resolution; thus, the conventional head architecture consisting of a fully connected layer followed by an activation function is not suitable. The output of Stage 4 block 2 would be a 4D feature map, which would then be compressed into a single 1D vector. This 1D vector would be used as an input to the fully connected layer, followed by a sigmoid activation to obtain probabilities for each label. The flaw of this approach is that it would produce a single probability per instrument for the entire patch - it is possible that this approach would produce accurate instrument detections, as predictions would be made on a larger context; however, the temporal resolution of predictions would then be dictated

by the patch size. Furthermore, the flattening of the 4D feature map collapses the time axis of the feature map, effectively removing all temporal information. A new head, designed to preserve temporal information, is proposed. It begins with an asymmetric 2D convolutional layer of size `(1, freq_dim)`, where freq_dim is the size of the frequency dimension after downsampling from the residual stages. This convolutional layer aggregates the frequency information into a single feature vector whilst preserving all temporal information. A squeeze operation is then applied to collapse the single feature vector of the frequency information. To mimic the effect of a fully connected layer, without collapsing the time axis, a 1D convolution is applied on the channels dimension of the 3D feature map. The application of the 1D convolution downsamples the size of the channels dimension to match the number of instruments, producing 44 independent prediction vectors, aligned with the 44 frames within a patch. A tabulation of the network architecture parameters of SlimResNet is shown in Table 5 on the subsequent page. SlimResNet's performance across the different feature representations is shown in Table 4 below:

Table 4. Performance comparison of SlimResNet across different feature representations.

Model	F1	Precision	Recall
SlimResNet + Mel spectrogram	0.4360	0.3500	0.6218
SlimResNet + MFCCs	0.6817	0.5880	0.8772
SlimResNet + HCQT	0.6562	0.6098	0.7491

3.6.2 Analysis of VGG-style architecture

The visual geometry group (VGG) architecture, introduced in [8], works off the design hypothesis that depth is the most crucial factor for performance in DNNs. Fundamental to VGG architectures is the VGG building block, which features a $3x3$ convolutional kernel and a $2x2$ max pooling layer that is used to downsample the input. An architecture diagram of the VGG building block is shown in Figure 11 below:

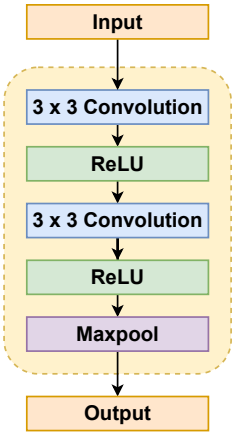


Figure 11. Standard VGG block.

VGG prioritises homogeneity by using the smallest possible kernel that can capture spatial

Table 5. Network architecture parameters of SlimResNet model.

Layer (Kernel, Strides)	Output Shape	Parameters	FLOPs (M)
Input	(5, 44, 84)	-	-
Stem			
Conv2D(7,7)(1,2) + BatchNorm	(16, 44, 42)	3,952	14.5
Stage 1 - Block 1			
Conv2D(3,3)(1,1) + BatchNorm	(16, 44, 42)	2,336	42.7
Conv2D(3,3)(1,1) + BatchNorm	(16, 44, 42)	2,336	42.7
Stage 1 - Block 2			
Conv2D(3,3)(1,1) + BatchNorm	(16, 44, 42)	2,336	42.7
Conv2D(3,3)(1,1) + BatchNorm	(16, 44, 42)	2,336	42.7
Stage 2 - Block 3			
Conv2D(3,3)(1,2) + BatchNorm	(32, 44, 21)	4,672	42.7
Conv2D(3,3)(1,1) + BatchNorm	(32, 44, 21)	9,280	85.4
Shortcut Conv2D(1,1)(1,2) + BatchNorm	(32, 44, 21)	576	0.5
Stage 2 - Block 4			
Conv2D(3,3)(1,1) + BatchNorm	(32, 44, 21)	9,280	85.4
Conv2D(3,3)(1,1) + BatchNorm	(32, 44, 21)	9,280	85.4
Stage 3 - Block 5			
Conv2D(3,3)(1,2) + BatchNorm	(64, 44, 11)	18,560	42.8
Conv2D(3,3)(1,1) + BatchNorm	(64, 44, 11)	36,992	85.6
Shortcut Conv2D(1,1)(1,2) + BatchNorm	(64, 44, 11)	2,176	0.2
Stage 3 - Block 6			
Conv2D(3,3)(1,1) + BatchNorm	(64, 44, 11)	36,992	85.6
Conv2D(3,3)(1,1) + BatchNorm	(64, 44, 11)	36,992	85.6
Stage 4 - Block 7			
Conv2D(3,3)(1,2) + BatchNorm	(128, 44, 6)	73,984	42.8
Conv2D(3,3)(1,1) + BatchNorm	(128, 44, 6)	147,712	85.6
Shortcut Conv2D(1,1)(1,2) + BatchNorm	(128, 44, 6)	8,448	0.2
Stage 4 - Block 8			
Conv2D(3,3)(1,1) + BatchNorm	(128, 44, 6)	147,712	85.6
Conv2D(3,3)(1,1) + BatchNorm	(128, 44, 6)	147,712	85.6
Head			
Conv2D(1,6)(1,1) + BatchNorm	(128, 44, 1)	98,560	25.8
Squeeze	(128, 44)	-	-
Dropout (p=0.5)	(128, 44)	-	-
Conv1D(1)(1)	(11, 44)	1,419	0.1
Output	(11, 44)	-	-
Total	-	**803,643**	**525.9**

Note: Backbone stride is (1, 1) and the first Conv2D of the head is set to (1, 13)(1, 1) for MFCCs.

information, this being a 3*x*3 kernel, as it can capture the concept of centre, left, right, up, and down. Simonyan et al. [8] investigated the relationship between the number of layers within a CNN and model accuracy on image classification tasks - the finding was that performance had improved as depth had increased. This performance improvement was a result of the network's receptive field increasing with more layers - the receptive field being the region of the input a neuron can "see". Whilst a large receptive field is desirable in image classification tasks, this property does not hold true for music information retrieval tasks. Koutini et al. [24] demonstrate that audio-related models are sensitive to the receptive field of the model, specifically the frequency dimension. Thus, to evaluate the performance of a VGG-style architecture, a custom lightweight VGG model is proposed; "**SlimVGG**". An architecture diagram of SlimVGG is shown in Figure 12 below:

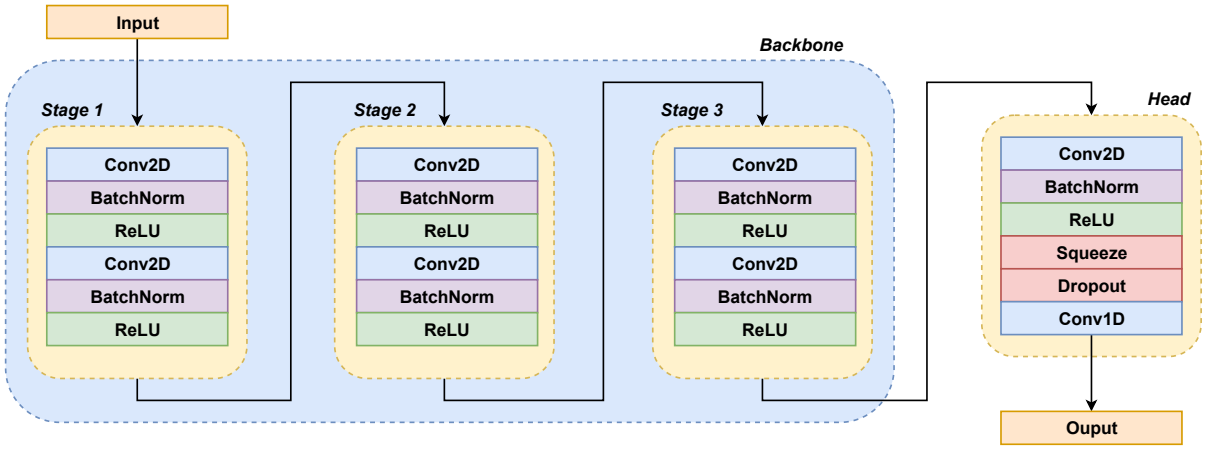


Figure 12. Architecture diagram of SlimVGG network.

A tabulation of the network architecture parameters of SlimVGG is shown in Table 6 on the subsequent page.

Shown in Figure 12 and Table 6, the feature second layer of each stage downsamples the feature map by a factor of 2 by using an anisotropic stride of (1, 2), shifting the kernel by 1 on the time axis and by 2 on the frequency axis. Contrasting this to max pooling, anisotropic strides provide a means of learnable downsampling. The model is forced to learn the optimal kernel weights to extract features and downsample the feature map simultaneously. Using max-pooling, a rule of taking the maximum value is used to downsample the feature map. This rule is not learnt, rather, it is a rigid feature of the architecture. Anisotropic strides also preserve all information within the feature map, as it considers all the features in the receptive field. The max pooling rule considers a set number of feature values; however, it only preserves the most active feature and discards the rest.

The decision to downsample by a factor of 2 has a twofold motivation: to provide a hierarchical feature representation and to create a computational pyramid in the forward pass. The rationale is that during the introductory layers, the model is exposed to high-resolution inputs to detect local patterns, such as a simple harmonic relationship or a musical texture. After downsampling occurs, the intermediate layers are forced to extract features from an abstract, lower-resolution representation. They are forced to combine the knowledge of local patterns with the extracted features from abstract representations to learn more complex features, such as a piano chord. As the feature map gets spatially coarse, the need for additional channels to learn abstract features

Table 6. Network architecture parameters of the SlimVGG model.

Layer (Kernel size, Strides)	Output Shape	Parameters	FLOPs (M)
Input	(5, 44, 84)	–	–
Backbone			
Conv2D(3,3)(1,1) + BatchNorm	(32, 44, 84)	1,504	107.0
Conv2D(3,3)(1,2) + BatchNorm	(32, 44, 42)	9,280	27.0
Conv2D(3,3)(1,1) + BatchNorm	(64, 44, 42)	18,560	68.0
Conv2D(3,3)(1,2) + BatchNorm	(64, 44, 21)	36,992	34.0
Conv2D(3,3)(1,1) + BatchNorm	(128, 44, 21)	73,984	68.0
Conv2D(3,3)(1,2) + BatchNorm	(128, 44, 11)	147,712	35.9
Head			
Conv2D(1,11)(1,1) + BatchNorm	(128, 44, 1)	180,480	88.0
Squeeze	(128, 44)	–	–
Dropout ($p = 0.5$)	(128, 44)	–	–
Conv1D(1)(1)	(11, 44)	1,419	0.1
Output	(11, 44)	–	–
Total	–	**469,931**	**475.9**

Note: Backbone stride is (1, 1) and the first Conv2D of the head is set to (1, 13)(1, 1) for MFCCs.

arises. Thus, it is typical for the number of channels in the model to increase with model depth.

As the number of channels increase with model depth, so does the computational cost of the convolution operation on the feature map. Therefore, downsampling is necessary to prevent the computational cost from exploding at the deeper layers of the network. A factor of 2 is a typical choice in CNNs, as this reduction factor is aggressive enough to maintain stable computational efficiency while creating a sufficient number of abstraction layers within the model. SlimVGG utilises the same head architecture as SlimResNet to provide a fair comparison between the feature aggregation of each model. SlimVGG's performance across the different feature representations is shown in Table 7 below:

Table 7. Performance comparison of SlimVGG across different feature representations.

Model	F1	Precision	Recall
SlimVGG + Mel spectrogram	0.4172	0.3597	0.5659
SlimVGG + MFCCs	0.6837	0.5874	0.8690
SlimVGG + HCQT	0.6851	0.6396	0.7563

3.6.3 Activation function

Activation functions are essential in the design of deep neural networks as they introduce non-linearity into the forward propagation of data - a neural network designed without any non-linearity would simply behave as a linear regression model, incapable of learning complex features from the data. Common choices for activation functions include: sigmoid function,

hyperbolic tangent function, rectified linear unit (ReLU), Leaky ReLU, exponential linear unit (ELU), and parametric rectified linear unit (PReLU).

The sigmoid activation function is shown in Equation 11 below:

$$f(x) = \frac{1}{1+e^{-x}}, \quad \text{Output Range: } (0,1) \tag{11}$$

The output of the sigmoid activation function is interpreted as a probability; thus, it is typically employed in output layers of networks. It is also prone to the vanishing gradient problem. This hinders the learning process of the deep neural network as the updates to the internal parameters become negligible. The function is also not zero-centred - all outputs are positive, which can lead to inefficient weight updates during the gradient descent. Furthermore, the exponential operation in the denominator is more computationally demanding than other activations.

The hyperbolic tangent activation function is shown in Equation 12 below:

$$f(x) = \tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}, \quad \text{Output Range: } (-1,1) \tag{12}$$

The hyperbolic tangent function improves upon the sigmoid function as it is zero-centred; however, it is also prone to the vanishing gradient problem and possesses four exponential terms, making it extremely computationally expensive when compared to other activations.

The ReLU activation function is shown in Equation 13 below:

$$f(x) = \max(0,x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}, \quad \text{Output Range: } [0,\infty) \tag{13}$$

The ReLU function, being one of the most widely adopted activation functions in deep learning applications, involves a simple thresholding operation, making it computationally friendly choice. It can also induce sparsity in training samples by setting negative inputs to zero, thus creating sparse feature maps in the forward propagation. Despite being widely adopted, there are drawbacks to the ReLU activation function: the "Dying ReLU" is a phenomenon where neurons within the network become frozen during training if an extreMely small gradient flows through the neuron. This small gradient causes the weights to update in a way that prohibits the neuron from ever activating again. Additionally, the ReLU activation function is not zero-centred.

The leaky ReLU activation function is shown in Equation 14 below:

$$\begin{gathered} f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases} \\ \text{where } \alpha \text{ is a small positive constant (e.g., 0.01)} \\ \text{Output Range: } (-\infty,\infty) \end{gathered} \tag{14}$$

Leaky ReLU maintains the same computational efficiency as the ReLU function and provides a solution to the dying ReLU problem by allowing a small gradient for negative inputs. This prevents neurons from updating in a way which renders them frozen. The performance of leaky ReLU within hidden layers of a DNN is sensitive to the choice of slope for negative inputs.

The PReLU activation function is shown in Equation 15 below:

$$f(x_i) = \begin{cases} x_i & \text{if } x_i > 0 \\ \alpha_i x_i & \text{otherwise} \end{cases} \tag{15}$$

where α_i is a learnable parameter for the i-th channel

Output Range: $(-\infty, \infty)$

The PReLU activation function benefits from adaptive learning as α is a learnable parameter. The adaptive learning potentially provides superior performance when compared to the leaky ReLU; however, it may overfit to the training data.

Within the context of feature maps of polyphonic audio, negative activations may contain valuable information for distinguishing between instruments. The leaky ReLU presents the most computationally efficient option that allows the model to learn from negative gradients.

3.6.4 Model architecture & design

It is shown in sections 3.6.1 and 3.6.2 that both the ResNet and VGG-style architectures can perform instrument detection with reasonable performance. However, the task of instrument detection can be segmented into sub-tasks of instrument detection per instrument family, being the brass, string, percussion and woodwind families. The rationale is that a single-head architecture is forced to learn a generalised set of weights that are a compromise of the acoustic features that represent each family, as the downsampled features of a percussive instrument would be different to those of the downsampled features of a brass instrument. Multi-task learning is a deep learning strategy where a model is trained to perform classification for several tasks using a backbone-and-head strategy. The motivation for this strategy is that the shared backbone provides a degree of positive transfer by learning general-purpose features for instrument detection - the model will learn low and mid-level features which are standard across all instrument families, such as pitch contours and harmonic structures. The feature map that is output from the backbone is then parsed as an input to independent heads for each task - a brass head, a string head, a percussion head and a woodwind head. Each head has a specialised set of weights to extract instrument family-specific features. As the set of weights is specific to each instrument family, the model is not forced to make a compromise on the extracted acoustic features. These independent heads output the instrument family-specific probabilities, which are then concatenated to produce the final output vector. The architecture diagram of the multi-head VGG architecture is shown in Figure 13 on the subsequent page.

The multi-head architecture also provides a degree of regularisation by segregating the instrument family-specific error that is backpropagated. The computed error for a specific instrument family is backpropagated through that instrument family head and then through the shared backbone. This enables the backbone to learn the generalised low to mid-level features, and allows the heads to become specialised. Since the heads are independent, error gradients of the majority classes will be isolated from the heads of the minority classes. A tabulation of the VGG multihead network architecture parameters is shown in Table 9 on Page 26.

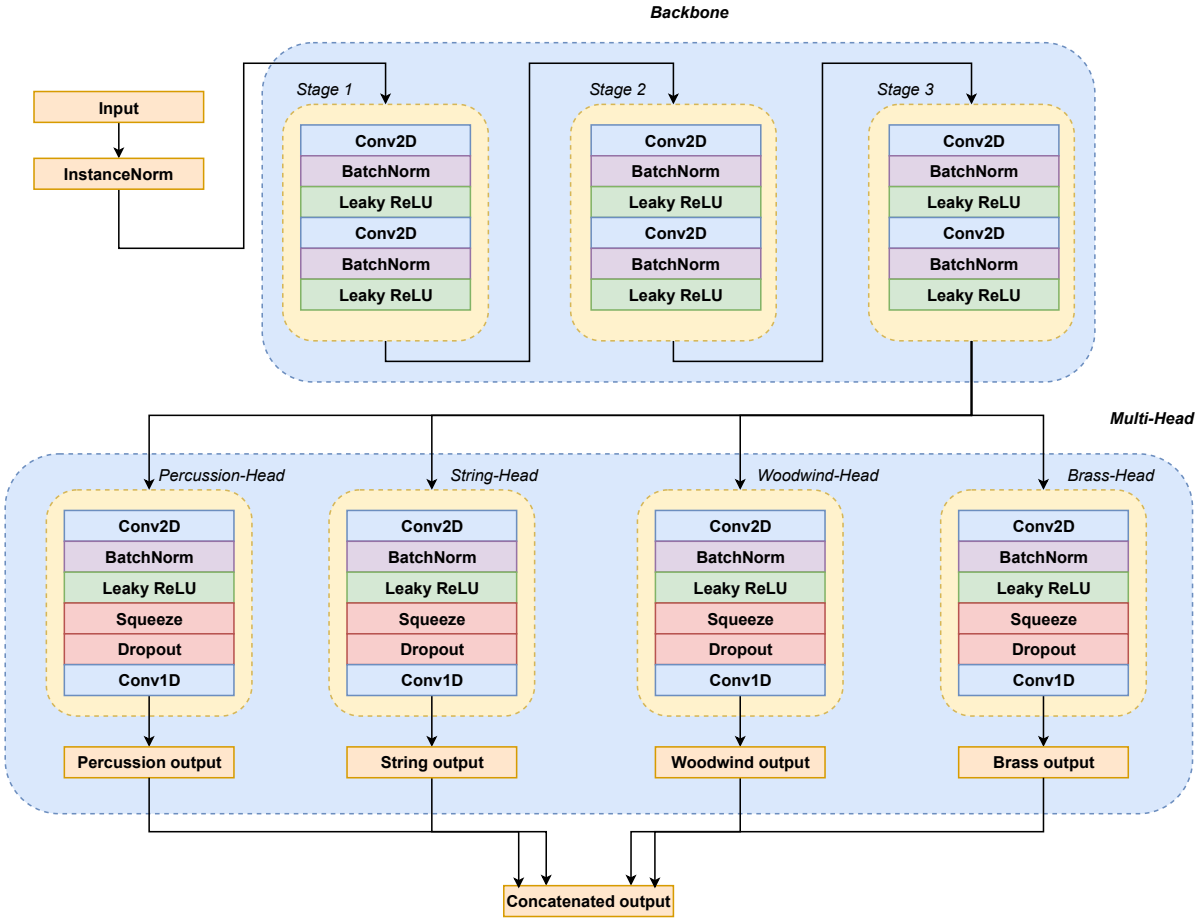


Figure 13. Architecture diagram of the multi-head VGG architecture.

The performance of the multi-head VGG architecture is shown in Table 8 below:

Table 8. Performance of the Multihead VGG CNN across instruments.

Instrument	F1	Precision	Recall
Piano	0.8273	0.7351	0.9461
Violin	0.8558	0.7752	0.9551
Cello	0.8157	0.7398	0.9090
Viola	0.8167	0.7473	0.9003
Contrabass	0.4263	0.3352	0.5854
Clarinet	0.6403	0.6660	0.6166
Bassoon	0.7329	0.6799	0.7949
Oboe	0.7161	0.6799	0.7563
Flute	0.6069	0.5183	0.7320
French Horn	0.6390	0.6848	0.5989
Trumpet	0.7979	0.8682	0.7381
Macro Average	**0.7159**	**0.6754**	**0.7757**

Table 9. Network architecture parameters of VGG multihead model.

Layer (Kernel size, Strides)	Output Shape	Parameters	FLOPs (M)
Input	(5, 44, 84)	–	–
Shared Backbone			
InstanceNorm2d	(5, 44, 84)	10	–
Conv2D(3,3)(1,1) + BatchNorm	(32, 44, 84)	1,504	60.0
Conv2D(3,3)(1,2) + BatchNorm	(32, 44, 42)	9,280	15.1
Conv2D(3,3)(1,1) + BatchNorm	(64, 44, 42)	18,560	38.1
Conv2D(3,3)(1,2) + BatchNorm	(64, 44, 21)	36,992	19.0
Conv2D(3,3)(1,1) + BatchNorm	(96, 44, 21)	55,488	28.6
Conv2D(3,3)(1,2) + BatchNorm	(96, 44, 11)	83,136	14.4
Piano Head			
Conv2D(1,11)(1,1) + BatchNorm	(96, 44, 1)	101,568	55.9
Squeeze	(96, 44)	–	–
Dropout ($p = 0.5$)	(96, 44)	–	–
Conv1D(1)(1)	(1, 44)	97	<0.1
Strings Head			
Conv2D(1,11)(1,1) + BatchNorm	(96, 44, 1)	101,568	55.9
Squeeze	(96, 44)	–	–
Dropout ($p = 0.5$)	(96, 44)	–	–
Conv1D(1)(1)	(4, 44)	388	<0.1
Woodwinds Head			
Conv2D(1,11)(1,1) + BatchNorm	(96, 44, 1)	101,568	55.9
Squeeze	(96, 44)	–	–
Dropout ($p = 0.5$)	(96, 44)	–	–
Conv1D(1)(1)	(4, 44)	388	<0.1
Brass Head			
Conv2D(1,11)(1,1) + BatchNorm	(96, 44, 1)	101,568	55.9
Squeeze	(96, 44)	–	–
Dropout ($p = 0.5$)	(96, 44)	–	–
Conv1D(1)(1)	(2, 44)	194	<0.1
Concatenate Heads	(11, 44)	–	–
Output	(11, 44)	–	–
Total	–	**612,309**	**399.2**

3.6.5 Forward pass implementation & verification

Convolution in 2D is a linear operation which uses a learnable set of weights, called a kernel, to produce an output feature map. Convolution involves two key steps: aligning the kernel over a local region of the input feature map, and performing a dot product between the kernel weights and the input values across all channels. The output, being a single value, is obtained by adding a single bias term.

This process can be mathematically represented, as shown in Equation 16 below:

$$\text{Output}(oc, oh, ow) = b_{oc} + \sum_{ic=0}^{C_{in}-1} \sum_{kh=0}^{k_h-1} \sum_{kw=0}^{k_w-1} W(oc, ic, kh, kw) \times I(ic, oh \cdot s_h - p_h + kh, ow \cdot s_w - p_w + kw) \tag{16}$$

where oc, oh, ow are the indices for the output channel, height, and width, ic, kh, kw are the indices for the input channel and kernel dimensions, b_{oc} is the bias for the output channel, W is the weight tensor of the kernel, I is the input tensor, s_h, s_w are the strides, and p_h, p_w are the padding values.

The forward pass implementation on the embedded platform mimics the logic of the forward pass in Python. The forward pass begins by allocating two aligned memory blocks, which serve as the memory workspaces for the forward pass, named `bufferA` and `bufferB`. Custom C data structures were created to store the exported model weights for each convolution layer and instance normalisation layer, which are used as the internal parameters for the `ModelWeights` custom data structure. After input normalisation, the feature map is passed through the convolution layers of the backbone by iterating through the `backbone_weights` internal parameter. Ping-pong buffering is used to manage the data flow, where the output of layer `i` is written to `bufferA` and used as input to layer `i + 1`. The output is written to `bufferB`, which is used as the input to layer `i + 2`, where to output is stored in `bufferA` - the process repeats for all backbone layers. This is done as a safety precaution against dynamic memory management-related errors. The output of the backbone, stored in `bufferB`, is then used as input to each of the `head_weights` parameters. The output of each head is then concatenated, and a sigmoid function is applied to obtain the final probabilities.

The `conv_relu_block` method serves as an abstraction of the most common sequential operations within the forward pass: two-dimensional (2D) convolution and leaky ReLU activation. The 2D convolution method makes use of three optimisations. The outermost loop of the convolution function is parallelised using OpenMP to distribute the workload across the available CPU cores. The principle of spatial locality is exploited to reduce the number of main memory access operations by using loop tiling - the entire convolution is broken down into $8x16$ feature maps to fit within the CPU's L1/L2 cache. The loop tiling feature map must be small enough to fit within the 32 kB L1/L2 cache of the Jetson Nano without occupying an excessive amount of memory. The required size of the $8x16$ feature map and weights when propagating through the layer with the largest kernel size, as it utilises the most memory of all layers, is shown below:

$$\text{Dimension size} = \text{(Output Size - 1)} \times \text{Stride} + \text{Kernel Size}$$

$$\text{Input height} = (8-1) \times 1 + 7 = 14$$

$$\text{Input width} = (16-1) \times 1 + 7 = 37$$

$$\text{Total size of input} = 14 \times 37 \times 32 \text{ bytes per float} = 2072 \text{ bytes}$$

$$\begin{aligned}\text{Total size of weights} &= \text{Input Channels} \times \text{Kernel Height} \times \text{Kernel Width} \times \text{bytes per float} \\ &= 96 \times 3 \times 3 \times 4 \text{ bytes per float} = 3456 \text{ bytes}\end{aligned}$$

The total working size of both the layer input and the weights is ≈ 5.4 kB fits within the available storage of the L1/L2 cache of the Jetson Nano. A ≈ 4x speedup is achieved by using ARM NEON SIMD intrinsics, which allow the processor to perform up to four floating operations in a single operation. The output of the convolution is then parsed to the leaky ReLU method, which also uses the ARM NEON SIMD intrinsics to optimise the comparison and conditional multiplication. An algorithmic block detailing the pseudocode of the convolution method is shown in Algorithm 1 on the subsequent page.

The outermost loop is parallelised and iterates over the channel dimension of the output feature map. The second and third loops iterate over the height and width of the current loop tile within the L1/L2 cache. The fourth and fifth loops iterate over the output height and width, with the fifth loop initialising the `sum` parameter to the bias of the current channel. The height sliding parameter `ih` is calculated based on the current output height, kernel height, striding and padding values, which is followed by a boundary check to ensure that the sliding parameter stays within the bounds of the input feature map. The innermost loop calculates the width sliding parameter, extracts four elements from the input and the kernel weights corresponding to the sliding parameters indices. The dot product of these elements is added to the accumulator. The final for loop iterates over the final elements of the feature map, which are smaller than the dimensions of the kernel. The final sum is then assigned to an index in the output feature map.

The C implementation was verified by generating a random test input, passing the input through the PyTorch implementation of the model and comparing the flattened layer-by-layer output between the PyTorch model and the C implementation. The data and error statistics of the flattened vectors are shown in Table 10 on Page 30.

Algorithm 1 Pseudocode of 2D convolution method.

Input: Input tensor I, Weights W, Biases B, Layer Parameters *Params*
Output: Output tensor O
Extract dimensions: $H_{in}, W_{in}, C_{in}, H_{out}, W_{out}, C_{out}, K_h, K_w$ from *Params*
$T_h \leftarrow 8$, $T_w \leftarrow 16$
for all output channel oc from 0 to $C_{out} - 1$ **do in parallel**
    $bias \leftarrow B[oc]$
    for all tile height oh_{tile} from 0 to $H_{out} - 1$ step T_h **do**
        for all tile width ow_{tile} from 0 to $W_{out} - 1$ step T_w **do**
            $oh_{end} \leftarrow \min(oh_{tile} + T_h, H_{out})$
            $ow_{end} \leftarrow \min(ow_{tile} + T_w, W_{out})$
            for all output height oh from oh_{tile} to $oh_{end} - 1$ **do**
                for all output width ow from ow_{tile} to $ow_{end} - 1$ **do**
                    $sum \leftarrow bias$
                    for all input channel ic from 0 to $C_{in} - 1$ **do**
                        for all kernel row kh from 0 to $K_h - 1$ **do**
                            $ih \leftarrow oh \cdot S_h - P_h + kh$
                            if $ih \geq 0$ **and** $ih < H_{in}$ **then**
                                $kw \leftarrow 0$
                                $sum_{vec} \leftarrow \vec{0}$
                                for all kw from 0 to $K_w - 4$ step 4 **do**
                                    $iw_{base} \leftarrow ow \cdot S_w - P_w + kw$
                                    if input region for 4 elements is valid **then**
                                        $I_{vec} \leftarrow$ Load 4 floats from $I[ic, ih, iw_{base}]$
                                        $W_{vec} \leftarrow$ Load 4 floats from $W[oc, ic, kh, kw]$
                                        $sum_{vec} \leftarrow$ Dot product$(sum_{vec}, I_{vec}, W_{vec})$
                                    end if
                                end for
                                $sum \leftarrow sum + sum_{vec}$
                                for all remaining kw to $K_w - 1$ **do**
                                    $iw \leftarrow ow \cdot S_w - P_w + kw$
                                    if $iw \geq 0$ **and** $iw < W_{in}$ **then**
                                        $sum \leftarrow sum + I[ic, ih, iw] \times W[oc, ic, kh, kw]$
                                    end if
                                end for
                            end if
                        end for
                    end for
                    $O[oc, oh, ow] \leftarrow sum$
                end for
            end for
        end for
    end for
end for

Table 10. Verification of custom C implementation against PyTorch implementation.

		PyTorch imp.		Custom C imp.		Error	
Layer	**Shape**	**Mean**	**Std**	**Mean**	**Std**	**Mean abs.**	**Max abs.**
Input	(18480,)	0.0029	0.0058	0.0029	0.0058	0.00e+00	0.00e+00
Input Norm.	(18480,)	0.2259	0.8210	0.2259	0.8210	4.00e-08	1.91e-06
Backbone							
Block 1	(118272,)	0.1916	0.3960	0.1916	0.3960	4.00e-08	2.86e-06
Block 2	(59136,)	0.2709	0.3878	0.2709	0.3878	1.40e-07	5.72e-06
Block 3	(118272,)	0.2907	0.3940	0.2907	0.3940	2.50e-07	4.53e-06
Block 4	(59136,)	0.3042	0.4441	0.3042	0.4441	3.10e-07	5.72e-06
Block 5	(88704,)	0.2569	0.4202	0.2569	0.4202	3.30e-07	6.68e-06
Block 6	(46464,)	0.2564	0.4237	0.2564	0.4237	3.30e-07	7.63e-06
Piano Head							
Aggregator	(4224,)	0.0488	0.1986	0.0488	0.1986	1.10e-07	4.44e-06
Classifier	(44,)	-1.0305	0.6009	-1.0305	0.6009	2.77e-06	7.33e-06
Sigmoid	(44,)	0.2783	0.1050	0.2783	0.1050	5.00e-07	1.64e-06
Strings Head							
Aggregator	(4224,)	0.0881	0.2183	0.0881	0.2183	1.30e-07	2.62e-06
Classifier	(176,)	-2.6467	1.3006	-2.6467	1.3006	2.04e-06	7.63e-06
Sigmoid	(176,)	0.1103	0.1133	0.1103	0.1133	1.50e-07	9.50e-07
Woodwinds Head							
Aggregator	(4224,)	0.1015	0.2279	0.1015	0.2279	1.40e-07	2.86e-06
Classifier	(176,)	-3.4319	1.5936	-3.4319	1.5936	1.87e-06	9.06e-06
Sigmoid	(176,)	0.0755	0.0876	0.0755	0.0876	1.10e-07	1.01e-06
Brass Head							
Aggregator	(4224,)	0.0671	0.1706	0.0671	0.1706	1.00e-07	3.99e-06
Classifier	(88,)	-2.7835	1.2554	-2.7835	1.2554	1.54e-06	7.63e-06
Sigmoid	(88,)	0.0920	0.0746	0.0920	0.0746	9.00e-08	5.20e-07

Note: 'Std' denotes standard deviation

3.7 Data augmentation

Data augmentation techniques were used to artificially expand the training data subset. Pitch shifting, time masking and synthetic data generation were used for offline augmentations. Frequency masking and time masking were used for online augmentations. This subsection provides a brief explanation of the theory behind the augmentation techniques.

3.7.1 Pitch shifting & Time warping

Pitch shifting is a technique which alters the pitch of the audio mix without changing the tempo or loudness of the mix. This can simulate variations in instrument tuning. Pitch shifting was achieved by using the `librosa.effects.pitch_shift()` method, which works by calculating the STFT of the audio mix, altering the phase information within each frame to the

desired pitch and then converting the signal back to the time domain using the inverse STFT. Pitch shifting can be mathematically modelled as shown in Equation 17 below:

$$y(t) = x(t) \cdot e^{j \cdot 2\pi \cdot f_{\text{shift}} \cdot t} \tag{17}$$

where $y(t)$ is the output pitch-shifted signal, $x(t)$ is the original input signal and f_{shift} represents the desired frequency shift.

Time warping alters the duration of the audio mix without changing the pitch or loudness. This can simulate various tempos. Time warping was achieved by using the `librosa.effects.time_stretch` method, which works by calculating the STFT of the audio mix, altering the phase information by either increasing or decreasing the temporal density and then converting the signal back to the time domain using the ISTFT. Time warping can be conceptualised in a simple manner as shown in Equation 18

$$y(t) = x(t/\alpha) \tag{18}$$

where $y(t)$ is the time-stretched output signal, $x(t)$ is the original input signal, and α is the stretching factor.

3.7.2 Synthetic data generation

A synthetic audio generation pipeline was created by downloading MIDI files of classical music pieces from MuseScore [25], adding humanisation effects, synthesising the audio with a soundfont, convolving the audio with an impulse response and mixing in a room tone.

MIDI files are digital instructions which contain information on how a track should be played. It consists of note-level instructions, where each note has parameters such as the note pitch, velocity, start time and end time. These files are far from the realistic rendition of a music piece as they are perfectly timed and dynamically flat; they do not account for imperfections such as variations in timing, playing styles and recording dynamics. Random variations were simulated by adding random constants sampled from a Gaussian distribution to note start times and velocity.

The humanised MIDI files were synthesised into artificial mixes using the VSCO-2 (Versilian Studios Chamber Orchestra 2) Community Edition soundfont collection and FluidSynth. A soundfont is a collection of individual note recordings of instruments. The synthesiser applies the soundfont to the individual instrument stems in the MIDI files to create realistic instrument sounds for each note level instruction. The output of this step is a dry audio signal, which lacks recording effects and is often void of different playing styles.

An impulse response of a recording chamber was downloaded from [26] and convolved with the dry audio signal to simulate the spatial acoustics of a recording chamber, such as reverberation and echo. The impulse response serves as an acoustic fingerprint of the recording chamber - it is a short and sharp sound with the loudest point being 1 dB SPL. It captures the reflections, echoes and the decay of sound within the environment. By convolving the dry audio signal with the impulse response of the recording chamber, the characteristics of the impulse response are applied to the dry audio signal. The convolution of the two signals is mathematically modelled as shown in Equation 19 below:

$$y[n] = (x * h)[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k] \tag{19}$$

where $y[n]$ is the wet stem, $x[n]$ is the dry stem, and $h[n]$ is the impulse response of the chamber hall. The output sample at time n is calculated as the weighted sum of all input samples, where the weights are determined by the time-reversed impulse response.

A silent room tone was recorded with amplitude varying between -54 dB SPL to -48 dB SPL and mixed with the wet stem. This was done to simulate the background hiss present from either the microphone or the environment. Annotations of the synthetic recordings were extracted from the MIDI files and saved alongside the synthetic audio track.

A new, balanced training set was curated using a combination of the aforementioned data augmentation techniques. The dataset contained ≈ 200 minutes per instrument. The multi-head VGG model was trained on this dataset. The performance using the synthetic training set is shown in Table 11 below:

Table 11. Performance of multi-head VGG with synthetic training set.

	F1-score	Precision	Recall
Multi-head VGG + Synthetic data	0.645	0.594	0.741

The use of synthetic data had caused the model performance to degrade - a hypothesis for this is presented in Section 3.10.

3.8 Training pipeline

3.8.1 Loss function implementation & verification

There exists a need for an empirical measure of performance on any given subset of data, whether training, validation or the test sets. This empirical measure comes in the form of an objective/loss function, $J(\theta)$, which quantifies the difference between the model's predictions and the ground-truth labels for a given set of data. Monitoring the loss value during training provides an indication if the model is "learning" the training set of data, quantifying the loss on the validation set provides an indication of the model's ability to generalise to unseen data, and quantifying the loss on the test set provides a measure of model performance in real-world scenarios. Each frame may have multiple instruments present; thus, the presence of each instrument is an independent binary classification problem, with the output of the model being a vector of probabilities for each instrument. The following loss functions are candidates for the training pipeline: Hamming loss, binary cross-entropy (BCE) [27] and focal loss [28].

The Hamming loss calculates the fraction of labels that are incorrectly labelled. Whilst the hamming loss is simple, it possesses the drawback that it works with binary predictions rather than probabilities, neglecting the model's confidence in predictions during evaluation. Furthermore, a model that predicts no labels are active for a sample with a single label active would achieve a low Hamming loss, providing an incorrect reflection of performance. BCE loss is the most common loss function for multi-label classification tasks. The loss function calculates the

difference between the model predictions and the target probabilities - the total loss is the average of the computed loss per label. Whilst the BCE loss function is conceptually simple, it may be numerically unstable if not used with sigmoid activation. In addition, it treats all labels equally, which may pose a challenge in situations of significant class imbalance, as the model will struggle to learn the minority class. BCE with logits loss combines a sigmoid activation layer with the loss layer into a single function. This combination allows for the log-sum computation, which improves numerical stability. Focal loss is an enhancement of the BCE loss designed to address the issue of class imbalance by adding a modulating factor that reduces the loss contribution from prevalent labels, allowing the model to focus its learning on less prevalent labels. Whilst focal loss may be beneficial in situations of extreme label imbalance, it introduces additional hyperparameters which need tuning during the training process. Poor tuning of these hyperparameters may lead to poor stability during training.

In an effort to minimise the hyperparameters introduced within the system and maximise numerical stability, the binary cross-entropy with logits loss function was employed. Furthermore, class weights of the labels are used to provide a remedy to the data imbalance of that dataset. A custom binary cross-entropy with logits loss function was implemented from first principles in Python, and functionality was verified by comparing the output to the PyTorch implementation of the loss function. The custom implementation takes the form of a function which accepts the logits of the model output and the target labels as input. The loss function first converts the raw logits into probabilities $p \in [0,1]$ by applying the sigmoid function, defined in Equation 20 below:

$$p = \sigma(x) = \frac{1}{1+e^{-x}} \tag{20}$$

The total loss is computed as shown in Equation 21 below:

$$\mathcal{L}(p,z) = -[z \cdot \log(p) + (1-z) \cdot \log(1-p)] \tag{21}$$

where p is the converted probability and z is the binary target label.

Substituting 20 into 21, a direct yet naive formula for the binary cross-entropy with logits loss function is obtained, shown in Equation 22 below:

$$\mathcal{L}(x,z) = -\left[z \cdot \log\left(\frac{1}{1+e^{-x}}\right) + (1-z) \cdot \log\left(1 - \frac{1}{1+e^{-x}}\right)\right] \tag{22}$$

This direct computation of the loss function may be numerically unstable in certain situations: considering a large positive logit x, $e^{-x} \to 0$, causing $p \to 1$. The term $\log(1-p)$ then approaches $\log(0)$, which is undefined and will produce an error at runtime. Conversely, for a large negative logit x, e^{-x} becomes an exponentially large number, which may cause overflow errors at runtime. This issue is circumvented by rearranging the formula in a manner which yields the same result. The rearranged formula is shown in Equation 23 below:

$$\mathcal{L}(x,z) = \max(x,0) - xz + \log(1+e^{-|x|}). \tag{23}$$

The above equation is stable as the exponential term $e^{-|x|}$ will always have a negative or zero exponent, thus, asserting the result is always between 0 and 1.

The Python function is a direct line-by-line implementation of Equation 23. A random test vector of logits and a random target vector were generated and parsed as inputs to both the custom implementation and the PyTorch implementation. The results of the comparison are shown in Table 12 below:

Table 12. Verification of the custom BCE with logits loss implementation.

Parameter	Value
Input data shape	(1, 11) for both logits and targets.
Input logits vector	`[[ 1.5, -0.8, 2.1, -3.0, 0.1, 0.5, -1.2, 4.0, -2.5, 0.9, 1.1 ]]`
Input targets vector	`[[ 1, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1 ]]`
Custom implementation output	0.26762780
PyTorch method output	0.26762780

3.8.2 Training optimizer implementation & verification

The primary objective of the training process is to minimise the value of the defined loss function. The model's behaviour is dictated by the internal parameters of the model, θ, within the loss function computation, $J(\theta)$. Thus, it is apparent that the process of minimising loss is an optimisation problem. The role of the Optimiser is to guide the training process in finding the optimal set of internal parameters that yields the lowest possible loss value. The optimiser iteratively updates the internal parameters of the model by calculating the gradient vector of the loss, $\nabla_\theta J(\theta)$, which points in the direction of steepest ascent of the loss landscape. The optimiser takes steps in the opposite direction of the gradient to systematically navigate the loss landscape during the training process - an algorithm known as gradient descent. This algorithm serves as the fundamental principle behind the iterative updates of the optimiser and is mathematically modelled as shown in Equation 24 below:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta J(\theta) \tag{24}$$

where η is the learning rate.

Whilst gradient descent is conceptually simple and guaranteed to move in the correct direction, it can be computationally expensive for large datasets and can get trapped in local minima. Stochastic gradient descent provides a computationally feasible means of parameter updates by approximating the gradient using a random sample from the training set at each update step, and is mathematically modelled as shown in Equation 25 below:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta J(\theta; x^{(i)}; y^{(i)}) \tag{25}$$

where $(x^{(i)}, y^{(i)})$ is a single training sample.

Whilst stochastic gradient descent is computationally efficient, computing gradients on a single sample introduces high variance in parameter updates, which creates oscillations in the loss landscape. Minibatch stochastic gradient descent provides a compromise between gradient descent and stochastic gradient descent by computing gradients on a random subset of the data and is mathematically modelled as shown in Equation 26 below:

$$\theta \leftarrow \theta - \eta \cdot \nabla_\theta J(\theta; x^{(i:i+n)}, y^{(i:i+n)}) \tag{26}$$

where n is the batch size.

Whilst minibatch stochastic gradient descent provides improved stability in gradient estimation and reduced variance in parameter updates, it is still prone to oscillation in the loss landscape. The momentum method was introduced to mitigate the oscillations that are inherent to stochastic methods by accelerating movement in the direction of consistent descent and dampening the

updates in the direction of high variance. The equations of the momentum method shown in Equations 27 and 28 below:

$$v_t \leftarrow \beta v_{t-1} + (1-\beta)\nabla_\theta J(\theta) \tag{27}$$

$$\theta \leftarrow \theta - \eta v_t \tag{28}$$

where v_t is the velocity, being an exponentially decaying moving average of past gradients, and β is the momentum coefficient, typically ~ 0.9.

The field of machine learning has seen clear progression in terms of optimisers, with each optimiser aiming to improve upon the predecessor. Four optimisers are considered candidates for the training pipeline: Adagrad, RMSProp, Adam and AdamW. The Adagrad optimiser [29] introduced the technique of adaptive learning rates per parameter within the model, allowing for smaller updates on frequently occurring features and larger updates for sparse features. The update step for the Adagrad optimiser is shown in Equation 29 below:

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{G_t + \varepsilon}} \cdot \nabla_\theta J(\theta_t) \tag{29}$$

where G_t is a diagonal matrix containing the sum of the squares of past gradients for each parameter and ε is a small constant for numerical stability.

Whilst Adagrad provides parameter-specific learning rates and is well-suited for sparse data, the learning rate monotonically decreases and can stop the learning process prematurely. RMSProp [30] addresses the diminishing learning rate by using a decaying average of the squared gradients, rather than the cumulative sum of the gradients. The update step for the RMSProp optimiser is shown in Equation 31 below:

$$E[g^2]_t \leftarrow \gamma E[g^2]_{t-1} + (1-\gamma)(\nabla_\theta J(\theta))^2 \tag{30}$$

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{E[g^2]_t + \varepsilon}} \cdot \nabla_\theta J(\theta_t) \tag{31}$$

where $E[g^2]_t$ is the decaying average of squared gradients and γ is the decay rate.

The Adam optimiser [31] leverages both the momentum of the gradient, the first moment estimate, and the adaptive scaling of the learning rate used in RMSProp, by means of an exponentially decaying moving average of past squared gradients, the second moment estimate, to provide quick and stable convergence. The update procedure for the Adam optimiser is shown in Equation 36 below:

$$m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1) g_t \tag{32}$$

$$\hat{m}_t \leftarrow \frac{m_t}{1-\beta_1^t} \tag{33}$$

$$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{34}$$

$$\hat{v}_t \leftarrow \frac{v_t}{1-\beta_2^t} \tag{35}$$

$$\theta_{t+1} \leftarrow \theta_t - \frac{\eta}{\sqrt{\hat{v}_t + \varepsilon}} \hat{m}_t \tag{36}$$

where m_t and v_t are the first and second moment estimates of the gradients, β_1 and β_2 are their respective decay rates, and ε is a small constant for numerical stability.

AdamW [32] improves on the Adam optimiser by decoupling the weight decay from the gradient updates step. Adam scales the weight decay term by the adaptive learning rate - this diminishes the regularising effect on the parameters with large gradients. The AdamW optimiser is considered state-of-the-art and is the de facto choice for deep learning tasks as it provides improved generalisation capabilities and lower validation loss. The update procedure for the AdamW

optimiser follows that of the Adam optimiser; however, with the modification that the weight decay is applied directly on the model parameters, shown in Equation 37, before the standard Adam update.

$$\theta_t \leftarrow \theta_t - \eta \lambda \theta_t \tag{37}$$

where λ is the weight decay factor.

The AdamW optimiser was implemented from first principles in Python and designed to replicate the performance of the PyTorch implementation of the optimiser. The implementation followed an object-oriented approach, with two key methods: the initialisation (`__init__`) and the step (`step`) method. The initialisation method accepts a list of parameters to set up the internal state of the optimiser: `lr` (learning rate), `betas` (tuple for the decay rates of the first and second moment estimates), `eps` (numerical stability term) and `weight_decay`. The internal state of the optimizer is encapsulated within the `state` dictionary which contains 3 elements: `m` (list of arrays, initialized to zeros, to store the first moment estimate), `v` (list of arrays, also initialized to zeros, to store the second moment estimate), and `step` (Integer counter, initialized to 0, to track the number of updates performed). The `step` method performs the five distinct logical operations needed to update each parameter per training iteration - an algorithmic logic block of the method is shown in Algorithm 2 below:

Algorithm 2 The AdamW Optimisation Step

Require: Learning rate η; decay rates β_1, β_2; weight decay λ; epsilon ε; model parameters θ_{t-1}; gradient $\mathbf{g}_t$; step t, first and second moment vectors from previous step: $\mathbf{m}_{t-1}, \mathbf{v}_{t-1}$

$\mathbf{g}_t \leftarrow \nabla_\theta J(\theta_{t-1})$ ▷ Calculate gradient for the current loss
$t \leftarrow t + 1$ ▷ Increment the timestep counter
for all parameter θ_i in θ **do**
  $\theta_i \leftarrow \theta_i \cdot (1 - \eta\lambda)$ ▷ Decouple the weight decay
  $\mathbf{m}_t \leftarrow \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1)\mathbf{g}_t$ ▷ Calculate the first moment
  $\mathbf{v}_t \leftarrow \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2)\mathbf{g}_t^2$ ▷ Calculate the second moment
  $\hat{\mathbf{m}}_t \leftarrow \mathbf{m}_t / (1 - \beta_1^t)$ ▷ Apply bias correction on first moment
  $\hat{\mathbf{v}}_t \leftarrow \mathbf{v}_t / (1 - \beta_2^t)$ ▷ Apply bias correction on second moment
  $\theta_t \leftarrow \theta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \varepsilon}$ ▷ Update parameters
end for
return Updated parameters θ_t, and state $\mathbf{m}_t, \mathbf{v}_t$

The custom AdamW implementation was compared to the PyTorch AdamW implementation using the Rosenbrock function - a common test case for optimisation problems, as it is non-convex, the global minimum, coordinates $[1, 1, 1]$, lies inside a narrow, parabolic-shaped valley with small gradients in the valley. These small gradients pose a complex challenge for optimisers to navigate. Both optimisers were initialised with the following parameters: a starting position of $[-1.0, 2.0, -0.5]$, learning rate of 0.01, weight decay of 0.01 and a simulation run for 1000 steps. The trajectory of the individual parameters (x, y, z) and the relative error in calculated loss at each step are shown in Figures 14 and 15, respectively, below:

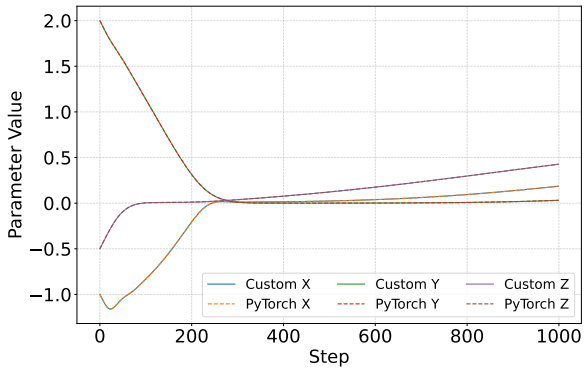


Figure 14. X, Y, Z parameter trajectory.

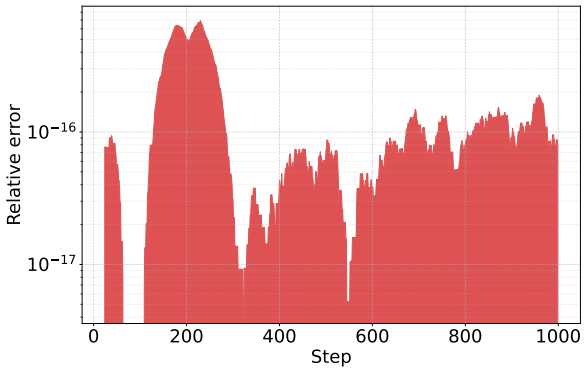


Figure 15. Relative error in calculated loss.

Figure 14 shows that the trajectory of the X, Y, Z parameters of the custom implementation overlaps that of the parameter evolution of the PyTorch implementation. From Figure 15, it is seen that the maximum relative error is $\approx 6.9e-16$ - a near-zero error which can be attributed to minor differences in how mathematical operations are performed at low-level, such as floating-point arithmetic discrepancies between NumPy and PyTorch, rather than algorithmic errors. To visually confirm the trajectory within the Rosenbrock loss landscape, a three-dimensional plot of the XY plane vs validation loss (setting Z = 1) and the corresponding two-dimensional contour plot are shown in Figures 16 and 17 below:

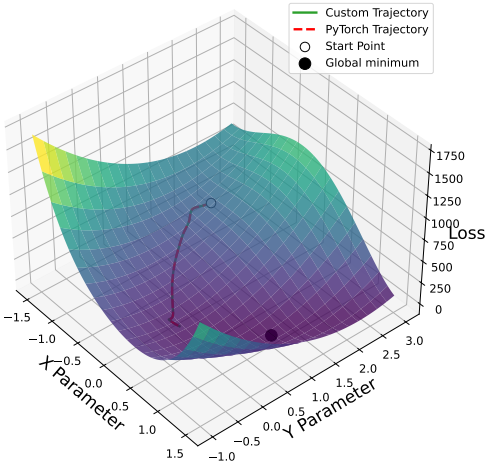


Figure 16. Comparison of optimiser trajectory on Rosenbrock surface.

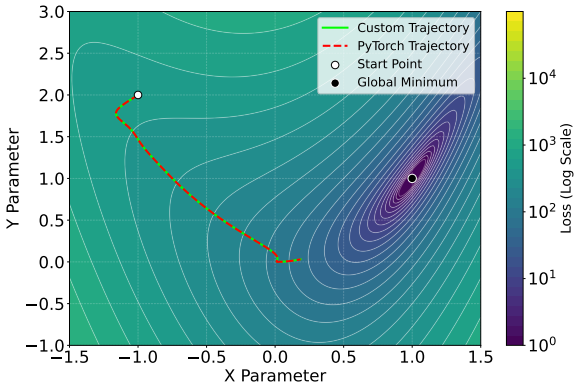


Figure 17. Comparison of optimiser trajectory on contour plot.

3.8.3 Learning rate scheduler

The learning rate, η, is a crucial hyperparameter used in the training of DNNs - it is responsible for the step size at which the model's weights are updated during the training process. Thus, selecting the appropriate learning rate and scheduling strategy is essential in ensuring that the model's internal parameters converge to an optimal solution within the loss landscape. A high learning rate may cause the model's internal parameters to overshoot the optimal solution, causing

the loss to oscillate around the optimal solution. Conversely, a low learning rate may lead to slow training as the model's internal parameters take very small steps toward the optimal solution. There also exists the possibility that the loss may get stuck in a suboptimal local minimum. The need for scheduling arises as a fixed learning rate typically produces suboptimal results. A learning rate scheduler varies the learning rate throughout the training process. Ideally, having large steps during the initial stage of training to find the valley of the global minimum, with smaller steps as training progresses to converge to the global minimum. There are four candidate learning rate schedules for this project: reduction on plateau, step decay, exponential decay and cosine annealing.

The reduction on plateau strategy is a simple strategy which reduces the learning rate by a predefined factor when no reduction in validation loss is seen for a certain number of epochs. The step decay strategy functions in a similar light to the reduction on plateau strategy, with the difference being that the reduction in the learning rate occurs at predefined epochs, rather than by monitoring the behaviour of the validation loss. Whilst both strategies are conceptually simple and have proven to be effective strategies, they both introduce additional hyperparameters, being the decay factor and the step epochs, that need to be tuned. Poor tuning of these hyperparameters can be disruptive to the training momentum and cause oscillations around the optimal solution. The exponential decay strategy multiplies the learning rate by a decay factor after every epoch, which results in a gradual decrease in the learning rate over time. Whilst the exponential decay strategy may result in faster convergence when compared to the step decay, it is also possible that the learning rate may decay too rapidly, causing the training to stop prematurely. The cosine annealing strategy varies the learning rate in the shape of a cosine curve - the learning rate starts high, decreases toward a minimum and then cycles back to a high value. The cyclical nature of the learning rate variation helps the model explore the loss landscape and escape from suboptimal local minima; however, this strategy does require more training epochs to fully explore the loss landscape.

Cosine annealing presents an improved chance at finding the global minimum of the loss landscape, without introducing additional hyperparameters within the training pipeline at the expense of an increased number of epochs. This was an acceptable trade-off; thus, the cosine annealing strategy was the chosen learning rate scheduler. This strategy was adopted in conjunction with a warm-up phase of the learning rate. A warm-up period allows the model to adjust the internal parameters gently during the initial phase of training by increasing the learning rate linearly from an extremely small value to the initialised value. Thereafter, allowing the cosine annealing scheduler to take effect. The rationale behind the warm-up phase is to prevent instability caused by random weight initialisation. Applying large updates during the initial stages of training, when weights are randomly initialised, can lead to instability - the warm-up phase provides a more stable foundation of weights before the main training begins.

3.8.4 Regularization methods

Regularisation techniques are employed in DNNs to enhance model performance and generalisation. They are crucial in mitigating the risk of overfitting to the training subset of data. The training pipeline and model architecture make use of several regularisation techniques.

L2 regularisation (weight decay) is a technique which penalises large weights within a neural network by adding a penalty term to the loss function that is proportional to the square of the

magnitude of the weights of the model. By including this penalty term in the loss function, the optimiser is then forced to minimise both the primary loss and the model weights. The model is then encouraged to learn more diffuse weights that are less prone to the spurious noise features in the training set that may arise from variations in the input audio. The weight decay equation is shown in Equation 38 below:

$$\mathcal{L}_{\text{reg}}(w) = \mathcal{L}_{\text{original}} + \frac{\lambda}{2m} \sum_{i=1}^{n} w_i^2 \tag{38}$$

where $\mathcal{L}_{\text{reg}}(w)$ is the regularized loss, $\mathcal{L}_{\text{original}}$ is the loss function, λ is the regularization hyperparameter that controls the penalty strength, m is the number of samples in the batch and $\sum w_i^2$ represents the sum of the squares for every weight w_i.

Dropout, introduced in [33], is a technique where, during training, neurons are randomly selected and do not contribute to the forward pass. Their weights are also not updated during the backpropagation of the error. The benefit of this technique is that it prevents neurons from excessively co-adapting with each other. Dropout forces neurons to learn robust features - if a neuron that is responsible for a specific harmonic of a violin is dropped out, the surrounding neurons are forced to identify the violin based on the other harmonic characteristics of the feature map. The dropout equation is shown in Equation 39 below:

$$a' = \frac{a \odot d}{p} \quad \text{where} \quad d_i \sim \text{Bernoulli}(p) \tag{39}$$

where a' is the output activation vector after applying dropout, a is the original activation vector, d is a binary mask vector where each element d_i is drawn from a Bernoulli distribution, p is the keep probability and $\odot$ denotes the element-wise Hadamard product.

Batch normalisation standardises the inputs to each hidden layer by calculating the mean and the variance of the activations from the output of the previous layer and then normalising the activations to zero mean and unit variance. The values within the intermediate feature maps can vary significantly between different audio files, based on the recorded volume of the mix. Batch normalisation stabilises the learning process by reducing the internal covariate shift introduced, by training with varying intermediate values. The batch normalisation equation is shown in Equation 40 below:

$$y = \frac{x - \mathbb{E}[x]}{\sqrt{\text{Var}[x] + \varepsilon}} \cdot \gamma + \beta \tag{40}$$

where y is the normalized output, x is the input from a mini-batch, $\mathbb{E}[x]$ is the mean of the mini-batch, $\text{Var}[x]$ is the variance of the mini-batch, ε is a small constant for numerical stability, and γ and β are learnable scale and shift parameters.

Instance normalisation works in a similar fashion to batch normalisation; however, instead of normalising according to the statistical properties of the minibatch, it normalises according to the statistical properties of each individual sample. Using instance normalisation at the input layer helps the model become invariant to the volume (amplitude) and dynamic range (contrast) of each sample.

Gradient clipping is a regularisation technique that is employed to prevent the exploding gradient problem. Supposing that a large error was computed, the backpropagated gradient would be

excessively large. This would lead to unstable training and large updates to the model's internal parameters as the optimiser would be taking large steps - oscillating around the optimum solution. Gradient clipping mitigates this by placing a limit on the value of the gradients, should the L2-norm of the gradients exceed a certain threshold.

$$\mathbf{g}_{\text{clipped}} = \begin{cases} \frac{C}{\|\mathbf{g}\|_2}\mathbf{g} & \text{if } \|\mathbf{g}\|_2 > C \\ \mathbf{g} & \text{otherwise} \end{cases} \tag{41}$$

where $\mathbf{g}_{\text{clipped}}$ is the gradient vector after clipping, $\mathbf{g}$ is the original gradient vector, C is the maximum norm threshold and $\|\mathbf{g}\|_2$ is the L2 norm of the original gradient vector.

Label smoothing is a regularisation technique that prevents the model from becoming overconfident in predictions by using softer labels rather than one-hot encoded labels. This decreases the magnitude of the computed error and the gradients used for adjusting the internal model parameters. The model will be encouraged to be less confident; however, the model may generalise better.

$$y_i' = y_i(1-\alpha) + (1-y_i)\alpha \tag{42}$$

where y_i' is the new smoothed label for the i-th instrument, y_i is the original hard binary label and α is the smoothing hyperparameter.

The configured hyperparameters for each of the employed regularisation techniques are presented in section 3.8.5.

3.8.5 Hyperparameter optimization

This subsection uses Frazier [34] as a guide to Bayesian optimisation and to understand fundamental theory.

Hyperparameter optimisation was conducted to maximise the performance of the deep learning model. Methods for hyperparameter optimisation include: grid search, random search, genetic algorithms and Bayesian optimisation. Grid search involves defining a set of all possible values for each hyperparameter within the scope of optimisation and trying all combinations of these hyperparameter values. Whilst exhaustive to the degree of the defined hyperparameter range, it is also extremely computationally expensive. Random search works in a similar fashion to grid search; however, it samples random combinations of hyperparameters and may potentially find a reasonable solution with far fewer evaluations. Genetic algorithms use the process of natural selection on the set of hyperparameters, using concepts such as mutation and crossover to find an optimal combination of hyperparameters over a set of generations. Bayesian optimisation is the "*de facto*" method for hyperparameter optimisation within deep learning applications - it works by building a probabilistic model (surrogate model) of an objective function, then using information from this model to determine the set of hyperparameters which will likely yield an improvement in the objective function. Bayesian optimisation is ideal for deep learning applications as it tends to find the optimal solution with far fewer trials compared to grid and random forest search, particularly when the cost per data point is expensive.

The collection of hyperparameters that are to be optimised can be considered a Gaussian process (GP), as each may be modelled as a random variable, and the joint distribution of these variables

is Gaussian. Thus, for any set of inputs to the hyperparameter set, the corresponding output variables are considered random variables that follow a multivariate normal distribution. GPs contain two key components: the mean function and the covariance function (kernel function). The mean function is defined as the expected value that the surrogate model will produce at any input. Typically, this value is assumed to be a constant or 0, as in most scenarios, there is not enough prior knowledge about the surrogate model's trend. The kernel function describes the relationship between the output values for different input values, encoding the properties of the function, such as the variability between a set of variables. Supposing the kernel function is used for two different inputs as shown in Equation 43 below:

$$\Sigma_0(x,x') = \mathbb{E}[(f(x)-\mu_0(x))(f(x')-\mu_0(x'))] \tag{43}$$

where $\Sigma_0(x,x')$ is the kernel function with inputs x and x', $f(x)$ is the objective function being modeled, and $\mu_0(x)$ is the prior mean function representing the expected value of $f(x)$ before observing any data.

The output of the kernel function can be thought of as a measure of correlation, with higher values indicating strong correlation and lower values indicating weaker correlation. There exist several different kernel functions, with the most common being the radial basis function kernel and the Matérn kernel shown in Equations 44 and 45, respectively, below:

$$\Sigma_0(x,x') = \alpha_0 \exp\left(-\sum_{i=1}^{d} \alpha_i (x_i - x_i')^2\right) \tag{44}$$

where $\Sigma_0(x,x')$ is the radial basis function kernel, d is the dimensionality of the input space, and $\alpha_{0:d}$ are the hyperparameters of the kernel; α_0 controls the overall variance, and each α_i for $i > 0$ controls the length-scale for the corresponding input dimension x_i. A key property of the radial basis function kernel is that it assumes that the objective function is differentiable at all points.

$$\Sigma_0(x,x') = \alpha_0 \frac{2^{1-\nu}}{\Gamma(\nu)} \left(\frac{\sqrt{2\nu}\|x-x'\|}{\ell}\right)^{\nu} K_\nu \left(\frac{\sqrt{2\nu}\|x-x'\|}{\ell}\right) \tag{45}$$

where $\Sigma_0(x,x')$ is the Matérn kernel, α_0 and ℓ are hyperparameters controlling variance and length-scale, $\Gamma(\cdot)$ is the gamma function, $K_\nu(\cdot)$ is the modified Bessel function, and the hyperparameter $\nu > 0$ controls the smoothness of the function. The ability to vary the smoothness of the kernel function allows for a more realistic model of the objective function - there are regions where the objective function may not be differentiable or have non-smooth, sharp gradients.

The prior mean function and prior covariance function determine the prior distribution over the multivariate Gaussian distributions, representing the belief before any trials. After conducting n trials, $f(x_{1:n})$ observations are obtained at the input points of $x_{1:n}$. Bayes' theorem is applied to update the current belief with the newly observed data. Since the prior is Gaussian distributed, the resulting posterior distribution is also Gaussian distributed, as shown in Equation 46:

$$f(x)|f(x_{1:n}) \sim \mathcal{N}(\mu_n(x), \sigma_n^2(x)) \tag{46}$$

where the posterior belief about the function value $f(x)$ at a new point x, given the n trials $f(x_{1:n})$, follows a gaussian distribution with posterior mean $\mu_n(x)$ and posterior variance $\sigma_n^2(x)$

The mathematical definition of the posterior mean is shown in Equation 47 below:

$$\mu_n(x) = \mu_0(x) + \Sigma_0(x, x_{1:n})\Sigma_0(x_{1:n}, x_{1:n})^{-1}(f(x_{1:n}) - \mu_0(x_{1:n})) \tag{47}$$

where $\mu_n(x)$ is the posterior mean function, the updated prediction for $f(x)$, $\mu_0(x)$ is the prior mean, $\Sigma_0(x, x_{1:n})$ contains the prior covariances between the new point x and all previously evaluated points $x_{1:n}$, $\Sigma_0(x_{1:n}, x_{1:n})$ contains the prior covariances among the evaluated points and $f(x_{1:n})$ contains the observed function values.

The mathematical definition of the posterior variance is shown in Equation 48 below:

$$\sigma_n^2(x) = \Sigma_0(x, x) - \Sigma_0(x, x_{1:n})\Sigma_0(x_{1:n}, x_{1:n})^{-1}\Sigma_0(x_{1:n}, x) \tag{48}$$

where $\sigma_n^2(x)$ is the posterior variance, which quantifies the uncertainty in the prediction at point x, $\Sigma_0(x, x)$ is the prior variance at x, the negative term represents the reduction in variance due to the information gained from the observed data points $f(x_{1:n})$.

An acquisition function is constructed using the posterior mean and variance of the GP, which is used to determine the point which minimises the objective function. The acquisition function typically requires a few trials to understand the loss landscape, referred to as the exploitation phase. As more trials are completed, the acquisition function can find promising regions which may contain the global minimum. The acquisition can then further explore these potential regions, referred to as the exploitation phase.

The most widely adopted acquisition function is the expected improvement function, shown in Equation 49 below:

$$\mathrm{EI}_n(x) = (\mu_n(x) - f_n^*)\Phi\left(\frac{\mu_n(x) - f_n^*}{\sigma_n(x)}\right) + \sigma_n(x)\phi\left(\frac{\mu_n(x) - f_n^*}{\sigma_n(x)}\right) \tag{49}$$

where $\mathrm{EI}_n(x)$ is the expected improvement at point x, f_n^* is the best acquisition value determined within the current landscape, $\Phi(\cdot)$ is the cumulative distribution function and $\phi(\cdot)$ is the probability density function of the standard normal distribution.

The rationale of this function is that it quantifies the expected improvement of a point using the probability distribution of the surrogate model. If the expected improvement is greater than zero, the optimiser conducts a trial using the hyperparameters of the point. If the actual improvement of the trial does not resemble that of the expected improvement, the trial is then pruned.

`Optuna` was used to perform hyperparameter optimisation of the deep learning model hyperparameters. By default, the surrogate model and the acquisition function are based on the tree-structured Parzen estimator (TPE) algorithm [35] . The difference between the TPE and GP models is that a GP model attempts to model the objective function - plotting a distribution of the validation loss given a set of hyperparameters. TPE plots the distribution of hyperparameters given the validation loss of hyperparameter combinations, and splits the conducted trials into two groups based on their validation loss. The algorithm creates density estimates $l(x)$ and $g(x)$ of the good and bad groups respectively, and then uses a kernel density estimator, similar to a kernel function, to build a probability distribution for each group. The acquisition function of TPE models the ratio of $l(x)$ over $g(x)$, thereafter, selects a value of x which is probable in $l(x)$ and improbable in $g(x)$. This procedure mimics the core idea of finding a set of hyperparameters which will yield an expected improvement. TPE is considered a more efficient solution than GP methods in cases where the search space is of high dimensionality or has complex relationships between the hyperparameters. The search configuration used and obtained optimal values are

shown in Table 13 below:

Table 13. Hyperparameter search space configuration and the optimal values found.

Parameter Name	Search Range	Scale	Optimal Value
`lr`	$[1.0\times10^{-6}, 1.0\times10^{-2}]$	Logarithmic	0.0021963789036605324
`weight_decay`	$[1.0\times10^{-7}, 1.0\times10^{-2}]$	Logarithmic	$2.1503642094991787\times10^{-5}$
`dropout_piano`	$[0.1, 0.8]$	Linear	0.14901739982187723
`dropout_strings`	$[0.1, 0.8]$	Linear	0.34426012250150945
`dropout_woodwinds`	$[0.1, 0.8]$	Linear	0.17159604789749142
`dropout_brass`	$[0.1, 0.8]$	Linear	0.40464834673404443

The learning rate and weight decay are the two non-architectural hyperparameters within the optimisation scope. The relationship between these regularisation hyperparameters significantly influences the learning process. The relationship between these hyperparameters can be modelled as a three-dimensional surface plot with respect to the validation loss to understand the loss landscape, and as a contour plot to understand the distribution of the completed trials within the Bayesian optimisation experiment. The surface plot of the validation loss landscape and contour plot of the validation loss landscape are shown in Figures 18 and 19 respectively, below:

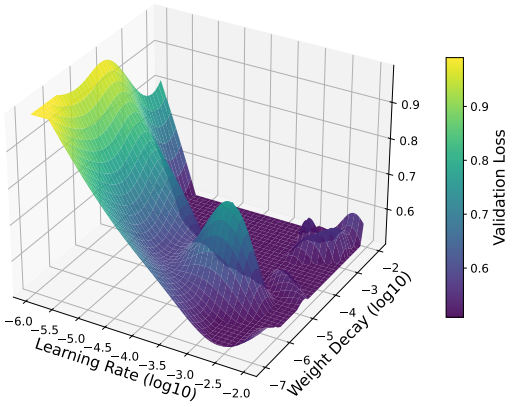


Figure 18. Surface plot of the validation loss landscape.

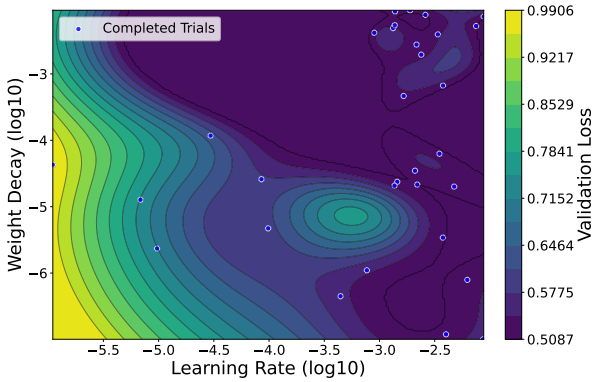


Figure 19. Contour plot of the validation loss landscape.

It can be seen that there is a cliff of high validation loss toward the far left of both plots. A sharp peak in validation loss can be seen in the region with log learning rate of −4 to −3 and a log weight decay between −5.5 and −4.5. The completed trials are scattered around this peak; however, the majority of points lie in the top left of the contour plot. This indicates that the optimiser identified the sharp peak and found a pit of lower validation loss to exploit. The slope of the line of best fit of the relationship between validation loss and each parameter can indicate the importance of each parameter. The graphs of learning rate versus validation loss and weight decay versus validation loss are shown in Figures 20 and 21, respectively, below:

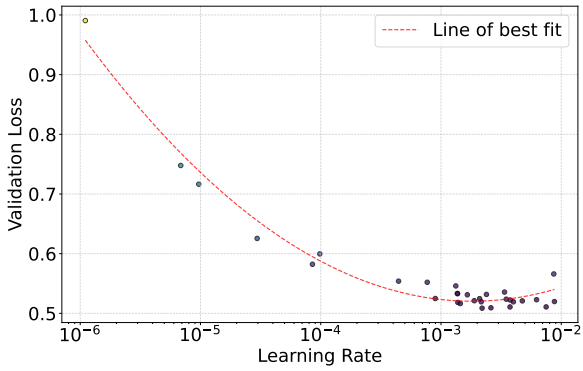


Figure 20. Graph of learning rate vs. validation loss.

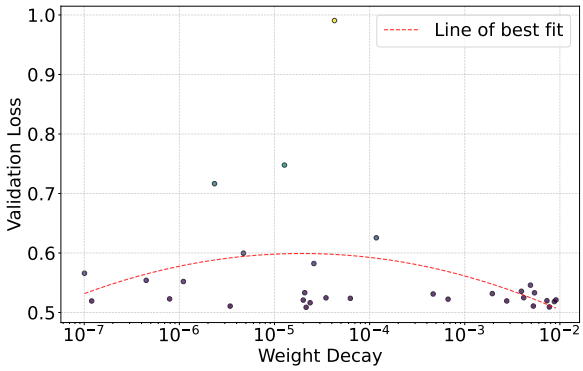


Figure 21. Graph of learning rate vs. validation loss.

The slope of the learning rate with respect to the validation loss shows a negative gradient, indicating that varying the learning rate impacts the validation loss. However, the slope of weight decay with respect to the validation loss is somewhat flat, indicating that weight decay does not carry significant importance. The relationship of the dropout parameters with respect to the validation loss is shown in Figures 43a - 43d in Section 6.4 of the Appendix.

The updated activity detection metrics of the multi-head VGG model are shown in Table 14 below:

Table 14. Performance of the multi-head VGG CNN with optimised hyperparameters across instruments.

Instrument	F1	Precision	Recall
Piano	0.8438	0.7473	0.9689
Violin	0.8656	0.7929	0.9530
Cello	0.8301	0.7596	0.9150
Viola	0.8276	0.7760	0.8866
Contrabass	0.4544	0.3821	0.5605
Clarinet	0.7202	0.6865	0.7574
Bassoon	0.8063	0.7437	0.8805
Oboe	0.7711	0.6921	0.8703
Flute	0.6915	0.6063	0.8047
French Horn	0.6398	0.6643	0.6170
Trumpet	0.8434	0.8882	0.8030
Macro Average	**0.7540**	**0.7036**	**0.8197**

3.9 Onset detection

3.9.1 Analysis of SuperFlux method

The SuperFlux algorithm, introduced in [36], improves upon the spectral flux onset detection algorithm. The improvement is the usage of a maximum filter to reduce the number of false positives in tremolo and vibrato playing styles. The first step of the algorithm is the computation of the log-spectrogram of the input audio. A maximum filter is then applied on the frequency axis of each time frame, making the value for any division the maximum between itself and its neighbours. This is shown in Equation 50 below:

$$X_{\text{max-filt}}(n,m) = \max(X(n,m-1),X(n,m),X(n,m+1)) \tag{50}$$

where X is the input spectrogram and $X_{\text{max-filt}}$ is the filtered output.

The effect of the maximum filter is that spectral peaks of onsets are now smeared; thus, the fluctuations in amplitude and frequency, attributed to tremolo and vibrato, are smoothened out. The core logic of the spectral flux is then applied - the energy change of each frequency bin per time frame is computed. A half-wave rectification is applied to discard negative energy changes. The application of the spectral flux logic is shown in Equation 51 below:

$$SF^*(n) = \sum_m H(X_{\text{max-filt}}(n,m) - X_{\text{max-filt}}(n-\mu,m)) \tag{51}$$

where $H(\cdot)$ is the half-wave rectifier, μ is the time lag, and $X_{\text{max-filt}}$ is the maximum-filtered spectrogram.

If the delta of the spectral energy across a specified number of time frames for a specific frequency bin exceeds a certain threshold, an onset is triggered. A design concept using the SuperFlux method in parallel to the CNN's predictions was considered, and is shown in Figure 22 below:

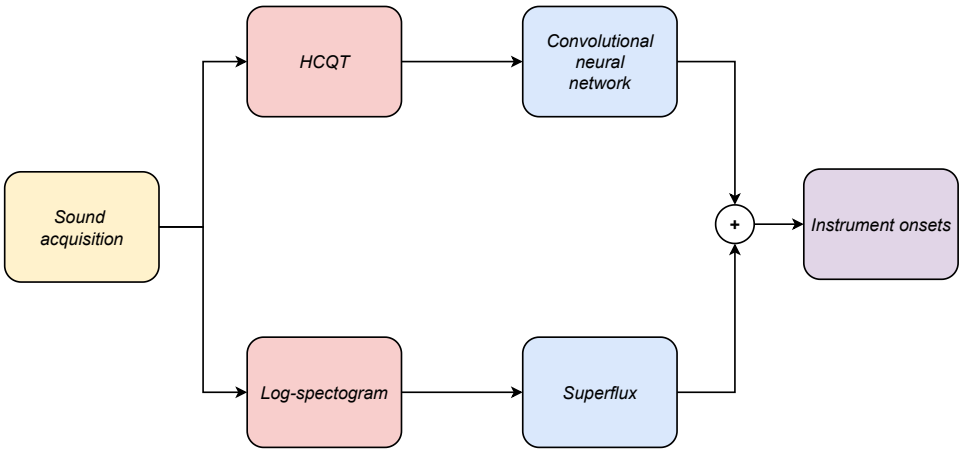


Figure 22. Flow diagram of superflux in parallel to CNN design concept.

The rationale is that the detected onsets of SuperFlux could be correlated to passages of increased activation from the CNN output. This design concept was discarded owing to the additional computational demands that would be introduced. An STFT would need to be performed, a

log spectrogram would need to be computed, and the SuperFlux algorithm would then need to be applied. The potential increase in onset detection accuracy would not justify the additional weight, time, and computational load. Additionally, most of the information obtained from the algorithm would be discarded as the entrance of an instrument is of interest, not the note onsets.

3.9.2 Analysis of probability flux method & implementation

Timbre is not a tangible property of an instrument - it cannot be measured, unlike pitch and loudness. Müller [1] discusses that to differentiate the note being played at the same loudness of two different instruments, one should review the attack, delay, sustain and rest (ADSR) envelope of the note being played. A schematic view of the ADSR envelope is shown in Figure 23 below:

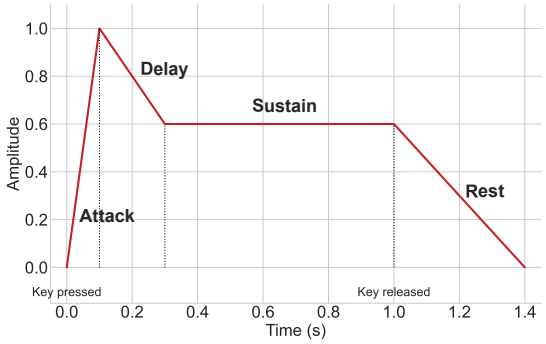


Figure 23. Schematic view of ADSR envelope adapted from Müller [1].

The attack phase consists of a short burst of energy, often called a transient of the note. The energy produced then stabilises in the delay phase and remains constant for a brief period in the sustain phase. The energy is then released in the delay phase. A naive, direct approach utilising the output probabilities of the CNN in correlation with the ADSR envelope of each instrument is proposed to determine instrument onsets. The rationale is that the ADSR envelope of each instrument is different, and this would reflect in the frame-wise activations produced by the CNN. A percussive instrument would exhibit a powerful and rapid attack phase with a low sustain in comparison to a string instrument, which would exhibit a gradual attack phase with a high sustain. The rate of change, flux, of the activations is used to determine the presence of an onset.

A timeline of instrument probabilities is constructed from the output of the CNN and used as input to two classes: `ActivityPeriodFinder` and `FluxPeriodFinder`. The activity period finder performs macro analysis on the binarised timelines to determine if an instrument produced sustained activity in the mix - this is done to filter out spurious detections. There are two crucial parameters to the operation of the activity period finder: `MIN_SILENCE_MS_TO_FILL`, which fills spurious gaps of silence and `MIN_ACTIVITY_MS`, which removes brief positive detections. The CNN's confidence may drop for a brief number of frames during a sustained period of activity, hence the need to fill gaps of silence. Conversely, the CNN may briefly produce predictions for an instrument which is not present, hence the need to remove spurious detections. The functionality of the activity period finder is illustrated by Figure 24 on the subsequent page.

Once all potential periods of sustained activity have been identified, a significance test is performed. The rationale behind the significance test is that certain instruments typically have shorter passages in classical ensembles, whilst others have longer. Each period of sustained

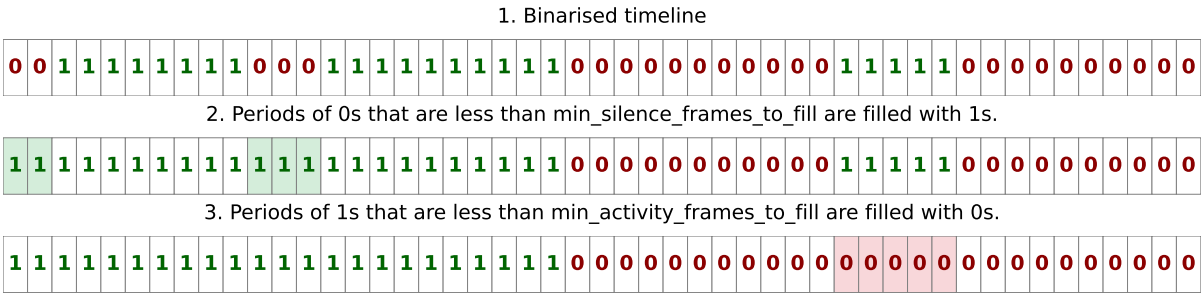


Figure 24. Illustration of activity period finder functionality.

activity is assigned a significance score and deemed significant according to the criteria shown in Equation 52 below:

$$\left(\bar{p}_{\text{period}} \cdot \frac{D_{\text{period}}}{D_{\text{total}}}\right) \geq \theta_{\text{sig}} \tag{52}$$

where $\bar{p}_{\text{period}}$ is the average detection probability during an activity period, D_{period} is the duration of that period, D_{total} is the total clip duration, and θ_{sig} is the minimum significance score threshold.

The flux period finder performs microanalysis to determine periods of rapid probability increase. There are three crucial parameters to the operation of the flux period finder: `LAG_FRAMES` which determines the delay between the current frame and the look-back frame, `FLUX_THRESHOLD` which is the threshold used to determine a high-flux event and `MIN_FLUX_PERIOD_FRAMES` which determines the length of the probability increase and used to filter out gradients from spurious detections. The calculation of the probability flux is shown in Equation 53 below:

$$\Phi(t) = \max\left(0, p(t) - p(t-\mu)\right) \tag{53}$$

where $\Phi(t)$ is the rectified probability flux, $p(t)$ is the instrument's detection probability at time frame t, and μ is the time lag in frames.

The functionality of the flux period finder is illustrated in Figure 25 below:

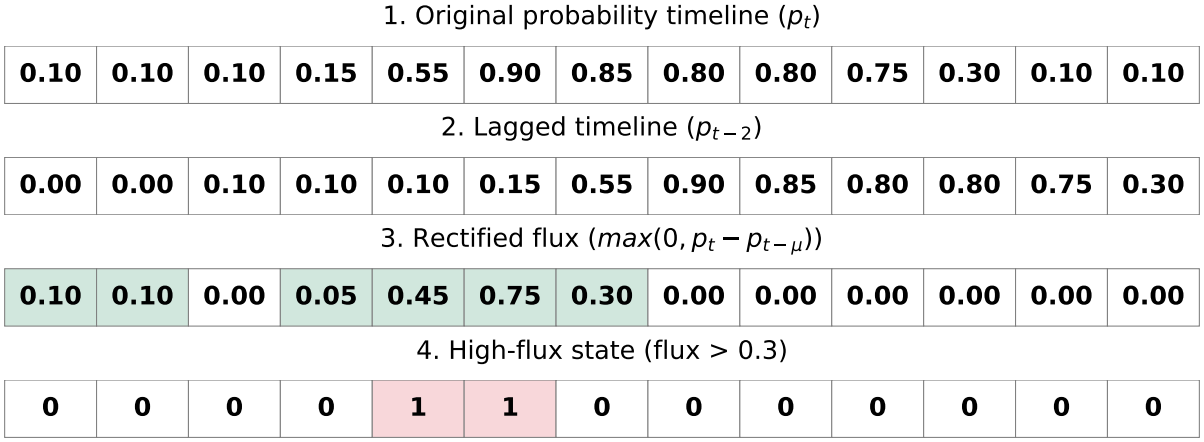


Figure 25. Illustration of flux period finder functionality.

Contextual validation is then performed by determining the periods of high flux and significant

periods that overlap and marking them as potential onsets. The final output of the system is the earliest candidate onset.

3.9.3 Parameter optimisation

Onset detection evaluation on the validation set takes ≈ 520 ms. Thus, a grid search was employed for parameter optimisation as the cost per data point is relatively inexpensive. The search configuration of the grid search is shown in Table 15 below:

Table 15. Onset detector parameter search space configuration.

Parameter Name	Search Range	Step
LAG_FRAMES	$[1, 7]$	1
FLUX_THRESHOLD	$[0.06, 0.18]$	0.01
MIN_FLUX_PERIOD_FRAMES	$[1, 7]$	1
MIN_ACTIVITY_MS	$[50, 750]$	50
MIN_SILENCE_MS_TO_FILL	$[300, 800]$	100
MIN_SIGNIFICANCE_SCORE	$[0.008, 0.16]$	0.001

The optimal parameters per instrument found by the grid search are shown in Table 16 below:

Table 16. Optimal parameter values found for the onset detector for each instrument.

Parameter	Piano	Violin	Cello	Viola	Contrabass	Clarinet	Bassoon	Oboe	Flute	French Horn	Trumpet
LAG_FRAMES	2	2	2	2	2	2	2	2	2	2	2
FLUX_THRESHOLD	0.16	0.09	0.16	0.16	0.05	0.08	0.08	0.08	0.07	0.08	0.04
MIN_FLUX_PERIOD_FRAMES	2	2	2	2	2	2	2	2	2	2	2
MIN_ACTIVITY_MS	150	150	150	150	150	150	150	150	150	150	50
MIN_SILENCE_MS_TO_FILL	500	400	400	400	700	600	700	500	700	500	800
MIN_SIGNIFICANCE_SCORE	0.02	0.01	0.03	0.03	0.01	0.015	0.05	0.015	0.11	0.01	0.006

3.10 System deployment

3.10.1 Audio capture implementation

Audio capture on the Jetson Nano was implemented using advanced linux sound architecture (ALSA), a software layer embedded in the Linux kernel. This interface uses pulse code modulation (PCM) for converting analogue signals into digital signals. A device handle to the microphone was created, where the capture parameters were initialised. The following recording parameters were used: a sample rate of 44.1 kHz, a channel count of 1 for mono configuration, a 16-bit bit depth and interleaved access mode.

For a mono configuration, a single sample is a single frame. Frames are read into periods, which are transferred between the kernel layer and the software at regular intervals. The kernel layers store these frames using a cyclic buffer and signal an interrupt if an error occurs or the period timer has run out. A memory block is allocated by the sound acquisition script, which receives

the frames sent by the kernel layer. Once transmission from the kernel layer has stopped, all 16-bit values are cast to floats, which are 32 bits, and then normalised to convert from integer range to floating-point range.

3.10.2 Analysis of recording domain

To obtain a better understanding of performance in real-world contexts, the frequency response of the speaker microphone system was measured using REW [37], and shown in Figure 26 below:

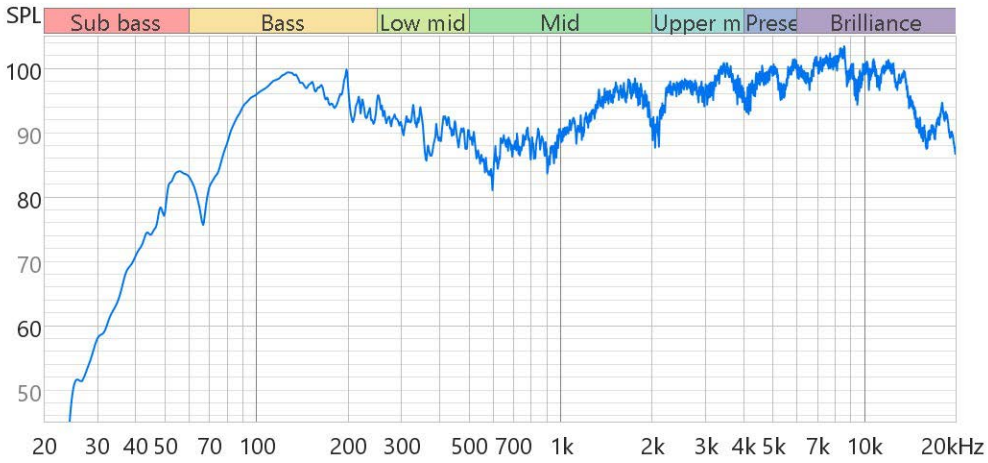


Figure 26. Frequency response of speaker microphone system.

There is a steep drop-off in the frequency response below 80 Hz, which is typical of nearfield monitor speakers, as the small woofers are not capable of reproducing sub-bass frequencies. In the bass region, the response rises to a peak of around 100 dB SPL. Within this region, the response is relatively controlled compared to other regions, which may be a result of the speaker being designed to enhance the bass response. The low-mid to mid range is the "scooped" region of the response, which is a result of the comb filtering effect caused by reflections from desk surfaces, walls and the ceiling. This presents significant non-linearity and distortion for instruments playing within this frequency range, altering the timbre of the instrument and the feature representation thereof. There is a sharp drop off at 2 kHz as it is the cross-over region between the woofer and tweeter of the speaker. The upper-mid to brilliance region is jagged and can possibly be attributed to reflections off the surfaces in the recording environment. In order to replicate the distortion introduced by the speaker microphone system, the augmentation procedure outlined in Section 3.7 was applied with the impulse response of the speaker microphone system. The Pearson correlation and cosine similarity of the HCQT of a test track under varying recording domains were calculated and are shown in Table 17 on the subsequent page.

The augmented data obtains a Pearson correlation and cosine similarity to the re-recorded data of 0.8933 and 0.9055, respectively. Whilst these results appear promising, there was no improvement in model performance with the augmented data. Thus, the ≈ 0.1 disparity in both Pearson correlation and cosine similarity indicates a significant domain mismatch. The following hypothesis is posed: Under varying recording conditions, the timbre of an instrument is altered. The feature representation of the instrument is also altered, thus rendering the learned features of the deep neural network unsuitable for the recording domain. The results from the synthetic data

experiment, shown in Table 11 support this hypothesis. It is for this reason, the entire dataset was re-recorded.

Table 17. HCQT similarity metrics under varying recording domains.

Comparison	Pearson correlation coefficient	Cosine similarity
MusicNet vs. re-recorded	0.8199	0.8406
Augmented MusicNet vs. re-recorded	0.8933	0.9055
MusicNet vs. augmented MusicNet	0.9209	0.9300

The Pearson correlation coefficient provides a measure of linear correlation between the two HCQTs, indicating how closely the amplitude values vary. The cosine similarity provides a measure of angular similarity between the HCQTs, indicating whether the spectral patterns are similar.

4. Results

4.1 Summary of results achieved

Intended outcome	Actual outcome	Location in report
Core mission requirements and specifications		
A minimum averaged F1 score of 0.75 across all instruments must be achieved, using a temporal collar of 100 ms applied to the ground truth instrument onset events.	A macro F1-Score of 0.743 across all instruments was measured, using a temporal collar of 100 ms applied to ground truth onset events.	Qualification Test 1
A minimum averaged precision score of 0.77 across all instruments must be achieved, using a temporal collar of 100 ms applied to the ground truth instrument onset events.	A macro precision score of 0.741 across all instruments was measured, using a temporal collar of 100 ms applied to ground truth onset events.	Qualification Test 2
A minimum averaged recall score of 0.73 across all instruments must be achieved, using a temporal collar of 100 ms applied to ground truth instrument onset events.	A macro recall score of 0.768 across all instruments was measured, using a temporal collar of 100 ms applied to ground truth onset events.	Qualification Test 3
The total processing time of the polyphonic mix, from when the signal is acquired to when the results are displayed, must not exceed 2x the duration of the polyphonic mix.	An average processing factor of 1.81 across 7 polyphonic mixes was measured.	Qualification Test 4
Field condition requirements and specifications		
The application must function in environments which have not been acoustically treated. The polyphonic audio may be subject to mild ambient noise and reverberation present in the environment.	The application was tested on re-recorded data which capture the mild ambient noise and reverberation present in the environment	Qualification Tests 1, 2, and 3

Table 18. Summary of results achieved.

4.2 Qualification tests

4.2.1 Design of evaluation software

Qualification tests 1 - 3 employ the custom recording and alignment script. The recording script uses the `playrec` method from the Sounddevice library in Python to play and capture audio simultaneously; however, it cannot account for hardware latency differences between the microphone and speaker. The alignment script computes the STFT of the original and re-recorded audio file, finds the total energy in each frame to obtain the spectral envelope of each audio file, and then performs correlation of the two spectral envelopes. The aligned audio is then trimmed at either the beginning or end to match the duration of the original audio.

The definition of the prediction outcomes is constant for Qualification tests 1 - 3. A true positive (TP) occurs when the application correctly identifies the presence of an instrument, and the predicted onset time of the instrument is within the specified onset tolerance window of the ground truth annotation. A false positive (FP) occurs when the application either incorrectly identifies the presence of an instrument or correctly identifies the instrument present, however, with a predicted onset time that is not within the onset tolerance window of the ground truth annotation. FPs can be sub-classified into three categories: spurious detection false positive (SDFP), instrument confusion false positive (ICFP) and timing error false positive (TEFP). An SD refers to an FP where the predicted instrument does not exist in the polyphonic mix, an ICFP refers to an FP where the predicted instrument is incorrect, however, the predicted onset time is within the onset tolerance of the annotation and a TEFP refers to an FP where the predicted instrument is correct, however, the predicted onset time is not within the onset tolerance window. A false negative (FN) occurs when the application fails to detect the presence of an instrument which exists in the ground truth annotations. A true negative (TN) occurs for any time segment for which the application predicts that no instrument onset has occurred, and the ground truth annotations indicate that no instrument onset occurred - TNs are not used in the calculation of evaluation metrics as they are practically infinite and carry little meaning. The evaluation script used for Qualification tests 1 - 3 uses the following marking logic, shown in Figure 27 on the subsequent page.

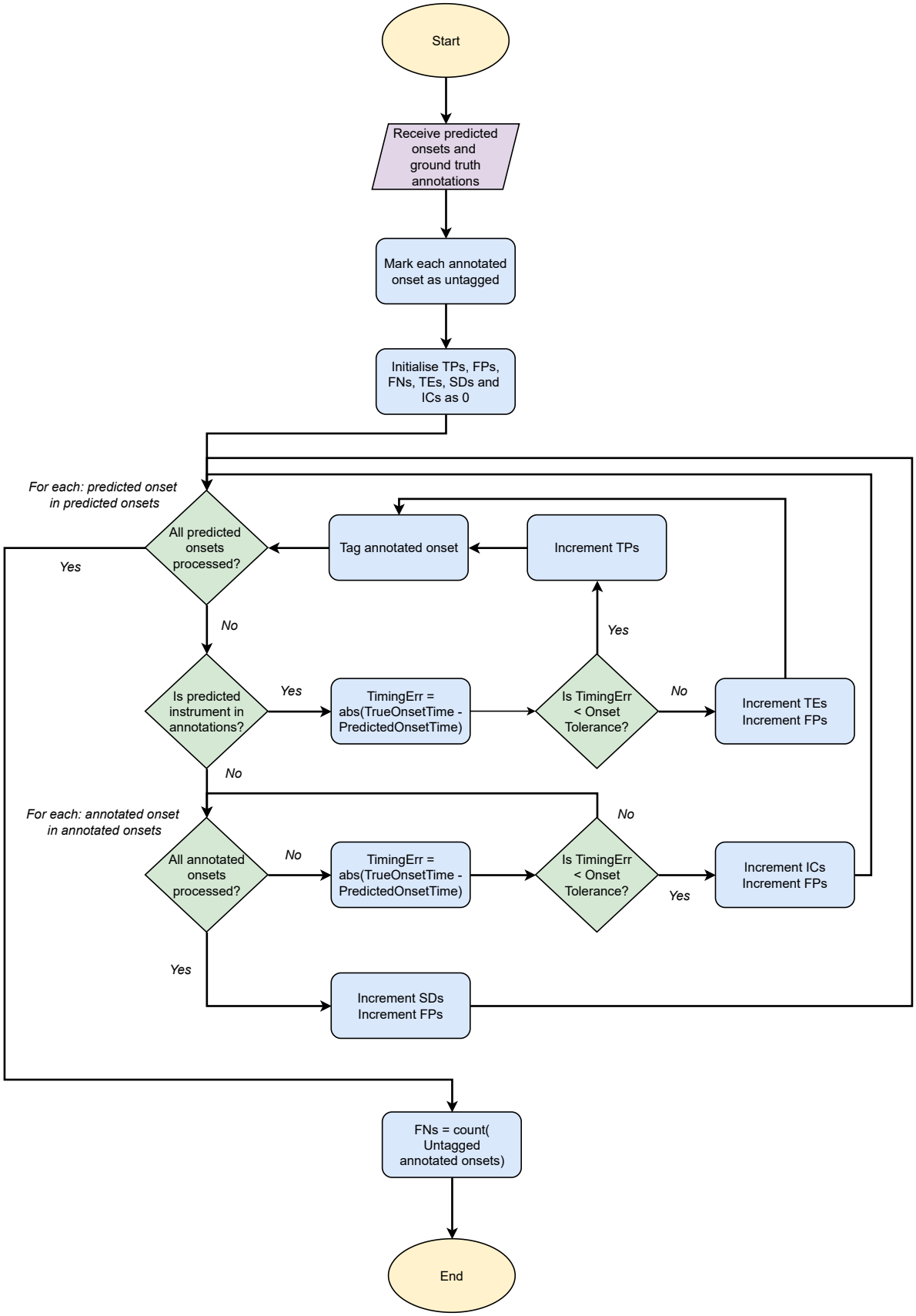


Figure 27. Marking logic used in application evaluation script.

Qualification test 1: Macro F1-score

Objectives of the test or experiment

To measure the application's ability to accurately and reliably detect the presence and onset time of each of the specified classical music instruments. A minimum averaged F1 score of 0.75 across all instruments must be achieved, using a temporal collar of 100 ms applied to ground truth instrument onset events.

Equipment used

1. Samson C01U Pro USB microphone to capture audio
2. Presonus Eris 3.5BT speaker to play audio. Only the left speaker was used as the audio was played in a mono configuration
3. ASUS ROG Zephyrus G15 laptop to perform model evaluation
4. PyCharm integrated development environment to run the evaluation script, recording and audio alignment scripts
5. PyTorch library to run model inference
6. Sounddevice library to record and capture audio
7. Dataset comprising 13 polyphonic audio recordings with corresponding ground truth annotations for instrument name and onset time for each recording
8. Pandas library to extract annotations

Test setup and experimental parameters

The Samson C01U Pro USB microphone was placed at a distance of 30 cm away from the Presonus Eris 3.5BT speaker, such that the diaphragm was facing the centre of the speaker cone. The speaker was also elevated 8 cm to align the vertical centre of the speaker cone with the microphone diaphragm. The microphone and speaker were connected, respectively, to the USB 3.0 and aux ports of the ASUS ROG Zephyrus G15 laptop. An onset tolerance window of 100 ms was used.

Steps followed in the test or experiment

1. The test set of the dataset was re-recorded by running the recording script using PyCharm IDE installed on the laptop, and then aligned with the original recordings using the audio alignment script
2. The HCQT of each of the re-recorded polyphonic audio tracks was computed and segmented into patches of 500 ms
3. The application's CNN performed inference on the patches in batches of 32 to produce frame-wise probability timelines for each of the 11 instruments
4. The frame-wise instrument timelines were used as input to the onset detection subsystem to obtain the predicted instruments present and their associated onset times

5. The annotations for each recording were loaded using pandas stored in target arrays. The predicted instruments present and onset times were compared against the instruments present and onset times in target arrays to determine the TPs, FPs, and FNs (as defined in 4.2.1) for each song using the marking logic shown in Figure 27

The aggregated number of TPs, FPs and FNs per instrument across all polyphonic recordings in the test set was used to compute the precision, recall and F1-score per instrument. The F1-score for an individual instrument, i, is shown in Equation 54 below:

$$F1\text{-}Score_i = 2 \cdot \frac{Precision_i \cdot Recall_i}{Precision_i + Recall_i} \tag{54}$$

The macro F1-score of the application is shown in Equation 55 below:

$$Macro\ F1\text{-}Score = \frac{1}{N} \sum_{i=1}^{N} F1\text{-}Score_i \tag{55}$$

where N is the total number of instruments.

Results or measurements

The results of Qualification Test 1 are shown in Table 19 below:

Table 19. Qualification Test 1 results.

Instrument	F1-Score	Precision	Recall	TP	FP	FN
Piano	0.714	0.714	0.714	5	2	2
Violin	0.933	0.875	1.000	7	1	0
Cello	0.727	0.800	0.667	4	1	2
Viola	0.889	1.000	0.800	4	0	1
Contrabass	0.667	0.500	1.000	1	1	0
Clarinet	0.909	0.833	1.000	5	1	0
Bassoon	0.667	0.667	0.667	2	1	1
Oboe	0.667	0.667	0.667	2	1	1
Flute	0.400	0.500	0.333	1	1	2
French Horn	0.600	0.600	0.600	3	2	2
Trumpet	1.000	1.000	1.000	1	0	0
Macro Average	**0.743**	**0.741**	**0.768**	**35**	**11**	**11**

Observations

1. The macro F1-score does not meet the specification by 0.007
2. Four instruments exceeded the 0.75 threshold: trumpet, violin, clarinet and viola
3. There is significant variation in F1-scores. The minimum score is 0.4, recorded by the flute, and the maximum score is 1.0, recorded by the trumpet

Qualification test 2: Macro precision

Objectives of the test or experiment

To measure the application's ability not to falsely detect the presence and onset time of each of the specified classical music instruments. A minimum averaged precision score of 0.77 across all instruments must be achieved, using a temporal collar of 100 ms applied to ground truth instrument onset events.

Equipment used

1. Samson C01U Pro USB microphone to capture audio
2. Presonus Eris 3.5BT speaker to play audio. Only the left speaker was used as the audio was played in a mono configuration
3. ASUS ROG Zephyrus G15 laptop to perform model evaluation
4. PyCharm integrated development environment to run the evaluation script, recording and audio alignment scripts
5. PyTorch library to run model inference
6. Sounddevice library to record and capture audio
7. Dataset comprising 13 polyphonic audio recordings with corresponding ground truth annotations for instrument name and onset time for each recording
8. Pandas library to extract annotations

Test setup and experimental parameters

The Samson C01U Pro USB microphone was placed at a distance of 30 cm away from the Presonus Eris 3.5BT speaker, such that the diaphragm was facing the centre of the speaker cone. The speaker was also elevated 8 cm to align the vertical centre of the speaker cone with the microphone diaphragm. The microphone and speaker were connected, respectively, to the USB 3.0 and aux ports of the ASUS ROG Zephyrus G15 laptop. An onset tolerance window of 100 ms was used.

Steps followed in the test or experiment

1. The test set of the dataset was re-recorded by running the recording script using PyCharm IDE installed on the laptop, and then aligned with the original recordings using the audio alignment script
2. The HCQT of each of the re-recorded polyphonic audio tracks was computed and segmented into patches of 500 ms
3. The application's CNN performed inference on the patches in batches of 32 to produce frame-wise probability timelines for each of the 11 instruments
4. The frame-wise instrument timelines were used as input to the onset detection subsystem to obtain the predicted instruments present and their associated onset times

5. The annotations for each recording were loaded using pandas stored in target arrays. The predicted instruments present and onset times were compared against the instruments present and onset times in target arrays to determine the TPs, FPs, and FNs (as defined in 4.2.1) for each song using the marking logic shown in Figure 27

The aggregated number of TPs, FPs and FNs per instrument across all polyphonic recordings in the test set was used to compute the precision per instrument. The precision score for an individual instrument, i, is shown in Equation 56 below:

$$Precision_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FP}_i} \tag{56}$$

The macro precision of the application is shown in Equation 57 below:

$$Macro\ Precision = \frac{1}{N} \sum_{i=1}^{N} Precision_i \tag{57}$$

where N is the total number of instruments.

Results or measurements

The results of Qualification Test 2 are shown in Table 20 below:

Table 20. Qualification Test 2 results.

Instrument	Precision	TP	FP
Piano	0.714	5	2
Violin	0.875	7	1
Cello	0.800	4	1
Viola	1.000	4	0
Contrabass	0.500	1	1
Clarinet	0.833	5	1
Bassoon	0.667	2	1
Oboe	0.667	2	1
Flute	0.500	1	1
French Horn	0.600	3	2
Trumpet	1.000	1	0
Macro Average	**0.741**	**35**	**11**

Observations

- The macro precision does not meet the specification by 0.029
- The trumpet and viola achieved perfect precision scores of 1.00
- The contrabass and flute produced the lowest precision scores of 0.50
- The French horn and piano had the highest FP count of 2

Qualification test 3: Macro recall

Objectives of the test or experiment

To measure the application's ability to correctly identify the presence and onset time of each specified classical instrument if present within the recording. A minimum averaged recall score of 0.73 across all instruments must be achieved, using a temporal collar of 100 ms applied to ground truth instrument onset events.

Equipment used

1. Samson C01U Pro USB microphone to capture audio
2. Presonus Eris 3.5BT speaker to play audio. Only the left speaker was used as the audio was played in a mono configuration
3. ASUS ROG Zephyrus G15 laptop to perform model evaluation
4. PyCharm integrated development environment to run the evaluation script, recording and audio alignment scripts
5. PyTorch library to run model inference
6. Sounddevice library to record and capture audio
7. Dataset comprising 13 polyphonic audio recordings with corresponding ground truth annotations for instrument name and onset time for each recording
8. Pandas library to extract annotations

Test setup and experimental parameters

The Samson C01U Pro USB microphone was placed at a distance of 30 cm away from the Presonus Eris 3.5BT speaker, such that the diaphragm was facing the centre of the speaker cone. The speaker was also elevated 8 cm to align the vertical centre of the speaker cone with the microphone diaphragm. The microphone and speaker were connected, respectively, to the USB 3.0 and aux ports of the ASUS ROG Zephyrus G15 laptop. An onset tolerance window of 100 ms was used.

Steps followed in the test or experiment

1. The test set of the dataset was re-recorded by running the recording script using PyCharm IDE installed on the laptop, and then aligned with the original recordings using the audio alignment script
2. The HCQT of each of the re-recorded polyphonic audio tracks was computed and segmented into patches of 500 ms
3. The application's CNN performed inference on the patches in batches of 32 to produce frame-wise probability timelines for each of the 11 instruments
4. The frame-wise instrument timelines were used as input to the onset detection subsystem to obtain the predicted instruments present and their associated onset times

5. The annotations for each recording were loaded using pandas stored in target arrays. The predicted instruments present and onset times were compared against the instruments present and onset times in target arrays to determine the TPs, FPs, and FNs (as defined in 4.2.1) for each song using the marking logic shown in Figure 27

The aggregated number of TPs, FPs and FNs per instrument across all polyphonic recordings in the test set was used to compute the recall score per instrument. The recall score for an individual instrument, i, is shown in Equation 58 below:

$$Recall_i = \frac{\mathrm{TP}_i}{\mathrm{TP}_i + \mathrm{FN}_i} \tag{58}$$

The macro recall score of the application is shown in Equation 59 below:

$$Macro\ Recall = \frac{1}{N}\sum_{i=1}^{N} Recall_i \tag{59}$$

where N is the total number of instruments.

Results or measurements

The results of Qualification Test 3 are shown in Table 21 below:

Table 21. Qualification Test 3 results.

Instrument	Recall	TP	FN
Piano	0.714	5	2
Violin	1.000	7	0
Cello	0.667	4	2
Viola	0.800	4	1
Contrabass	1.000	1	0
Clarinet	1.000	5	0
Bassoon	0.667	2	1
Oboe	0.667	2	1
Flute	0.333	1	2
French Horn	0.600	3	2
Trumpet	1.000	1	0
Macro Average	**0.768**	**35**	**11**

Observations

1. The macro recall score exceeds specification by 0.038
2. Four instruments achieved a perfect recall of 1.00: violin, contrabass, clarinet and trumpet
3. The flute produced the lowest recall score of 0.33
4. There is significant variance in the recall scores, with the lowest score being 0.33 and the highest score being 1.00

5. Piano, cello and French horn had the highest FN count of 2

Qualification test 4: Total processing time

Objectives of the test or experiment

To ensure that the application can process the input polyphonic mix and provide instrument detection results on the deployment hardware within a reasonable wait time. The total processing time of the polyphonic mix, from when the signal is acquired to when the results are displayed, must not exceed 2x the duration of the polyphonic mix.

Equipment used

1. Samson C01U Pro USB microphone to capture audio
2. Presonus Eris 3.5BT speaker to play audio. Only the left speaker was used as the audio was played in a mono configuration
3. NVIDIA Jetson Nano
4. ASUS ROG Zephyrus G15 laptop to interface with the Jetson Nano
5. Visual Studio Code to develop software on the Jetson Nano
6. Time library in the C standard library set to run timers in the application scripts
7. Dataset comprising of seven recordings with durations of 15s, 17.5s, 20s, 22.5s, 25s, 27.5s and 30s.

Test setup and experimental parameters

The Samson C01U Pro USB microphone was placed at a distance of 30 cm away from the Presonus Eris 3.5BT speaker, such that the diaphragm was facing the centre of the speaker cone. The speaker was also elevated 8 cm to align the vertical centre of the speaker cone with the microphone diaphragm. The speaker was connected to the aux port of the ASUS ROG Zephyrus G15 laptop, and the microphone was connected to the USB 3.0 port of the Jetson Nano.

Steps followed in the test or experiment

Each of the seven audio files in the test set was played through the speaker and captured using a microphone. Three timers were initialised in the application script on the Jetson Nano. The first was initialised in the main loop, immediately after the completion of the audio capture function and terminated at the end of the main loop to measure the total processing time once the audio was captured. The second timer was initialised at the beginning of the HCQT function and terminated at the end of the HCQT function to measure the total processing time of the HCQT computation. The third timer was initialised at the beginning of the model forward pass function and terminated at the end of the model forward pass function to measure the total model inference time.

The processing factor for each song was calculated using Equation 60 shown below:

$$\textit{Processing Time Factor} = \frac{T_{\text{processing}}}{T_{\text{audio}}} \tag{60}$$

where $T_{\text{processing}}$ is the total measured processing time from audio acquisition to result display, and T_{audio} is the duration of the original audio recording.

The curve of the processing time factor vs. audio length was plotted by interpolating the measured data points.

Results or measurements

The results of Qualification Test 4 are shown in Table 22 below:

Table 22. Qualification Test 4 results.

Duration (s)	Total time (s)	Model inference time (s)	HCQT time (s)	Processing Factor
15.0	27.200	10.482	16.603	1.81
17.5	32.083	12.800	19.090	1.83
20.0	36.630	14.685	21.697	1.83
22.5	41.086	16.579	24.322	1.83
25.0	45.134	17.109	27.669	1.81
27.5	49.830	19.921	29.502	1.81
30.0	52.943	20.776	31.941	1.76
Average	–	–	–	**1.81**

Observations

1. The average processing time factor is 1.81, which is below the specified maximum limit of 2
2. The minimum processing factor is 1.76, and the maximum processing factor is 1.83
3. HCQT time is greater than model inference time for all data points

5. Discussion

5.1 Critical evaluation of the design

5.1.1 Interpretation of results

A key observation in the results of Qualification Test 1 is the perfect F1-score of the trumpet. While an excellent result, it should be considered in the context of the test set, which only contains a single trumpet sample. Additionally, this single sample occurs in a duet ensemble, presenting a simple test case that would be clear to detect. It cannot be said with confidence that the perfect F1-score is a true reflection of the CNN's ability to detect trumpet. A similar situation occurs for the contrabass, which only has a single sample in the test set; however, within a quintet ensemble. The contrabass achieves perfect recall, indicating that it captured this sample; however, upon closer inspection of the ensemble distribution in Table 27 in Section 6.4 of the Appendix, it is seen that 59% of contrabass samples occur in quintets. This result could be attributed to an ensemble bias or to the usage of a string head in the CNN architecture. It is possible that the features learned in detecting violin, viola and cello assist in detecting the contrabass features as they are part of the same instrument family. The results of these two instruments are evidence that evaluation of the application is constrained by the shortcomings of the dataset constraints indicated in section 3.4.

A correlation between instrument family F1-scores and the instrument family data distribution can be seen in Table 23 below:

Table 23. Macro F1-score and total duration by instrument family.

Instrument Family	Macro F1-score	Total duration (mm:ss)
Strings	0.804	948:37
Brass	0.800	97:23
Percussion	0.714	319:58
Woodwinds	0.661	300:51

Disregarding the inflated score of the trumpet in the brass family, it can be seen that instrument family performance is consistent with data abundance. Violin, viola, cello and piano produced high F1-scores, which can be attributed to the CNN being exposed to an abundance of data samples to learn distinguishing features. The woodwind family shows mixed results, with the flute producing the lowest F1-score of all instruments and the clarinet producing the third-highest F1-score of all instruments. A possible hypothesis for this observation is that the learned woodwind features are biased toward clarinet behaviour - the majority of woodwind data samples occur in wind quartets and wind quintets, and amongst woodwind instruments, the clarinet is the majority class. It is likely that the model has learned to associate a high confidence in clarinet with any form of woodwind behaviour.

The macro F1-score is limited by the per-instrument precision, rather than the recall. The macro

recall exceeds specification by 0.038, whereas the macro precision falls short of specification by 0.029. This is indicative that if an onset exists, the application is likely to detect this onset. Instrument precision scores are affected by the FP count; thus, an analysis of the FPs produced is conducted. A matrix of the ICFP distribution is shown in Figure 28 below:

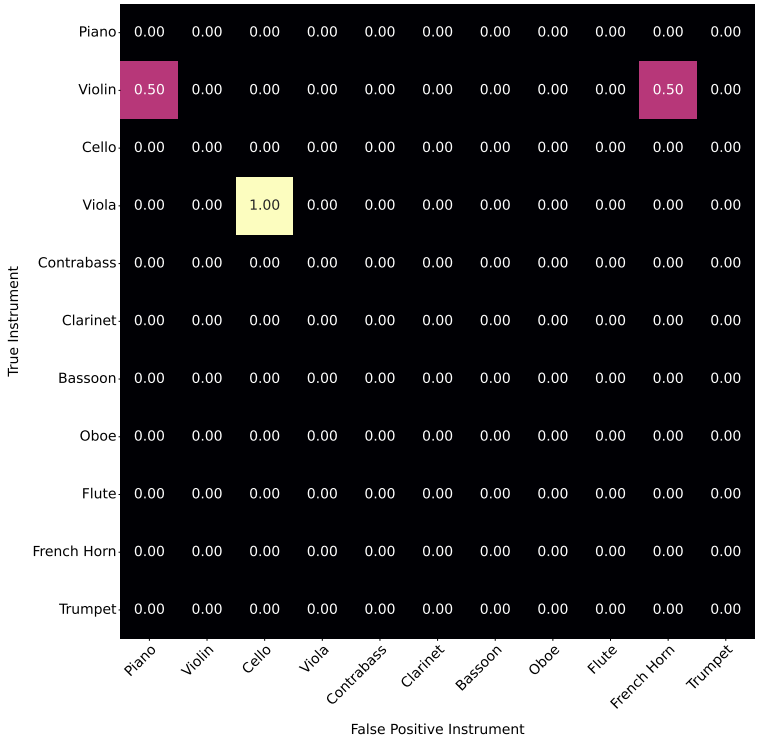


Figure 28. Distribution of instrument confusion false positives.

It is shown that in two instances, the application confused the piano and French horn with the violin. This can be explained by a prediction bias introduced by the violin being the majority class in the dataset, rather than a similarity in instrument features. It is also possible that the piano ICFP could be a result of playing style: a pizzicato playing style which produces a plucked sound that would produce sharp transients, which could be mistaken for the sharp transients of a percussion instrument [38]. Another ICFP instance is the application confusing a cello for a viola. This, however, can be explained by similar features of instruments of the same family. Upon close inspection of the HCQT of the cello and viola, shown in Figures 35c and 41c in Section 6.4, it can be seen that there is a similarity in the harmonic characteristics of these instruments. Additionally, there is an overlap of the typical ranges of these instruments. The typical cello range covers C2-A5, and the typical viola range covers C3-A6. This plot, however, only indicates the ICFPs of the application - a bar graph showing the distribution of all FPs is shown in Figure 29 on the subsequent page.

The precision drop for the woodwind instruments is caused by TEFPs, rather than misclassifications. This means that the model correctly identified the instrument but mistimed the onset. The issue then lies within the onset detection logic, specifically with these instruments' softer, less percussive attack envelopes. The majority of the FPs can be attributed to SDFPs, where the application imagines the presence of an instrument. The cause of the SDFPs can be investigated by an analysis of the instrument activity - Table 14 shows that the low precision, high recall trend appears in the frame-level activity detection results. A matrix plotting the distribution of FPs of

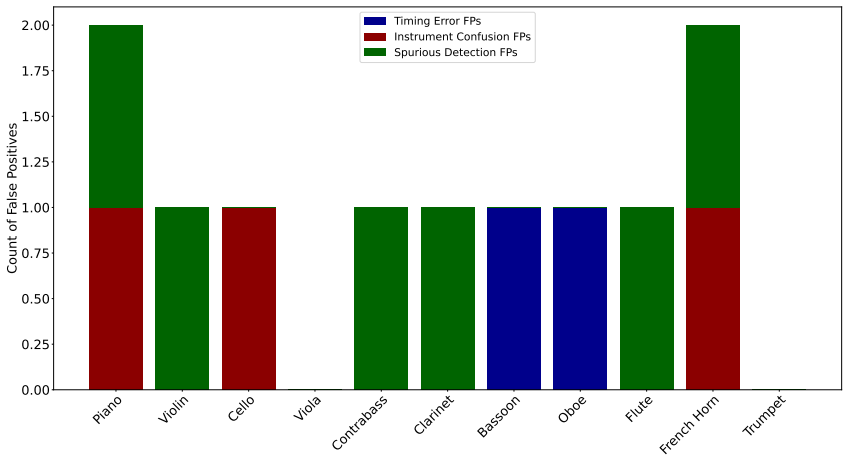


Figure 29. Distribution of all false positives.

the CNNs' predictions is shown in Figure 30 on the subsequent page.

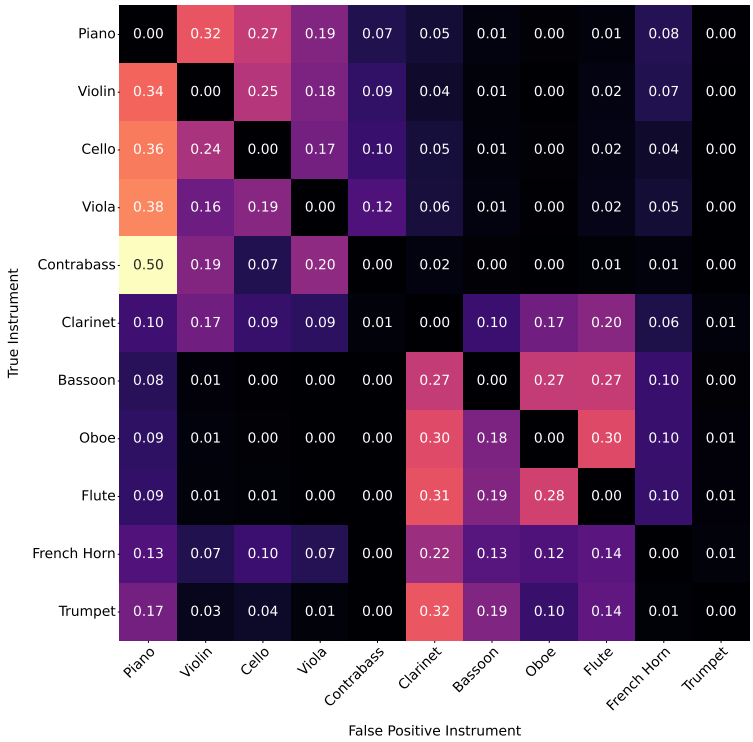


Figure 30. Distribution of false positives for frame-level activity predictions.

The top left and bottom right regions indicate that there is significant co-occurrence bias among instruments in the string family and in the woodwind family. This can be attributed to the recurring ensembles in which these instruments co-occur. Examples include the piano-violin duet, woodwind ensembles and string trios. Such a bias has most likely taught the model to predict that all instruments that occur in an ensemble will also occur in ambiguous situations. The lack of examples of these instruments in smaller ensembles or in isolation has not assisted the model in learning which features discriminate the co-occurring instruments from each other.

The low precision, high recall trend is also a symptom of a design decision in the CNN archi-

tecture. By introducing predictions every 11.61 ms to achieve fine temporal resolution, there is an increase in the number of opportunities for the model to produce false positives. Every prediction has at most 500 ms of context - onsets which occur toward the boundaries of a patch may not have context of before or after the onset occurs. It is common for multi-label classifiers to exhibit the low precision, high recall trend when coupled with BCE loss, as the presence of each label is treated as a binary classification problem. In cases of ambiguity, the model may take the safer route in predicting the presence of an instrument rather than the absence.

In the field of MIR, the term "onset" typically refers to note-level onsets rather than the musical event of an instrument entering a mix. There exists no previous work to benchmark performance in this regard. There does exist previous work for the task of instrument activity detection. Krause et al. [7] conduct a study on hierarchical classification for instrument activity detection in orchestral music recordings. It should be noted that the dataset used in the comparable literature contains orchestral music rather than classical chamber music. Orchestral music typically features deeper instrumentation and higher degrees of polyphony. [7] also perform evaluation on 18 labels, including classification for the instrument family, vocal presence and vocal gender - achieving a macro F1-score of 0.79. A comparison of instrument macro F1-scores between activity detection results is shown in Table 24 below:

Table 24. Comparison of instrument activity detection F1-scores against Krause et al. [7]

Instrument	Krause & Müller F1-Score	Multi-head VGG F1-Score
Piano	N/A	0.844
Trumpet	0.60	0.843
Bassoon	0.75	0.806
Oboe	0.74	0.771
Clarinet	0.72	0.720
Flute	0.69	0.692
Cello	0.87	0.830
Viola	0.84	0.828
Violin	0.91	0.866
French Horn	0.71	0.640
Contrabass	0.87	0.454
Timpani	0.66	N/A

Krause et al. achieve improved results in the string violin, viola, cello, French horn, bassoon and contrabass instrument classes. Performance across woodwinds is comparable, with similar F1-scores for flute, clarinet and oboe. It is also found that instrument family detection, F1-score = 0.82, is more reliable than instrument activity detection, F1-score = 0.79, indicating that FPs caused as a result of instrument co-occurrence are a common challenge for this task. There is a similarity in methodology employed - [7] make use of the HCQT input to a CNN model. Albeit using 252 bins per octave for increased frequency resolution, a patch size of 2.3 seconds and a CNN model with a parameter count of ≈ 1.8 million.

Plots of the relationship between processing factor, HCQT time and model inference time are shown in Figures 31 and 32 on the subsequent page.

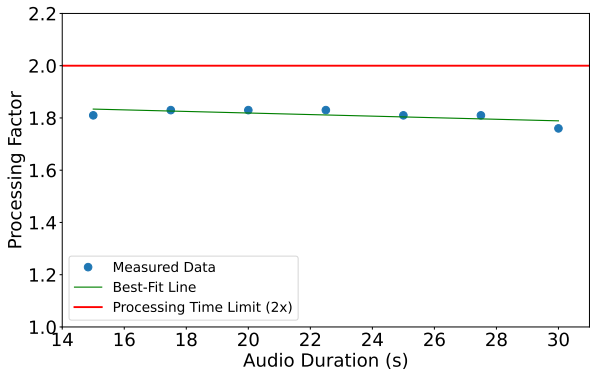


Figure 31. Plot of processing factor against audio duration.

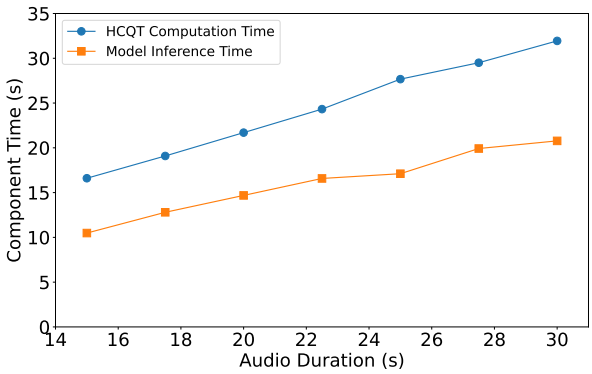


Figure 32. Plot of HCQT and model inference time against audio duration.

The processing factor of the application consistently remains within the acceptable bounds of 2. From Figure 32, the bottleneck in wait time is evident - the computation of the HCQT. On initial thought, this outcome is counterintuitive given the complexity of the CNN model; however, this is a symptom of the repetitive FFT computations used in the direct approach to the HCQT computation.

5.1.2 Critical evaluation

A key design choice was the approach to implementing the CQT algorithm in C. Using the direct method as opposed to the recursive downsampling method did provide simplicity of the implementation; however, the results of Qualification Test 4 do indicate that the HCQT computation time accounts for the majority of the total wait time to process a song.

The numerical differences between the C implementation of the model forward pass and the PyTorch implementation yield a max absolute error on the order of a millionth, and the differences between the C implementation of the CQT computation and the librosa method yield a similarity score of 0.9962. Whilst these error statistics are not significant, it does cause minor differences in output on the Jetson Nano. This is due to the internal parameters of the instance normalisation layer learning the statistical properties it is exposed to during training. A deviation in the computation of the input causes a shift in the statistical properties of the input, and this error can propagate throughout the network.

The choice to use a direct method for onset detection, rather than computing the SuperFlux algorithm in parallel, proved to be the better choice. The additional computational load and wait time introduced by the computation of the input features of the SuperFlux algorithm would have caused Qualification Test 4 to fail. In hindsight, had the recursive downsampling method of the CQT been implemented, it is possible that the reduction in wait time provided could have provided a sufficient time gap to include the SuperFlux algorithm.

The performance optimisations being loop tiling, vectorised operations and caching, employed to speed up the convolution function and the CQT function, are crucial in passing Qualification Test 4.

A limiting factor of the onset detection system is that the performance of the subsystem is

coupled to the performance of the activity detector. This is evident from the low precision, high recall trend being constant in both subsystems. Whilst being the more computationally feasible approach, it does constrain the performance of the final output of the system.

From a technical standpoint, the project was both rigorous and feasible. The software needed for deployment on an embedded platform was verified and implemented within the constrained timeframe of EPR402. From an economic standpoint, the cost of the Jetson Nano was within the allocated budget of EPR402. It proved to be a powerful platform for embedded-related tasks.

5.1.3 Unsolved problems

The most critical unsolved problem is the domain mismatch between the studio-quality data and the re-recorded data. The analysis of the frequency response of the speaker microphone system in section 3.10 indicates that significant non-linear distortion is introduced into the system, which alters the acoustic characteristics of the input audio. The operation of the application is tied to the domain of the speaker microphone system and varies with the recording conditions.

The application is also constrained by the data imbalance and data scarcity of the compiled dataset. The lack of testing samples for trumpet and contrabass raises concerns about the confidence of these results. The poor performance of the flute is a result of poor diversification of training samples and an uneven ensemble distribution. The co-occurrence of instruments within the same instrument family is a driver of the low precision scores. Acquiring well-annotated data that matches the scope of the project is not an easy task. Solving this problem would involve recording classical music and acquiring the assistance of musical experts to annotate the data at the note level.

The model interpretability remains a problem. While the analysis conducted provides several plausible hypotheses as to why it makes certain decisions, it cannot be said with full certainty. The CNN can be viewed as a black box element, and it would be difficult to pinpoint root-cause issues for future improvement.

5.1.4 Strong points of the design

A key strength is the use of the multi-head architecture as a direct response to instrument co-occurrence biases and to provide a means of positive learning. Whilst the effect of instrument co-occurrence biases can be seen in the final results of the application, this architectural decision does reduce the impact of such a data imbalance. Having specialised heads with configurable dropout rates provided the CNN with the best chance at generalising for an instrument family. The design choice to intentionally refrain from downsampling the time dimension of the input and intermediate feature maps allows the CNN to make decisions based on the downsampling of the frequency dimension.

The use of the HCQT as the feature representation method is also considered a strength of the application, as it provides a time-frequency representation that aligns with the Western musical scale and captures the harmonic structure of instruments. The harmonic structure of instruments is key to learning the timbral elements of an instrument.

The onset detection logic is motivated by theory, and also includes configurable parameters

to learn the ADSR envelope of each instrument. The strength lies in the custom thresholding logic per instrument and the filtering logic to remove spurious moments of confidence. This approach is sensitive to the flux of the model probabilities for each instrument, and complements the decision to use a frame size of 11.61 ms. Such a fine temporal resolution allows the onset detection system to capture the transients of the attack phase, which is key to determining the onset of an instrument.

5.1.5 Expected failure conditions

Despite not passing Qualification Tests 1 and 2, the application's performance is sufficient to say that it does solve the broader problem of the project's scope. The degree to which the application fails to meet specifications 1 and 2 is not significant enough to say that it cannot detect an instrument onset if placed within a live theatre performance.

Considering this, the application is a specialist tool as it may only operate in the recording domain of the speaker-microphone system. Performance would degrade drastically if recording conditions are altered. If the speaker produces poor-quality audio textures, the computed HCQT would not be a true representation of the instrument timbre. If the microphone is placed in an orientation whereby the audio signal is not in the microphone's field of view, the computed HCQT would contain little to no information.

The CNN was trained solely on Western classical chamber music on a predefined set of 11 instruments. It is expected that the application would fail if presented with other genres, production styles or instruments outside the project scope. The flute displays the lowest F1-score of all instruments, with a score of 0.333. In a scenario where the detection of a flute is paramount, it would likely fail to detect the flute. Dataset analysis was limited to the distribution of instrument co-occurrence and the ensemble distribution - no analysis was conducted on the distribution of playing styles. If presented with an unconventional playing style, such as col legno or pizzicato, it is expected that the application would produce ICFPs.

In such cases, the system is application is designed to fail gracefully. The failure would be more an output of incorrect instrument onsets, not a software crash or the application switching off.

5.2 Considerations in the design

5.2.1 Design ergonomics

All key parameters are contained in a single configuration file instead of being placed in the source code directly - this improves the usability for a technical user. Output is produced in a standardised manner which is machine-readable. This output can be interpreted by a GUI that is connected, and for future research. The output log of the C implementation is verbose and gives detailed feedback on which stage of processing the application is at. There is a high degree of modularity incorporated in the project structure. This allows for a professional software engineering workflow.

5.2.2 Health and safety

The design of the application does not pose significant health risks, aside from those typical to the usage of consumer electronics, such as shocks or overheating components. That considered, the chosen components comply with national and international safety standards for electrical equipment, including ICASA. The application's design is primarily software; thus, the code itself poses no danger. It is possible that the Jetson Nano may overheat under significant computational load; however, there is a passive heat sink used to ensure that it operates within safe thermal limits. Another potential health consideration is the loudness of the system's audio - this is a setting defined by the user, not configured by the application itself.

5.2.3 Environmental impact

The environmental impact of the application is minimal. The application runs on the NVIDIA Jetson Nano, which is a low-power single-board computer that consumes 5-10 watts when compared to a traditional PC. The software optimisations promote energy efficiency by placing less strain on the hardware.

5.2.4 Social and legal impact

Legal considerations around the use of data and the associated copyright laws are of concern. Training was done on datasets which contain permissive academic licenses; however, it is the responsibility of the end user to ensure that the audio used as input during deployment does not infringe on any copyright laws.

The potential social impact can be realised in fields of musicology and music education as the application automates a highly cumbersome task of identifying instrument onsets within polyphonic mixes. This opens the door for researchers to conduct large-scale analyses of historical archives.

5.2.5 Ethics clearance

Ethics clearance was not needed.

6. Conclusion

6.1 Summary of the work completed

The design, implementation and evaluation of an application for instrument detection in polyphonic recordings that is to be deployed onto an embedded platform has been completed. The project scope included the detection of 11 instruments within classical chamber ensembles, ranging from duets to quintets. After a comprehensive literature review, a deep learning-based approach was used. An investigation to determine the optimal combination of feature representation and model architecture was conducted: the candidate representations were the mel spectrogram, MFCCs and HCQT, and the candidate CNN architectures were ResNet and VGG. Both the CNN and feature representation were implemented from first principles in C. A custom onset detection system using probability flux analysis was developed based on the musical theory of instrument onsets.

6.2 Summary of observations and findings

The application achieved a macro F1-Score of 0.754 for instrument activity detection and a macro F1-Score of 0.743 for instrument onset detection. A key finding was that the application's limitation is not in the ability to detect an instrument onset, but in its tendency to generate false positives. The cause for these false positives is rooted in the characteristics of the compiled dataset, which contained extreme co-occurrence among woodwind instruments and string instruments. The system also achieved an average processing factor of 1.81, well below the 2x specification. It was found that the bottleneck in the processing time was the computation of the HCQT rather than the forward pass of the CNN.

6.3 Contribution

The primary contribution is the design of a bespoke VGG-style CNN that incorporates a multi-head architecture. Each head is designed to produce predictions per instrument family for every 11.61 ms of audio. A first principles implementation of the HCQT, model forward pass, an onset detection subsystem and AdamW optimiser were implemented from first principles. Theory relating to Bayesian optimisation, regularisation techniques and learning rate scheduling is also presented.

6.4 Suggestions for future work

Future work should address the domain mismatch and focus on a means of making the application invariant to the recording domain. The accuracy of the onset detection subsystem could be improved by incorporating a combination of either Mel spectrogram, MFCC and the SuperFlux algorithm. Lastly, the dataset scarcity and imbalance should be resolved by acquiring real-world data with the assistance of experts.

7. References

[1] M. Müller, *Fundamentals of Music Processing: Using Python and Jupyter Notebooks*, 2nd ed., ser. Springer Textbooks in Signal and Image Processing. Springer, 2021.

[2] D. D. Lee and H. S. Seung, "Learning the parts of objects by non-negative matrix factorization," *Nature*, vol. 401, pp. 788–791, 1999.

[3] D. A. Reynolds, *Gaussian Mixture Models*. Springer, 2009.

[4] L. R. Rabiner, *A tutorial on hidden Markov models and selected applications in speech recognition*. IEEE, 1989, vol. 77, no. 2.

[5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[6] N. Ahmed, T. Natarajan, and K. R. Rao, "Discrete cosine transform," *IEEE Transactions on Computers*, vol. C-23, no. 1, pp. 90–93, 1974.

[7] M. Krause and M. Müller, "Hierarchical classification for instrument activity detection in orchestral music recordings," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2567–2578, 2023.

[8] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2015. [Online]. Available: https://arxiv.org/abs/1409.1556

[9] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] J. J. Bosch, J. Janer, F. Fuhrmann, and P. Herrera, "A comparison of sound segregation techniques for predominant instrument recognition in musical audio signals," in *13th International Society for Music Information Retrieval Conference (ISMIR)*. FEUP Edições, 2012, pp. 559–564.

[11] E. J. Humphrey, S. Durand, and B. McFee, "Openmic-2018: An open dataset for multiple instrument recognition," in *Proceedings of the 19th International Society for Music Information Retrieval Conference (ISMIR)*, 2018, pp. 583–590.

[12] M. Goto, H. Hashiguchi, T. Nishimura, and R. Oka, "RWC music database: Popular, classical, and jazz music databases," in *Proceedings of the 3rd International Society for Music Information Retrieval Conference (ISMIR)*, 2002, pp. 287–288.

[13] R. M. Bittner, M. Cartwright, C. Cella, J. Salamon, E. J. Humphrey, and J. P. Bello, "Medleydb 2.0: A multitrack dataset for annotation-intensive mir research," in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, 2017, pp. 592–599.

[14] J. Thickstun, Z. Harchaoui, and S. M. Kakade, "Learning features of music from scratch," in *International Conference on Learning Representations (ICLR)*, 2017.

[15] B. Li, X. Liu, K. Dinesh, Z. Duan, and G. Sharma, "Creating a multitrack classical music performance dataset for multimodal music analysis: Challenges, insights, and applications," *IEEE Transactions on Multimedia*, vol. 21, no. 2, pp. 522–535, 2018.

[16] J. C. Brown, “Calculation of a constant Q spectral transform,” *The Journal of the Acoustical Society of America*, vol. 89, no. 1, pp. 425–434, 1991.

[17] C. Schörkhuber and A. Klapuri, “Constant-q transform toolbox for music processing,” in *Proceedings of the 13th International Conference on Digital Audio Effects (DAFx-10)*, Graz, Austria, 2010.

[18] Z. K. Abdul and A. K. Al-Talabani, “Mel frequency cepstral coefficient and its applications: A review,” *IEEE Access*, vol. 10, pp. 122 136–122 158, 2022.

[19] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *CoRR*, vol. abs/1512.03385, 2015. [Online]. Available: http://arxiv.org/abs/1512.03385

[20] ——, “Identity mappings in deep residual networks,” *CoRR*, vol. abs/1603.05027, 2016. [Online]. Available: http://arxiv.org/abs/1603.05027

[21] S. Xie, R. B. Girshick, P. Dollár, Z. Tu, and K. He, “Aggregated residual transformations for deep neural networks,” *CoRR*, vol. abs/1611.05431, 2016. [Online]. Available: http://arxiv.org/abs/1611.05431

[22] G. Huang, Z. Liu, and K. Q. Weinberger, “Densely connected convolutional networks,” *CoRR*, vol. abs/1608.06993, 2016. [Online]. Available: http://arxiv.org/abs/1608.06993

[23] R. Wightman, “PyTorch Image Models,” https://github.com/rwightman/pytorch-image-models, 2019.

[24] K. Koutini, H. Eghbal-zadeh, and G. Widmer, “Receptive field regularization techniques for audio classification and tagging with deep convolutional neural networks,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 29, pp. 3041–3055, 2021.

[25] MuseScore, “Musescore,” https://musescore.org, 2025, accessed: 2025-11-03.

[26] University of York, “Audio lab,” https://www.openair.hosted.york.ac.uk/?page_id=36, 2025, accessed: 2025-11-03.

[27] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[28] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal loss for dense object detection,” in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[29] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” *Journal of Machine Learning Research*, vol. 12, pp. 2121–2159, 2011.

[30] G. Hinton, N. Srivastava, and K. Swersky, “Lecture 6.5e: Rmsprop,” COURSERA: Neural Networks for Machine Learning, 2012, uRL: http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.

[31] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: http://arxiv.org/abs/1412.6980

[32] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," *CoRR*, vol. abs/1711.05101, 2017. [Online]. Available: http://arxiv.org/abs/1711.05101

[33] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutsever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[34] P. I. Frazier, "A tutorial on bayesian optimization," *arXiv preprint arXiv:1807.02811*, 2018.

[35] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyper-parameter optimization," in *Proceedings of the 24th International Conference on Neural Information Processing Systems (NIPS'11)*. Red Hook, NY, USA: Curran Associates Inc., 2011, pp. 2546–2554.

[36] S. Böck and G. Widmer, "Maximum filter vibrato suppression for onset detection," in *Proceedings of the 16th International Conference on Digital Audio Effects (DAFx-13)*, Maynooth, Ireland, September 2013.

[37] J. Mulcahy, "Room EQ Wizard (REW)," https://www.roomeqwizard.com/, 2025, version 5.20. Accessed: 2025-11-03.

[38] A. Wood, *The Physics of Music*, 7th ed. Chapman and Hall, 1975, revised by J. M. Bowsher.

Appendix

Ensemble definitions

Solo: Any mix which contains exactly 1 unique instrument.

Brass duo: Any mix which contains exactly 2 instruments from the Brass family.

String duo: Any mix which contains exactly 2 instruments from the Strings family.

Woodwind duo: Any mix which contains exactly 2 instruments from the Woodwinds family.

Duet (Mixed): Any other mix which contains exactly 2 instruments.

Piano trio: Any mix which contains exactly 3 instruments, specifically piano and 2 instruments from the Strings family.

String trio: Any mix which contains exactly 3 instruments from the strings family.

Woodwind trio: Any mix which contains exactly 3 instruments, all of which are from the woodwinds family.

Trio (Mixed): Any other mix which contains exactly 3 instruments.

Piano quartet: Any mix which contains exactly 4 instruments, specifically piano and 3 instruments from the strings family.

String quartet: Any mix which contains exactly 4 instruments from the strings family.

Woodwind quartet: Any mix which contains exactly 4 instruments, all of which are from the woodwinds family.

Quartet (Mixed): Any other mix which contains exactly 4 instruments.

Piano quintet: Any mix which contains exactly 5 instruments, specifically piano and 4 instruments from the strings family.

Wind quintet: Any mix which contains exactly the following 5 instruments: flute, oboe, clarinet, bassoon, and French horn.

Quintet (Mixed): Any other mix which contains exactly 5 instruments.

Octet: Any mix which contains exactly 8 unique instruments.

Dataset statistics

Table 25. Instrument duration statistics.

Instrument	Total (mm:ss)	Train (%)	Val (%)	Test (%)
Bassoon	73:13	71.4	15.0	13.6
Cello	290:23	70.2	15.0	14.8
Clarinet	125:19	70.4	14.5	15.1
Contrabass	23:38	70.1	13.8	16.2
Flute	53:28	70.1	14.8	15.0
French Horn	74:14	70.1	14.6	15.3
Oboe	48:51	66.6	15.9	17.6
Piano	319:58	70.1	14.9	15.0
Trumpet	23:09	70.9	16.8	12.4
Viola	260:04	69.3	15.6	15.1
Violin	374:32	70.3	14.9	14.8

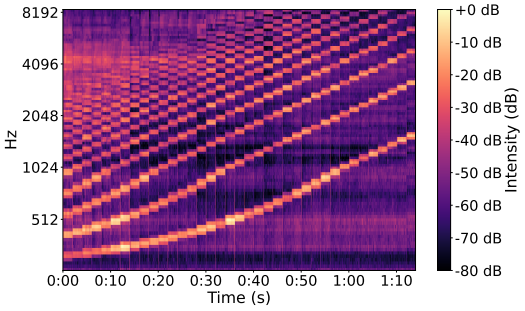
Table 26. Ensemble statistics.

Ensemble	Files	Train (%)	Val (%)	Test (%)
Solo	15	100.0	0.0	0.0
Duet	34	85.3	5.9	8.8
Trio	41	80.5	9.8	9.8
Quartet	18	72.2	16.7	11.1
Quintet	13	38.5	30.8	30.8
Octet	2	100.0	0.0	0.0

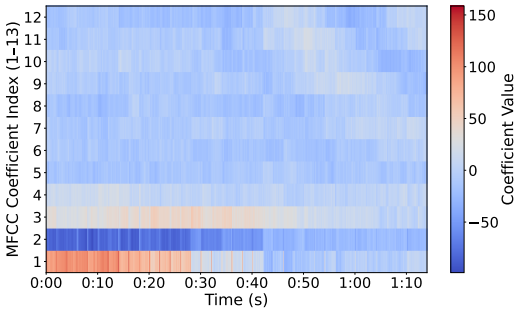
Table 27. Distribution of instrument onsets across validation and testing sets.

Instrument	Validation	Testing	Total
Piano	6	7	13
Violin	8	7	15
Cello	7	6	13
Viola	5	5	10
Contrabass	1	1	2
Clarinet	6	5	11
Bassoon	4	3	7
Oboe	3	3	6
Flute	4	3	7
French Horn	5	5	10
Trumpet	2	1	3
Total Onsets	**51**	**46**	**97**

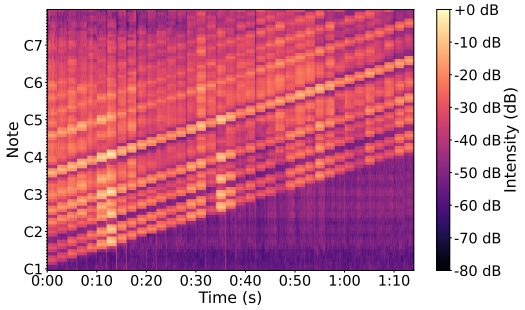
Instrument feature representations



(a) Mel-spectrogram of violin note sweep.

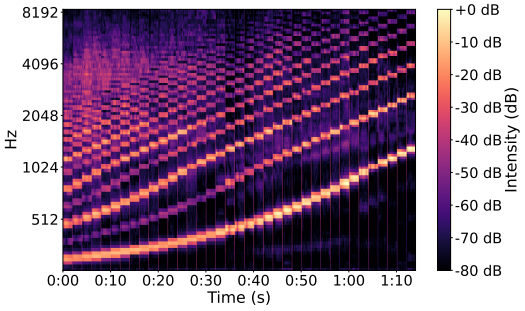


(b) MFCCs of violin note sweep.

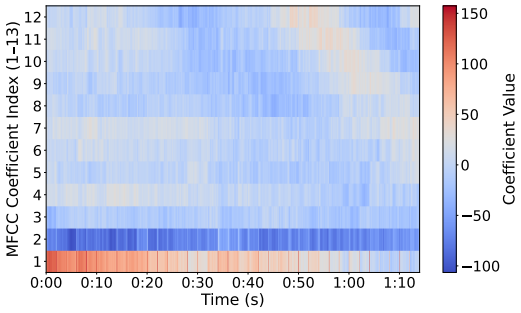


(c) HCQT of violin note sweep.

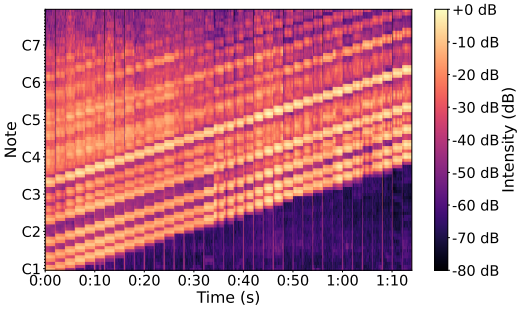
Figure 33. Feature representation of violin note sweep .



(a) Mel-spectrogram of clarinet note sweep.

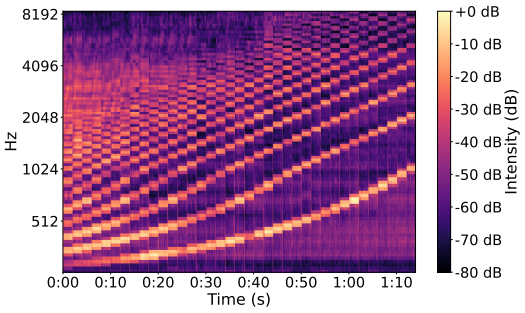


(b) MFCCs of clarinet note sweep.

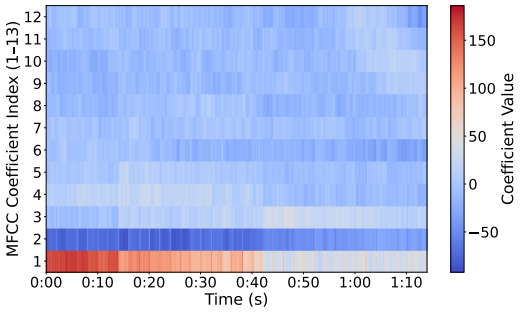


(c) HCQT of clarinet note sweep.

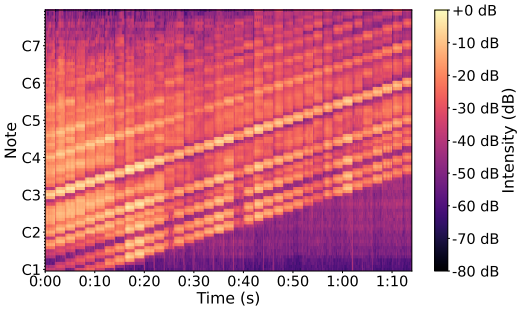
Figure 34. Feature representation of clarinet note sweep .



(a) Mel-spectrogram of viola note sweep.

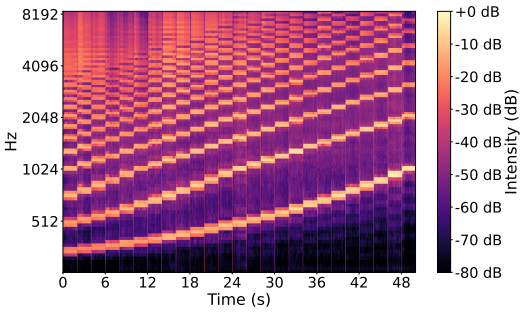


(b) MFCCs of viola note sweep.

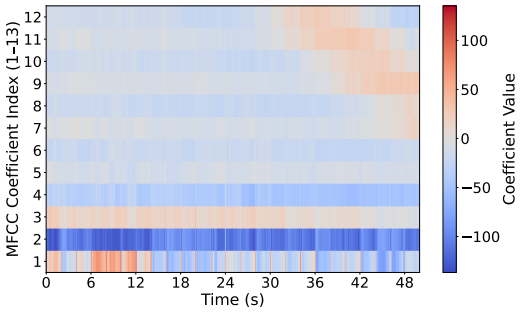


(c) HCQT of viola note sweep.

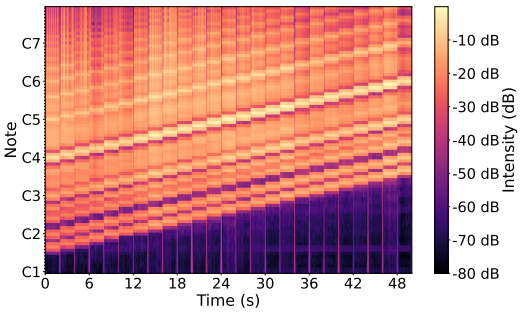
Figure 35. Feature representation of viola note sweep .



(a) Mel-spectrogram of trumpet note sweep.

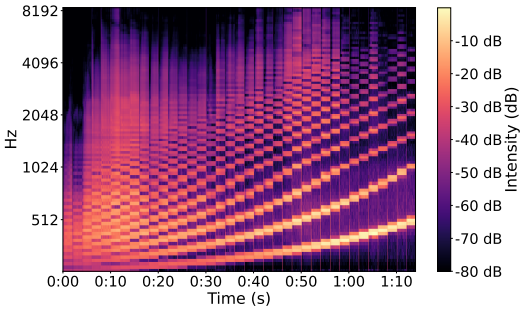


(b) MFCCs of trumpet note sweep.

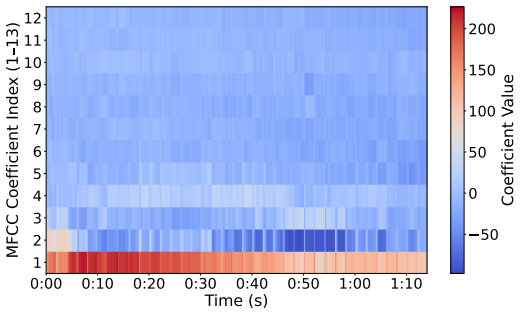


(c) HCQT of trumpet note sweep.

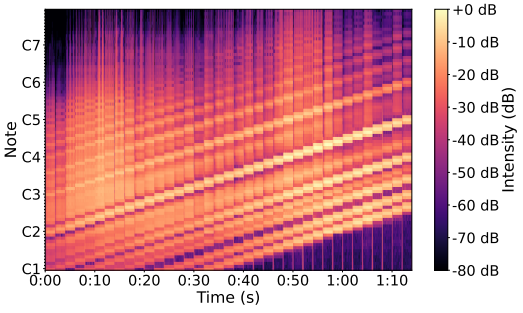
Figure 36. Feature representation of trumpet note sweep .



(a) Mel-spectrogram of French horn note sweep.

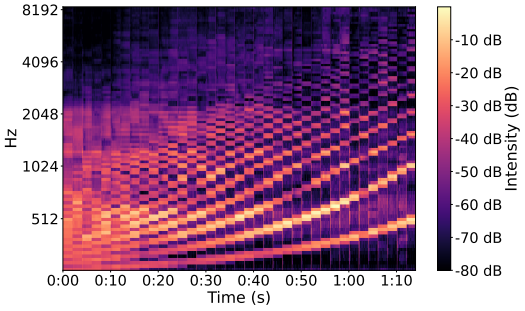


(b) MFCCs of French horn note sweep.

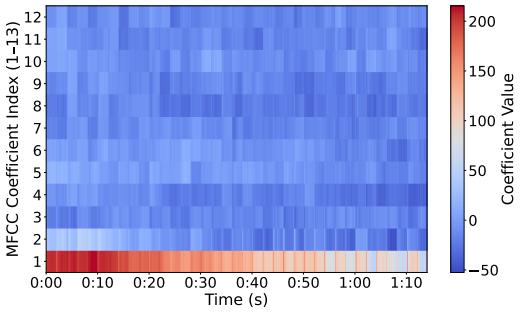


(c) HCQT of French horn note sweep.

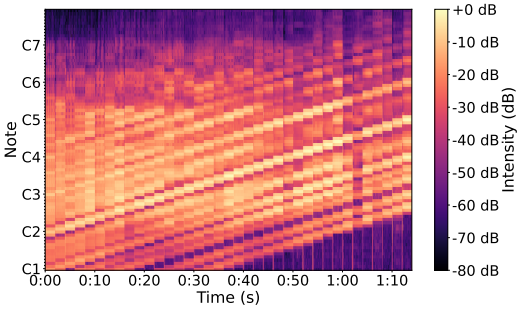
Figure 37. Feature representation of French horn note sweep.



(a) Mel-spectrogram of bassoon note sweep.

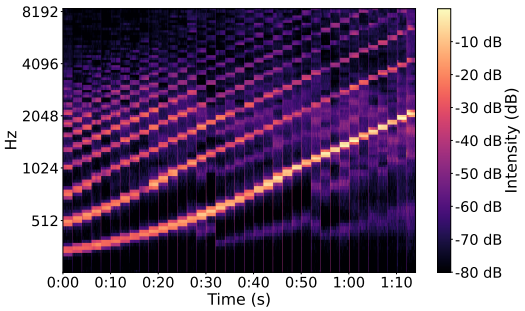


(b) MFCCs of bassoon note sweep.

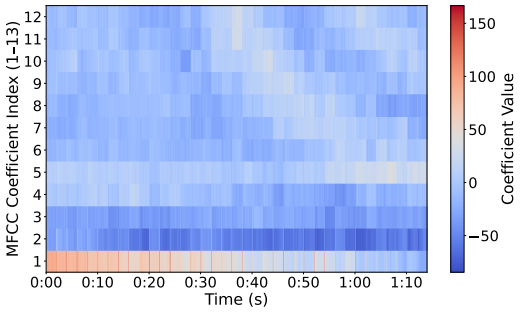


(c) HCQT of bassoon note sweep.

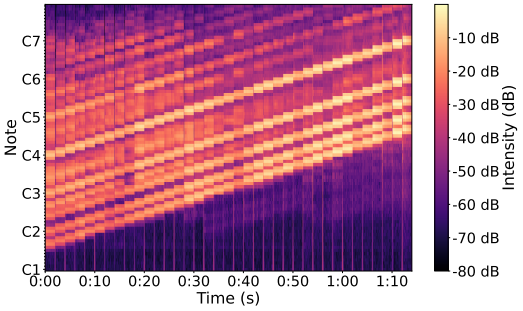
Figure 38. Feature representation of bassoon note sweep.



(a) Mel-spectrogram of flute note sweep.

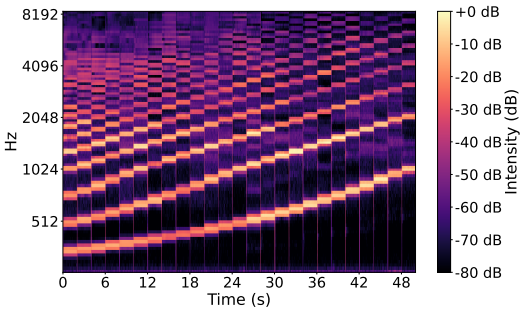


(b) MFCCs of flute note sweep.

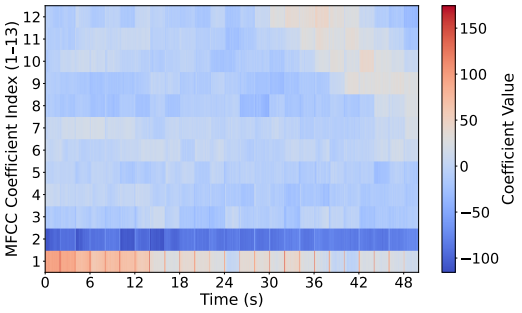


(c) HCQT of flute note sweep.

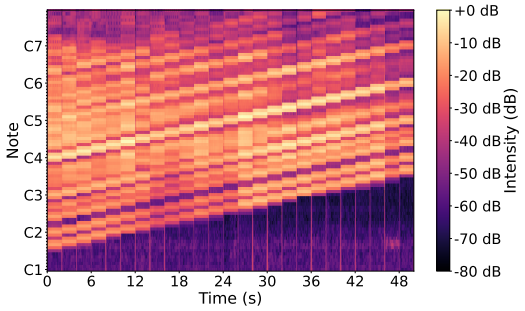
Figure 39. Feature representation of flute note sweep.



(a) Mel-spectrogram of oboe note sweep.

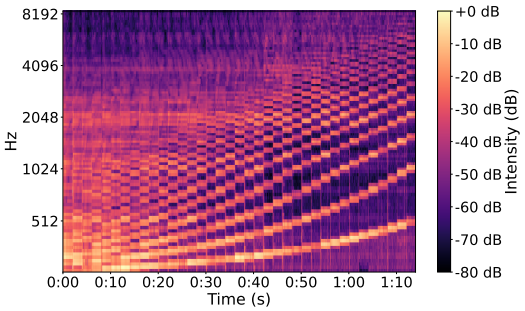


(b) MFCCs of oboe note sweep.

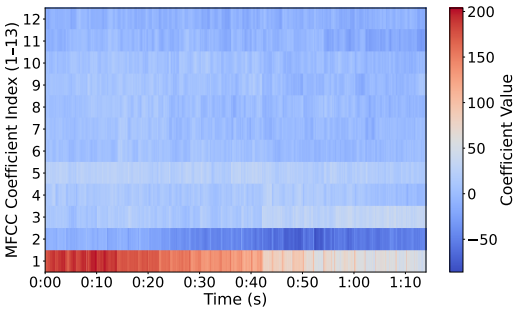


(c) HCQT of oboe note sweep.

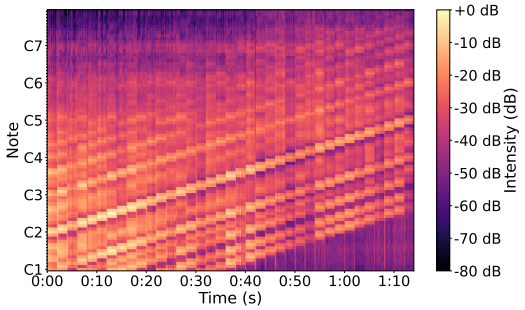
Figure 40. Feature representation of oboe note sweep.



(a) Mel-spectrogram of cello note sweep.

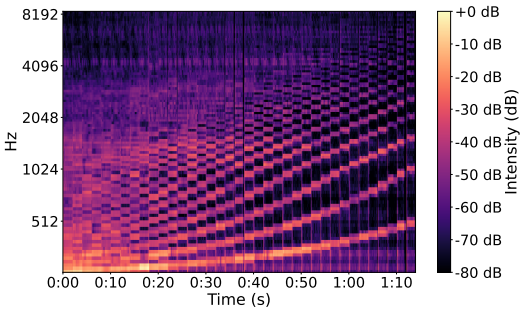


(b) MFCCs of cello note sweep.

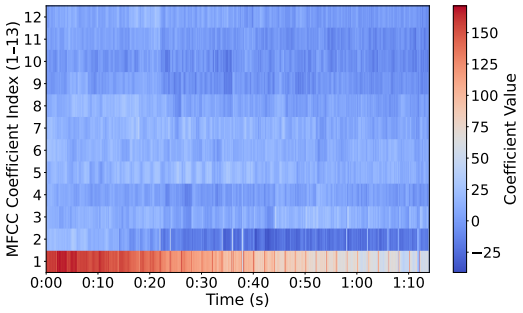


(c) HCQT of cello note sweep.

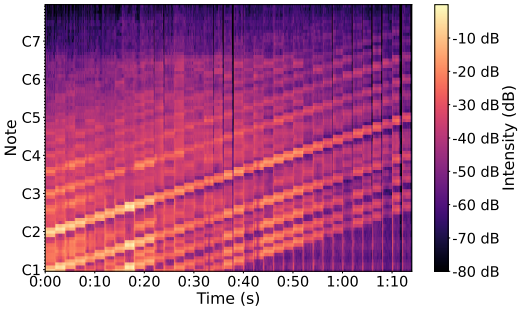
Figure 41. Feature representation of cello note sweep.



(a) Mel-spectrogram of contrabass note sweep.



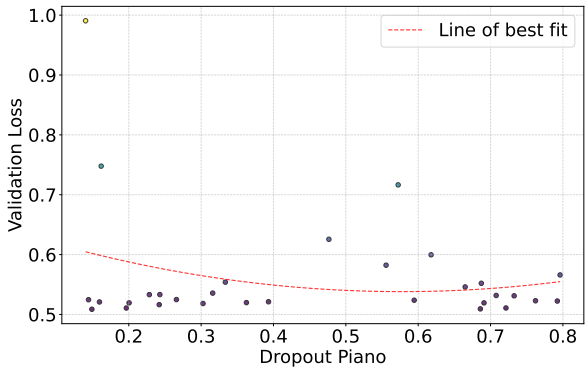
(b) MFCCs of contrabass note sweep.



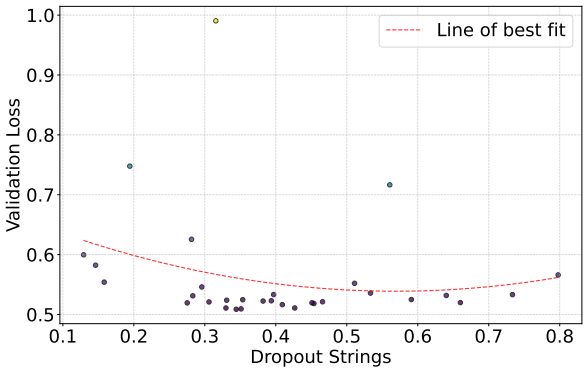
(c) HCQT of contrabass note sweep.

Figure 42. Feature representation of contrabass note sweep.

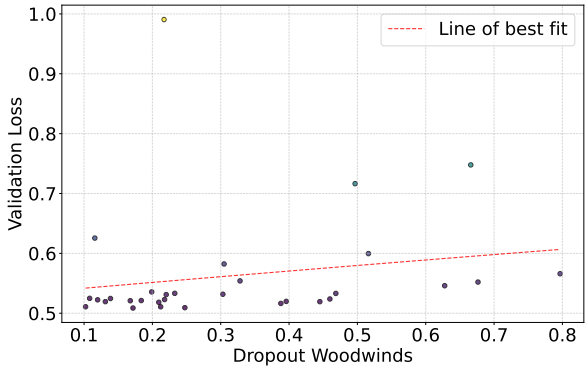
Hyperparameter optimisation



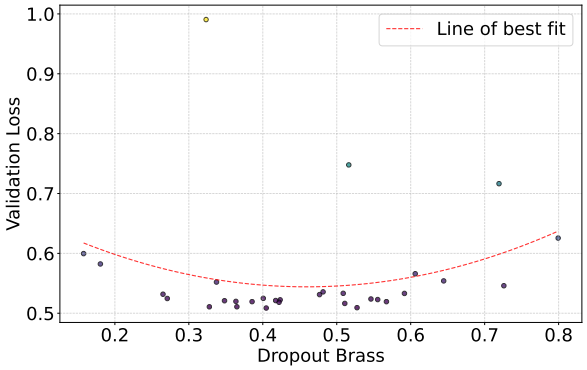
(a) Graph of `dropout_piano` vs. validation loss.



(b) Graph of `dropout_strings` vs. validation loss.



(c) Graph of `dropout_woodwinds` vs. validation loss.



(d) Graph of `dropout_brass` vs. validation loss.

Figure 43. Dropout parameters vs. Validation loss.